

SYSTEM DESIGN SPECIFICATION

Real-Time Fraud & Anomaly Detection Platform

A streaming, explainable, graph-aware fraud detection system with human-in-the-loop review and full MLOps + LLMOps observability.

KAFKA

DATABRICKS

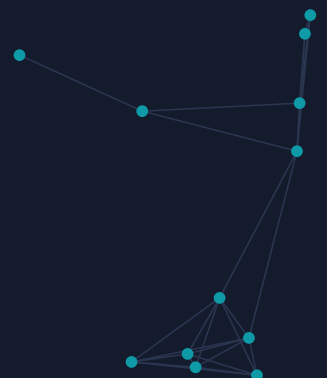
MONGODB

STREAMLIT

LANGCHAIN

LANGSMITH

Prepared for: Mohammed Shadab Ameen · Edison Internship, GE HealthCare
Version 1.0 · Portfolio / System-Design Review Edition



00

Contents

#	Section	What it covers
01	Executive Summary	Problem, value, and what makes it stand out
02	Objectives & Requirements	Functional + non-functional (latency, throughput, reliability)
03	System Architecture	Hot path / cold path / analyst-assist — full diagram
04	Component Deep-Dive	Producers, Kafka, Databricks, MongoDB, Streamlit
05	Machine Learning Layer	Models, MLflow, thresholds, imbalance strategy
06	Graph-Based Ring Detection	Neptune/Neo4j, community detection, graph features
07	LLM Analyst-Assist Layer	LangChain chains, RAG, investigation agent
08	LLMOps with LangSmith	Tracing, evaluation, monitoring, feedback
09	Data Engineering Backbone	Delta medallion, Feature Store, orchestration, quality
10	End-to-End Data Flow	The 8-step story, step by step
11	Data Models & Schemas	Kafka payloads + MongoDB collections
12	Trade-offs & Design Decisions	What to defend in a review
13	Security & Reliability	PII, adversarial, DLQ, exactly-once
14	Build Plan & Milestones	Phased 4-week (and compressed 2-week) plan
15	Tech Stack & Success Metrics	Full stack table + KPIs

01

Executive Summary

This document specifies a **real-time fraud and anomaly detection platform** that ingests a high-velocity transaction stream, scores every event in near real-time, flags suspicious activity for human review, and continuously improves itself from analyst feedback. It is engineered as a portfolio-grade system-design showcase: it demonstrates streaming architecture, applied machine learning, graph analytics, retrieval-augmented LLM tooling, and disciplined MLOps/LLMOps.

The design is deliberately split into a **hot path** that makes fast, deterministic scoring decisions and a **cold path** that stores everything durably and periodically retrains the model. Large-language-model reasoning is kept strictly **off the hot path** — it powers an on-demand "AI fraud analyst" that explains why a transaction was flagged, grounded in real feature attributions and precedent cases.

What makes this project stand out

- **Hybrid detection:** a per-transaction ML model *plus* a graph layer that catches fraud *rings* a point-model cannot see.
- **Explainable by design:** SHAP feature attributions feed a LangChain summariser, so every alert ships with a plain-English, cited rationale.
- **Two observability planes:** MLflow watches the scoring model; LangSmith watches the LLM assistant — a mature, production-credible story.
- **Closed feedback loop:** analyst decisions retrain the model and grow an LLM evaluation set, so both systems get better over time.
- **Cost- and latency-aware:** autoscaling streaming clusters, on-demand LLM calls, and a live cost/SLA dashboard.

The one-line pitch

"An adaptive, explainable, graph-aware fraud-detection system with proper MLOps and LLMOps" — not just a streaming pipeline.

02

Objectives & Requirements

Functional requirements

- Ingest a continuous stream of transaction events without loss.
- Enrich each event with rolling behavioural features and graph context, then score it for fraud risk.
- Assign a risk band (low / medium / high) and persist decisions to a low-latency serving store.
- Surface flagged transactions on a live dashboard with a human review workflow.
- Generate a grounded, natural-language explanation for any flagged case on demand.
- Retrain and promote improved models from accumulated analyst labels — without redeploying the stream.

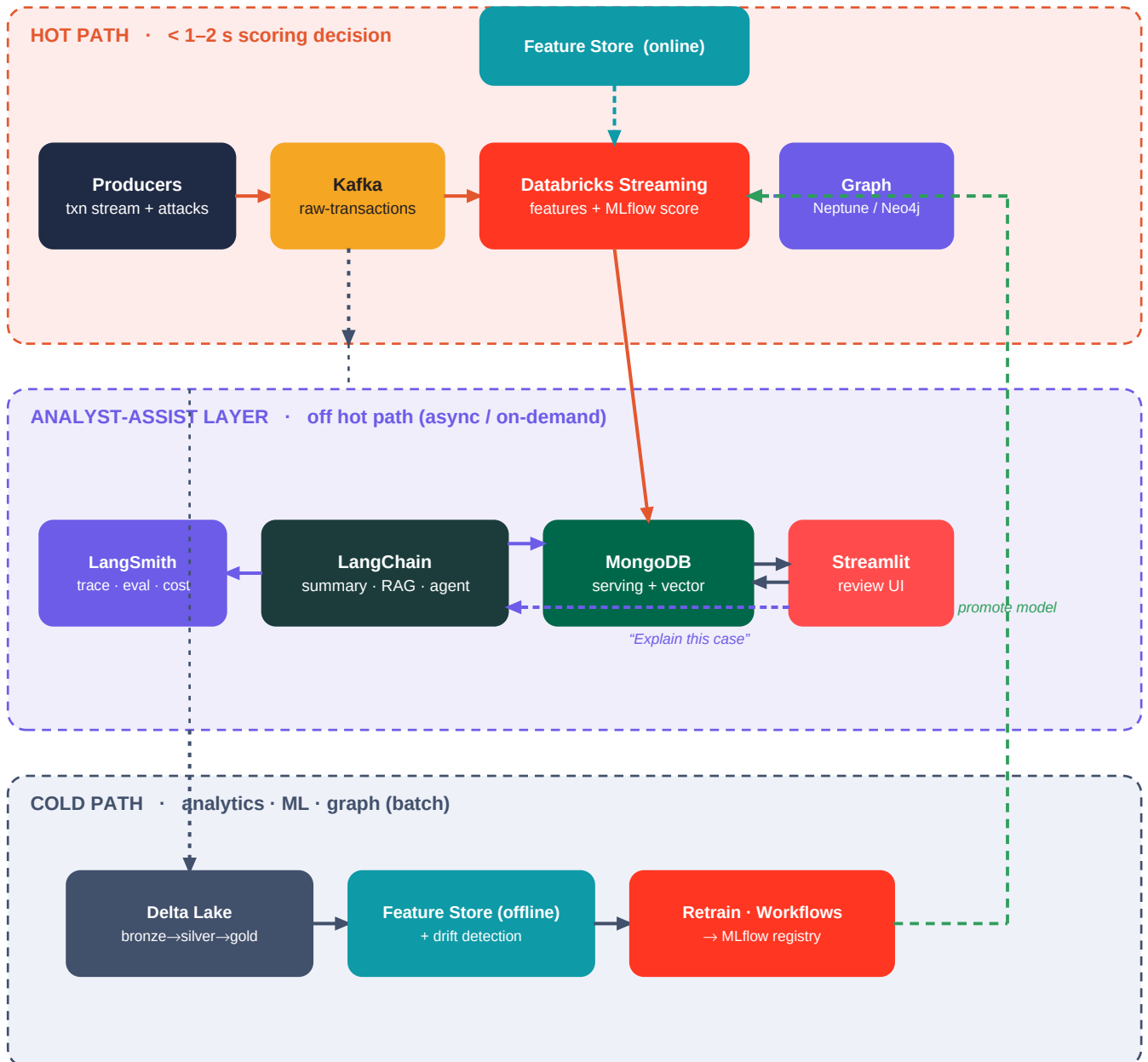
Non-functional requirements

Attribute	Target	How the design meets it
Latency	< 1–2 s per event (hot path)	Stateful streaming; model held in-memory; LLM kept off hot path
Throughput	5k–50k events/sec, bursty	Kafka buffering + partitioning by user_id; autoscaling clusters
Reliability	No lost transactions	At-least-once ingest + idempotent upserts keyed on txn_id
Auditability	Every decision reproducible	Raw + enriched history in Delta; full model/feature lineage
Adaptability	Models never go stale silently	Drift detection triggers retraining; challenger shadow deploys
Cost	Predictable, observable	Autoscaling job clusters; on-demand LLM; live cost dashboard

03

System Architecture

The platform is organised into three planes. The **hot path** (top) turns raw events into scored decisions in under two seconds. The **analyst-assist layer** (middle) runs LLM reasoning on demand, off the critical path. The **cold path** (bottom) is the durable analytics and retraining backbone. Green dashed arrows show the feedback loops that make the system adaptive.



Reading the diagram: solid coral arrows are the low-latency scoring path; purple dashed arrows are on-demand LLM/graph lookups; grey dashed arrows feed the cold path; the green dashed arrow is the model-promotion loop back into the live stream.

Design principle #1 — separate hot and cold

The single most important idea to articulate in a review: fast decisions and heavy learning have different SLAs, so they run on different paths against different stores (MongoDB for serving, Delta Lake as the source of truth for training).

04

Component Deep-Dive

Producers — the event source

A transaction service (or a synthetic generator for the demo) emits events with fields such as `txn_id`, `user_id`, `amount`, `merchant`, `merchant_category`, `geo`, `device_id`, `ip`, `timestamp`. The generator injects a controllable fraud ratio (~2%) with realistic patterns — velocity spikes, geo-impossible travel, card-testing bursts — so the dashboard always has something meaningful to show. Seed realism with the Kaggle Credit-Card-Fraud dataset or the PaySim mobile-money simulation.

Apache Kafka — ingestion & buffering

- Topics: `raw-transactions` (input), optionally `scored-transactions` and `alerts`.
- **Partition by `user_id`** so a user's events stay ordered on one partition — essential for velocity features.
- Acts as a shock absorber: decouples bursty producers from processing and provides replayability.
- Design talking points: partition count vs consumer parallelism, `acks=all` durability, offset management, a dead-letter topic for poison messages.

Databricks (Structured Streaming) — the brain

Reads Kafka via the native source and performs **stateful stream processing** with watermarking to compute rolling features per user: transaction counts over 1/5/60-minute windows, rolling mean and standard deviation of amount, distinct merchants and geos, and time since last transaction. It pulls the current model from the MLflow registry, scores each event, then writes decisions to MongoDB (serving) and appends raw + scored rows to Delta Lake (history).

- Design talking points: exactly-once vs at-least-once semantics, watermarking & late data, checkpointing for fault tolerance, state TTL to bound memory.

MongoDB — the serving layer

Enriched transactions are naturally document-shaped (nested feature objects, variable merchant metadata) and the dashboard needs fast reads — a strong, defensible fit for MongoDB.

Collection	Purpose	Key / index
<code>flagged_transactions</code>	Full enriched doc + score + risk band	<code>_id = txn_id</code> ; idx on <code>user_id</code> , <code>risk_band</code> , <code>ts</code>
<code>user_rolling_state</code>	Latest aggregates per user (idempotent upsert)	<code>_id = user_id</code>
<code>alerts</code>	Open/closed alert lifecycle for review	idx on <code>status</code> , <code>priority</code>
<code>case_embeddings</code>	Vectors of resolved cases (Atlas Vector Search)	vector index (cosine)

Streamlit — UI & operations console

- Live alerts feed (auto-refresh) with geo-map and score-distribution charts.
- Review panel: analyst marks fraud / not-fraud, writing a label back to Mongo that feeds retraining.
- "Explain this case" button that invokes the LangChain assistant on demand.
- KPIs: flag rate, alerts/hour, model version in use, and live p50/p95/p99 scoring latency.

05

Machine Learning Layer

Fraud labels are scarce and highly imbalanced, so the model strategy evolves in stages rather than assuming clean supervision from day one.

Stage	Model	When / why
Bootstrap	Isolation Forest / Autoencoder	Unsupervised; works before you have labels
Mature	XGBoost / LightGBM	Supervised, once review labels accumulate
Augmented	+ graph features (see §06)	Ring signals lift recall on coordinated fraud

MLflow — experiment tracking & registry

- Tracks experiments, parameters, and metrics; versions every model.
- The stream loads the model tagged **Production** via the registry — promotion happens without a redeploy.
- Enables challenger / shadow evaluation before a model is promoted.

Thresholds & imbalance strategy

- Map raw score → risk band: **high** = auto-flag; **medium** = queue for human review; **low** = pass.
- Optimise for **recall on the fraud class** while watching false-positive fatigue for analysts.
- Rank alerts by **expected loss** (score × amount) so analysts work the highest-value cases first.

Design principle #2 — plan for drift

Fraud is adversarial: attackers adapt, so models decay fast. Monitor feature drift (population stability index) and score-distribution drift, and trigger retraining when thresholds are crossed — don't rely on a nightly cron alone.

06

Graph-Based Ring Detection

Fraud is rarely a lone transaction — it is organised into **rings**: accounts sharing devices, IPs, or billing addresses, and money mules moving funds in chains. A per-transaction model is blind to these relationships. Adding a graph layer (AWS Neptune or Neo4j) makes them visible.

Model & techniques

- Model relationships: User → Device → Merchant → Card / IP → Address.
- Run **community detection** and **connected-components** to surface suspicious clusters.
- Compute **graph features** — e.g. "how many flagged users share this device?" — and feed them back into the scoring model as additional inputs.

Why this is the standout addition

It converts a standard streaming project into a hybrid graph + ML system — the single most impressive differentiator, and a natural showcase for knowledge-graph expertise.

07

LLM Analyst-Assist Layer (LangChain)

Golden rule — keep LLMs off the hot path

A per-transaction scoring decision must never block on an LLM call. LangChain and LangSmith live in the analyst-assist layer, triggered AFTER a transaction is flagged, on demand.

Three concrete LangChain uses

1 · Case-summary chain

Takes the flagged transaction + its SHAP feature attributions + the user's recent history and produces a plain-English rationale — e.g. "amount 8× the user's average, a new geo, and 4 transactions in 60 seconds." This is the LLM realisation of the explainability requirement.

2 · RAG over fraud playbooks & past cases

A retrieval chain queries **MongoDB Atlas Vector Search** over embeddings of resolved fraud cases and internal fraud-policy documents, so each summary is grounded in precedent: "this pattern matches three confirmed card-testing cases from last month."

3 · Investigation agent (advanced)

A LangChain agent equipped with tools — `query_graph(user_id)` (Neptune/Neo4j ring lookup), `get_user_history(user_id)` (Mongo), and `similar_cases(embedding)` (vector search). The analyst asks "is this part of a ring?" and the agent orchestrates the lookups — tying graph + retrieval + LLM into one showpiece.

Two things that stop it looking bolted-on

(1) Make the LLM call explicitly async / button-triggered and say so in the review. (2) Always ground the chain in real SHAP features + retrieved cases — an ungrounded summary adds nothing; a grounded, cited one is the whole point.

08

LLMOps with LangSmith

LangSmith provides observability and evaluation for the LLM layer — the LLMOps counterpart to MLflow.

Capability	What it does here	Why it matters
Tracing	Captures prompt, retrieved context, tool calls, latency & token cost per case	Debug why a summary was wrong
Evaluation	Faithfulness / hallucination / relevance evaluators on a curated case set	A hallucinated fraud rationale is worse than none
Monitoring	Analyst thumbs up/down captured per summary; tracks quality drift	Closes an LLM-specific feedback loop
Cost	Token spend per chain	Pairs with the cluster cost dashboard

The symmetry to pitch

Two parallel observability stacks: MLflow watches the fraud-scoring model on the hot path; LangSmith watches the LLM analyst assistant. Together they form a mature MLOps + LLMOps narrative.

07B Fine-Tuned Analyst LLM (Feedback Loop)

The analyst-assist layer is not static. Every summary the analyst rates with a thumbs-up/down is captured, and those judgements are used to fine-tune a dedicated analyst LLM. Crucially, LangChain is retained as the orchestration layer — the fine-tuned model is swapped in behind the same summary / RAG / agent chains, so the rest of the system is unchanged.

The closed-loop pipeline

- **Capture** — The dashboard persists each thumbs-up / thumbs-down with the prompt context (transaction, SHAP features, retrieved cases) locally and to LangSmith.
- **Curate** — Thumbs-up, grounded, cited summaries are exported to a chat-format JSONL dataset once a minimum example threshold is reached.
- **Fine-tune** — A managed fine-tune (OpenAI) runs when a key is present; otherwise a local LoRA / PEFT adapter is trained on a small base model — no cloud required.
- **Serve** — Setting `LLM_PROVIDER=finetuned` routes the LangChain chains to the fine-tuned model (hosted ft: id or local adapter) via one provider seam.
- **Evaluate** — LangSmith faithfulness / relevance / hallucination evaluators gate the fine-tuned model before it replaces the incumbent analyst LLM.

Provider abstraction (one seam, four modes)

Mode	Model served	When it is used
fake	Deterministic FakeListLLM	No keys — offline demos & unit tests
openai / anthropic	Hosted foundation model	API key present; base analyst quality
finetuned (hosted)	ft:gpt-... fine-tune	After a managed fine-tune job succeeds
finetuned (local)	Base + LoRA adapter	Air-gapped / no-cost fine-tune fallback

Design principle #4 — keep the LLM swappable and off the hot path

LangChain stays the stable orchestration seam; the model behind it evolves from foundation → fine-tuned as analyst feedback accumulates. The fine-tuned model still only runs on demand when an analyst clicks “Explain”, never inline in scoring.

09

Data Engineering Backbone

Just enough data-engineering tooling to signal production-readiness without over-scoping the project.

Tool	Role in this project	Why it matters
Delta Lake (medallion)	bronze = raw Kafka dump · silver = cleaned/enriched · gold = training features	Lineage, audit trail, reproducible retraining
Databricks Workflows / Airflow	Orchestrate periodic retraining & evaluation	Separates streaming (hot) from scheduled batch (cold)
Feature Store (Databricks / Feast)	Same feature definitions online & offline	Eliminates training/serving skew
Auto Loader	Incremental ingest of archived/batch files	Backfills without custom code
Great Expectations	Data-quality gate on the silver layer	Stops garbage features poisoning the model

Keep it a hint, not a pivot

For a lightweight footprint, Delta Lake + one orchestrator + the Feature Store is enough to prove you understand demo-vs-production — without ballooning scope.

10

End-to-End Data Flow

The eight-second story, at a glance:



Step	What happens
1	Producer emits a transaction to Kafka topic raw-transactions.
2	Databricks stream reads it, updates the user's rolling-window state, and builds the feature vector (incl. graph features from Neptune).
3	The stream loads the MLflow Production model, scores the transaction, and assigns a risk band.
4	Decision written to MongoDB (flagged_transactions + user_rolling_state upsert); raw + scored row appended to Delta bronze/silver.
5	Streamlit auto-refreshes and shows the new alert on the feed and geo-map.
6	Analyst clicks "Explain this case" → LangChain summary + RAG chain pulls SHAP features, history, and similar past cases (Mongo Vector Search) to generate a grounded narrative; optional agent queries the graph.
7	LangSmith traces the whole run — prompt, context, tool calls, latency, tokens — and evaluates faithfulness.
8	Analyst labels the case and rates the summary → labels feed MLflow retraining; summary feedback grows the LangSmith eval set. Better models are promoted with no stream redeploy.

11

Data Models & Schemas

Kafka event payload (raw-transactions)

```
{
  "txn_id": "b8f1...e2",      "user_id": "U_10432",
  "amount": 1299.00,         "currency": "INR",
  "merchant": "QuickCart",   "merchant_category": "electronics",
  "geo": { "lat": 19.07, "lon": 72.87, "country": "IN" },
  "device_id": "D_88",       "ip": "203.0.113.7",
  "timestamp": "2026-07-25T04:51:22Z"
}
```

MongoDB — flagged_transactions document

```
{
  "_id": "b8f1...e2",          // = txn_id (idempotent)
  "user_id": "U_10432",
  "amount": 1299.00,
  "risk_score": 0.87,          "risk_band": "high",
  "features": { "amt_z": 8.1, "txn_1m": 4, "new_geo": true },
  "graph": { "ring_id": "R_22", "shared_device_flags": 3 },
  "shap_top": [ ["amt_z", 0.41], ["txn_1m", 0.28] ],
  "status": "open",           "created_at": "2026-07-25T04:51:23Z"
}
```

12

Trade-offs & Design Decisions

Decision	Trade-off / rationale
Stateful streaming windows	State grows with active users → bound it with watermarks + state TTL.
Imbalanced learning	Optimise recall on fraud; accept some false positives but manage analyst fatigue.
Hot vs cold consistency	MongoDB is eventually consistent with Delta — acceptable: Delta is truth for training, Mongo for serving.
LLM off the hot path	Adds a manual step, but protects the sub-second scoring SLA and controls token cost.
MongoDB as serving store	Chosen for document-shaped enriched records + fast reads + optional vector search — defensible over a relational store here.
Cost of always-on stream	Streaming clusters run continuously → use small autoscaling job clusters; single-node for the demo.

13

Security & Reliability

Security & data integrity

- **PII protection:** tokenize/encrypt sensitive fields (PAN, card number) with MongoDB Queryable Encryption or field-level encryption.
- **Input validation** on the Kafka ingress to reject malformed / poisoned events — guards the retraining set against data-poisoning.
- **Adversarial testing:** simulate fraudsters probing thresholds (slow-ramp amounts) and show the system detecting the pattern, not just the point.

Reliability & scale

- **Dead-letter queue** in Kafka for un-processable events.
- **Backpressure & autoscaling** for the Databricks stream under burst load.
- **Chaos test:** kill a consumer mid-stream and demonstrate checkpoint recovery with zero data loss (exactly-once proof).

Demo-winning polish

A live "inject a card-testing burst now" button in Streamlit, plus a cost dashboard (events processed vs cluster spend) and a p50/p95/p99 latency panel, make the demo gripping and quantified.

14

Build Plan & Milestones

Phased four-week plan

Phase	Focus	Deliverable
Week 1	Core plumbing	Kafka + synthetic producer → Databricks stream → Mongo + Delta; dummy scorer; basic Streamlit feed
Week 2	Real ML + graph	Rolling features, Isolation Forest via MLflow, Feature Store, Neptune graph + ring detection, SHAP in review panel
Week 3	LLM assistant	LangChain summary + RAG over Mongo Vector Search; wire the "Explain" button; add the investigation agent if time allows
Week 4	Observability & polish	LangSmith tracing + eval + feedback; drift detection + auto-retrain; cost/SLA dashboards; live attack-injection demo

Compressed 2-week option

Fold the LangChain summary chain (no agent) into Week 2, and LangSmith tracing (no formal eval) into a stretch task. You still have a demo-able LLM-explainability story — matching an aggressive-but-realistic timeline.

15

Tech Stack & Success Metrics

Full technology stack

Layer	Technology
Ingestion / streaming bus	Apache Kafka (Kafka Connect, Debezium optional)
Stream processing / ML compute	Databricks Structured Streaming (PySpark)
ML lifecycle	MLflow (tracking + registry), Isolation Forest → XGBoost/LightGBM, SHAP
Graph	AWS Neptune or Neo4j (Gremlin / openCypher)
Serving store	MongoDB (+ Atlas Vector Search)
LLM orchestration	LangChain (chains, RAG, agents)
LLM observability	LangSmith (tracing, evaluation, monitoring)
Storage / table format	Delta Lake (medallion) — Iceberg optional
Orchestration	Databricks Workflows or Airflow / Dagster
Data quality	Great Expectations / Soda
UI / dashboard	Streamlit

Success metrics (put these in your README)

Metric	What good looks like
End-to-end scoring latency	p95 < 2 s under target load
Throughput	Sustains target events/sec without lag growth
Detection quality	Precision/recall on fraud class + expected-loss captured
Explanation faithfulness	LangSmith faithfulness score above threshold; low hallucination rate
Adaptability	Retrain triggered on drift; challenger beats production before promotion
Cost efficiency	\$ per million events processed, trended over time

Bottom line: this specification describes a system that is real-time, explainable, graph-aware, and observably self-improving — a portfolio piece that reads like production engineering, not a demo.

PART II

Interview Preparation Guide

Part I of this document is the system design specification. Part II is the companion interview preparation guide: the elevator pitches, the use cases and data sourcing, twenty annotated code exhibits, the design decisions and their costs, the alternatives that were considered and rejected, the known gaps to volunteer before an interviewer finds them, and roughly a hundred practice questions with model answers.

Section	What it covers
1 Elevator pitches	Four lengths: one-line, 30s, 90s, 3-minute technical
2 Skills & tools inventory	Vocabulary surface + lift-verbatim resume bullets
2B Use cases & data sourcing	Who uses it, the attack patterns, and where real data comes from
3 Architecture walkthrough	The data path, narrated file by file
3B Annotated code exhibits	20 code blocks with 'what to say' for each
4 Design decisions	26 decisions, each with the cost accepted
5 Alternatives not used	~70 alternatives across 15 categories, and why not
6 Difficulties	Solved, latent, and production-grade
7 Known gaps	18 real weaknesses + the honest line to say
8 Questions by skill	~90 Q&As across 12 skill areas
9 Scale-up whiteboard	20/s to 50,000/s, bottleneck first
10 Numbers cheat sheet	Every default, and how to measure real latency
11 STAR stories	Six behavioural answers drawn from the build
12 Questions to ask	Nine questions, and what each one signals
13 48-hour prep plan	Hour-by-hour drill schedule

How to use this document. Sections 1–3 are what you say. **Section 3B is what you show** — twenty annotated code exhibits, so the code travels with you even when the repository can't. Sections 4–6 are what you *defend*. Section 7 is what you *volunteer before they find it* — that section is the single highest-value part of this guide, because every gap listed there is real and findable in ten minutes of code reading. Sections 8–13 are drills.

Rule for the whole interview: **never claim a measured number you haven't measured**. Say "designed for", "the bottleneck is", "I haven't load-tested it — here's what I'd expect and why". Senior interviewers reward that; they punish invented p99s.

Table of contents

1. Elevator pitches (four lengths)
2. Skills & tools inventory — say these words out loud
 - 2B. Use cases, users, and where the data comes from
3. The 3-minute architecture walkthrough
 - 3B. Annotated code walkthrough — 20 exhibits
4. Design decisions and why (the defensible core)
5. Alternatives you did NOT use, and why
6. Difficulties: solved, latent, and production-grade
7. Known gaps — pre-empt these
8. Interview questions by skill, with model answers
9. The scale-up whiteboard question
10. Numbers & facts cheat sheet
11. Behavioural / STAR stories from this project
12. Questions you ask them
13. 48-hour prep plan

1. Elevator pitches

One-liner (for a resume headline or a recruiter)

A streaming fraud-detection platform: Kafka → real-time feature engineering → graph-aware ML scoring → MongoDB, with SHAP explanations, a LangChain analyst copilot that never touches the hot path, and a closed feedback loop that retrains both the model and the LLM.

30 seconds (non-technical / HR / hiring manager)

"I built an end-to-end real-time fraud detection platform. Transactions stream through Kafka, and within a second or two each one gets scored for fraud risk using rolling behavioural features plus graph signals — like whether the user shares a device with other known-fraud accounts. Flagged cases land in a review console where an analyst sees *why* it was flagged, in plain English, with the evidence cited. Their thumbs-up/thumbs-down feeds back into retraining. The whole thing runs on a laptop with Docker, and adding cloud credentials transparently upgrades it — no code change."

90 seconds (engineering screen)

"The core idea is **three planes with three different SLAs**, and the discipline is keeping them separate.

The **hot path** is sub-second and completely deterministic: a Kafka consumer partitioned by `user_id` computes rolling velocity and amount z-score features, enriches with graph features from Neo4j, scores with the model that's currently aliased `Production` in the MLflow registry, and upserts to MongoDB. No LLM inline — ever. That's a deliberate constraint, not an omission.

The **analyst-assist plane** is on-demand and off the hot path. LangChain orchestrates a RAG chain that retrieves similar resolved cases from a vector store and produces a grounded, cited summary, plus a tool-calling investigation agent with three tools — query the graph, get user history, find similar cases. LangSmith traces it.

The **cold path** is batch: the lakehouse, PSI-based drift detection, and retraining where a challenger model only gets promoted if it beats the incumbent. Because the stream reads the model by *registry alias* and hot-reloads on a timer, I can promote a new model with zero stream downtime.

The thing I'm proudest of is the symmetry: the classical model has MLflow, PR-AUC and drift detection; the LLM has LangSmith, faithfulness scoring and a fine-tune loop from analyst feedback. Same MLOps discipline applied to both."

3 minutes (deep technical — add these beats)

Everything above, plus the three things that show engineering maturity:

1. **Correctness under failure.** "At-least-once ingest with manual offset commits *after* a successful write, and every write is an idempotent upsert on a natural key — `_id == txn_id`. So the composite behaviour is effectively-once for the serving store, and Kafka can be replayed safely. Poison messages go to a dead-letter topic instead of crash-looping the consumer. There's a `make_chaos_drill` that publishes known IDs, kills the consumer mid-stream, restarts it, and verifies nothing was lost."
2. **Graceful degradation as a design principle.** "Every external dependency has a fallback that keeps the pipeline running: no MLflow model → deterministic heuristic scorer; no Neo4j → graph features return zeros; no OpenAI key → a deterministic `FakeListLLM`; no sentence-transformers → a hashing embedder; no Atlas → brute-force cosine; no GDS tier → networkx connected components. The system never hard-fails on a missing credential, which is also what makes it demoable and testable offline."
3. **Testability by construction.** "Feature engineering is a pure function — `compute_features(event, state) -> (features, new_state)`, no Kafka, no Mongo, doesn't mutate its input. That's why I can unit-test velocity windows and z-score behaviour with plain dicts, and why the same code path is used by the stream, the tests, and the training-data generator."

2. Skills & tools inventory

Say these words in the interview. Interviewers pattern-match on vocabulary before they evaluate depth. This is your keyword surface — and every item is genuinely in the repo.

Area	Tools / techniques you can legitimately claim
Streaming	Apache Kafka (Redpanda, Kafka-API compatible), confluent-kafka / librdkafka, partitioning by key for ordering, consumer groups, manual offset management, <code>enable.auto.commit=false</code> , <code>acks=all</code> , idempotent producer, <code>auto.offset.reset=earliest</code> , dead-letter queue, topic admin (<code>AdminClient</code> , <code>NewTopic</code> , 6 partitions), at-least-once semantics, replay safety
Stream processing	Stateful per-key stream processing, rolling/velocity windows (1m/5m/1h), Welford's online algorithm for running variance, pure-function feature computation, event-time handling, bounded state, backpressure reasoning
Serving store	MongoDB 7, idempotent upserts (<code>update_one(..., upsert=True)</code>), natural-key <code>_id</code> design, compound index on (<code>status</code> , <code>priority</code>), index strategy for read patterns, Atlas <code>\$vectorSearch</code> aggregation pipeline, brute-force cosine fallback
Graph	Neo4j Aura (managed cloud), Cypher, MERGE for idempotent graph writes, uniqueness constraints, entity-resolution graph modelling (<code>User -[:USED]-> Device</code> , <code>-[:PAID]-> Merchant</code> , <code>-[:FROM_IP]-> IP</code>), Weakly Connected Components for ring detection, Neo4j GDS (<code>gds.graph.project.cypher</code> , <code>gds.wcc.write</code>), networkx fallback, fraud-ring / shared-device features
ML	Unsupervised anomaly detection (<code>IsolationForest</code>), supervised gradient boosting (<code>XGBoost</code>), extreme class imbalance (<code>scale_pos_weight</code> , <code>eval_metric=aucpr</code>), PR-AUC vs ROC-AUC for imbalanced data, precision/recall/flag-rate as operational metrics, score normalisation to [0,1], threshold banding, synthetic data generation with controlled label distributions
Explainability	SHAP, <code>TreeExplainer</code> for tree ensembles, model-agnostic <code>Explainer</code> fallback, signed top-N feature attributions, explanation surfaced to a human reviewer, magnitude-ranking fallback
MLOps	MLflow tracking (params/metrics/artifacts), MLflow Model Registry, alias-based deployment (<code>models:/name@Production</code>), champion/challenger promotion gated on a metric, hot-reload of the production model without process restart, PSI (Population Stability Index) drift detection with a 0.2 threshold, drift-triggered retraining, model versioning surfaced in the UI
LLM / GenAI	LangChain, LCEL (<code>prompt llm StrOutputParser</code>), <code>ChatPromptTemplate</code> , provider abstraction (OpenAI / Anthropic / fake / fine-tuned behind one factory), RAG, embeddings (<code>sentence-transformers</code> , <code>all-MiniLM-L6-v2</code> , 384-dim), vector similarity search, grounded prompting with citation enforcement, citation extraction/verification via regex, tool-calling agents (<code>create_tool_calling_agent</code> , <code>AgentExecutor</code> , <code>max_iterations</code>), <code>@tool</code> decorator, deterministic fallback orchestration for non-tool-calling models
LLMOps	LangSmith tracing, project-scoped runs, human feedback capture (thumbs-up/thumbs-down), offline eval harness with faithfulness / relevance / hallucination proxies, no-op tracing wrapper so local runs never break
Fine-tuning	Preference-data collection from real human feedback, chat-format JSONL dataset construction, OpenAI fine-tuning API (file upload → job → poll → ft: model id), PEFT/LoRA (<code>r=8</code> , <code>lora_alpha=16</code> , <code>lora_dropout=0.05</code> , <code>task_type=CAUSAL_LM</code>), HuggingFace <code>Trainer</code> + <code>TrainingArguments</code> , <code>DataCollatorForLanguageModeling</code> , serving an adapter through <code>HuggingFacePipeline</code> behind the same LangChain interface
Backend / Python	Python 3.11+, Pydantic v2 data contracts (<code>aliases</code> , <code>field_validator</code> , <code>populate_by_name</code> , computed properties), pydantic-settings typed config with <code>LruCache</code> singleton, structlog structured logging, context managers, graceful SIGTERM/SIGINT shutdown, module-level singletons, lazy imports for heavy deps
Frontend / ops UI	Streamlit (multi-column layout, <code>st.cache_data</code> with TTL, session state, metrics row, map, live auto-refresh), human-in-the-loop review workflow, alert triage by expected loss = score × amount
Engineering practice	Docker Compose (Redpanda + Mongo + MLflow with healthchecks and named volumes), Makefile task runner, <code>pyproject.toml</code> with optional-dependency extras, pytest (fixtures, <code>monkeypatch</code> , <code>tmp_path</code>), ruff, black, mypy with the pydantic plugin, twelve-factor config (nothing hard-coded), architecture + runbook docs, chaos/recovery drill
Domain	Card-testing bursts, geo-impossible travel, velocity spikes, account-takeover signals, fraud rings / shared-device collusion, analyst review queues, expected-loss triage, false-positive cost asymmetry

Resume bullets you can lift verbatim

- Built a real-time fraud detection pipeline on Kafka (Redpanda) processing partitioned transaction streams with at-least-once delivery, idempotent MongoDB upserts and a dead-letter queue, verified by an automated kill-and-recover chaos drill.
- Engineered stateful rolling-window features (multi-horizon velocity, Welford online variance z-score, geo/device novelty) as a pure, unit-tested function decoupled from the streaming runtime.
- Added graph-aware fraud-ring detection on Neo4j Aura using an entity-resolution graph (user/device/merchant/IP) and Weakly Connected Components, with a networkx fallback for non-GDS tiers.
- Implemented MLflow-registry-driven deployment: challenger models promoted to the `Production` alias only on a PR-AUC gate, hot-reloaded by the running stream with zero downtime; PSI drift detection triggers retraining.
- Delivered a LangChain analyst copilot (grounded RAG summaries with enforced citations + a 3-tool investigation agent) kept strictly off the scoring hot path, traced with LangSmith.
- Closed the human-feedback loop: analyst thumbs-up/thumbs-down on LLM summaries exports to a chat-JSONL dataset and fine-tunes the analyst model (OpenAI FT API or local LoRA/PEFT), served back through the identical LangChain interface.

2B. Use cases, users, and where the data comes from

Why this section matters. Two of the most common interview questions about a personal project are "*who would actually use this?*" and "*where did the data come from?*" — and the second one is a trap. If you say "synthetic" and stop, you sound like you avoided the hard part. If you say "synthetic, **here's why that was the right call, here are the four public datasets I'd move to, and here's what real production data looks like and why I couldn't have it**", you sound like someone who has thought about data provenance. Same fact, opposite impression.

2B.1 The use case actually built

Card-not-present payment fraud detection with human-in-the-loop review.

A payment processor or e-commerce platform authorises transactions in real time. Each one must be scored for fraud risk before the authorisation completes. High-risk transactions are held and routed to a fraud analyst who must decide, in seconds-to-minutes, whether to release or block — and that analyst needs to know *why* the system flagged it. The system detects three concrete attack patterns end to end:

Attack pattern	What it looks like	Which features catch it
Card testing / BIN attack	5–12 tiny authorisations (₹1–25) in under 60 seconds from new devices, validating stolen card numbers before the real purchase	txn_1m, txn_5m, seconds_since_last, new_device
Geo-impossible travel	Two high-value transactions 8,000 km apart within 90 seconds on the same card	new_geo, distinct_geos_1h, seconds_since_last, and the haversine_km helper
Fraud ring / collusion	Multiple accounts sharing one device or IP, cashing out in parallel — no single transaction looks unusual	shared_device_flags, component_size, ring_id (the whole reason the graph layer exists)
Account takeover (partial)	Sudden behavioural break: amount far outside the user's own baseline, from an unrecognised device	amt_z (Welford z-score against the user's own history), new_device, new_geo

The key framing: the first two are catchable per-transaction. **The third is only catchable in a graph** — and the fourth is only catchable if you model each user against *their own* baseline rather than a population average. That's the argument for all three of my feature families existing.

2B.2 Who uses it (four personas, four different surfaces)

Persona	What they need	What the system gives them
Fraud analyst (the primary user)	Triage a queue fast; decide release-or-block with confidence; not read a page of prose per case	Alerts ranked by expected loss , SHAP top-5 reasons, ring/graph context, a ≤120-word cited LLM summary on demand, one-click FRAUD / NOT-FRAUD
Fraud ops manager	Is the queue survivable? Is the model still good?	Flag rate, volume scored, model version in production, score distribution
Data scientist / ML engineer	Retrain safely; know when the model has decayed	MLflow experiments + registry, PR-AUC/precision/recall/flag-rate, PSI drift per feature, promotion gate
Platform / on-call engineer	Is the pipeline healthy and lossless?	Structured event logs, DLQ as an explicit surface, the chaos/recovery drill, alias-based instant model rollback

The value proposition in one line, per persona: the analyst gets **throughput** (that's what the LLM buys — it compresses a case file into a paragraph); the manager gets **control**; the data scientist gets **safe iteration**; the engineer gets **replayability**.

2B.3 Where the data comes from — the honest answer

Today: fully synthetic, and deliberately so.

Two generators, serving two different purposes:

Generator	Produces	Parameters
producer/generator.py	The live Kafka event stream	500 users, 8 merchants across 6 categories, 6 cities (3 domestic + 3 international), ~20 events/s, 2% attack rate, injectable attacks on demand
ml/synthetic.py	The labelled training frame	8,000 rows, 3% fraud, with fraud rows drawn from deliberately shifted distributions ($\text{amt}_z \sim N(6.0, 2.5)$ vs $N(0, 1)$, $\text{txn_1m} \sim \text{Poisson}(6)+3$ vs $\text{Poisson}(1)$)

Say this, in this order:

1. **"Real payment data is not obtainable for a personal project, and that's not a convenience excuse — it's a regulatory fact.** Card data is governed by PCI-DSS: you cannot store or process a primary account number outside a certified environment. No payment processor releases transaction-level data with user identifiers. So every honest option is either synthetic or a heavily anonymised public dataset."
2. **"Synthetic bought me something a real dataset couldn't:** a labelled attack I can inject *on demand*, mid-demo. The dashboard's attack injector drops a signal file the producer polls, so I can trigger a card-testing burst and watch it surface as a HIGH-band alert within seconds. That's how you demonstrate a detection system rather than describe one."
3. **"And I'd name the cost immediately:** my synthetic training distribution doesn't match my serving distribution, which is exactly the train/serve skew in §7. So synthetic validates the *pipeline* and produces meaningless *accuracy numbers*. Those are different claims and I keep them separate."

2B.4 The public datasets I'd move to, and what each one is for

This is the answer to "okay, so where would you get real data?" — and being specific about **what each dataset can and cannot support** is what makes the answer credible.

Dataset	Shape	What it's good for	The limitation that matters
IEEE-CIS Fraud Detection (Kaggle / Vesta, 2019)	~590k transactions, 400+ features, ~3.5% fraud	The most realistic public tabular fraud set: real e-commerce, real imbalance, rich device/card/email-domain features. My best drop-in for training XGBoost	Features are partly obfuscated; no clean per-user event ordering, so rolling velocity features are awkward to reconstruct
Credit Card Fraud Detection (ULB / Worldline)	284,807 transactions, 492 frauds (0.172%), features V1-V28 (PCA) + Time + Amount	The canonical extreme-imbalance benchmark. Perfect for demonstrating PR-AUC over ROC-AUC and cost-sensitive thresholding	PCA-anonymised, so no interpretable SHAP (what does "V17" mean to an analyst?) and no user or device id , so no behavioural features and no graph. Structurally incompatible with two-thirds of my pipeline
PaySim (agent-based mobile-money simulator)	~6.3M transactions with nameOrig / nameDest account identifiers	The graph one. Because it has sender <i>and</i> receiver ids, you can build a real transaction graph and run WCC/Louvain on it — which the ULB set cannot support at all	Synthetic, and the fraud logic is rule-generated, so a model can learn the simulator rather than fraud
Sparkov / "Credit Card Transactions Fraud Detection" (Kaggle)	~1M+ transactions with merchant, category, lat/lon, timestamp, customer id	Closest match to my actual schema — it has the fields my TransactionEvent contract needs, so it's the least-friction real-ish substitution	Synthetic (Sparkov generator), so same caveat as PaySim
BankSim	~600k bank-payment records with customer and merchant nodes	Merchant-side fraud and bipartite customer↔merchant graph analysis	Small, synthetic, limited feature richness

Dataset	Shape	What it's good for	The limitation that matters
Elliptic / Elliptic++	Bitcoin transaction graph: ~204k nodes, ~234k edges, 49 time steps, ~2% illicit / 21% licit / 77% unknown	Graph-native, genuinely labelled, and time-stepped — the right dataset to benchmark hand-crafted graph features against a GNN (GraphSAGE), which is my \$5.6 argument	Crypto, not card payments; and the 77% unlabelled fraction is itself the lesson about label censoring
AMLSim / IBM AML synthetic	Synthetic account graph with injected laundering typologies (fan-in, fan-out, cycles)	The AML adjacency — multi-hop typology detection, which is where graph databases really earn their place	Synthetic typologies are cleaner than reality
CMS Medicare /Medicaid claims + HHS-OIG LEIE exclusion list	Provider-level claims data + a labelled list of excluded (sanctioned) providers	The healthcare analogue: the LEIE gives you real labels for provider-level fraud/abuse, which is rare in open data	Provider-aggregate, not per-transaction, so latency isn't the problem — volume and multi-year patterns are

The line that ties it together: "My Pydantic contract is the integration seam. Adopting any of these is one adapter file that maps their columns onto TransactionEvent — everything downstream (features, scoring, graph, Mongo, the LLM layer) is unchanged. That was a deliberate property of putting all the schemas in one module, not an accident."

2B.5 Where the data comes from in a real production deployment

Ranked by how load-bearing each source is. This is the answer that shows you know what you *would* be plugging into.

Source	What it provides	How it arrives
Payment authorisation stream	The core event: amount, currency, merchant, MCC, card token, timestamp	The card-network authorisation message (ISO 8583) or a gateway webhook, published straight to Kafka. This is the hot-path input
Core banking / ledger CDC	Balances, account age, historical spend, account status	Change-data-capture via Debezium off the database's write-ahead log, into Kafka — no dual writes, no polling
Device-fingerprinting SDK	device_id, browser/OS fingerprint, emulator and root detection	Client SDK → collection endpoint → Kafka. This is what makes the graph layer possible — without a stable device id there are no shared-device edges and no rings
IP / geolocation intelligence	Country, city, lat/lon, proxy/VPN/Tor and datacentre-ASN flags	A vendor lookup (e.g. MaxMind) enriched inline or in a preceding enrichment stage
3-D Secure / issuer signals	Step-up authentication outcome, issuer decline reason codes	Returned in the auth flow; a very strong feature and also the <i>action</i> the system can request
Analyst case-management system	Analyst disposition per case — the fast label source	This is my set_label() write path. Labels in hours
Chargeback / dispute feed	Confirmed fraud from the network — the authoritative label	Batch file or API from the acquirer. Labels in 30–90 days
KYC / identity graph	Shared addresses, phone numbers, emails, national identifiers	Batch load into Neo4j; these are <i>additional edge types</i> on the same entity graph, and they're what catch synthetic-identity rings
Sanctions / watchlists / consortium data	Known-bad cards, devices, IPs, mule accounts; cross-institution signals	Periodic batch; a consortium feed is the highest-signal external data in fraud and the hardest to get
Merchant onboarding / risk profile	Merchant category, tenure, historical chargeback rate	Batch reference data joined at enrichment time

2B.6 The label problem — say this unprompted, it's the mark of someone who's thought about it

"The hardest data problem in fraud isn't volume, it's **labels**, and specifically that they arrive at four different latencies with four different levels of authority."

Label source	Latency	Authority	Coverage
Analyst disposition	Hours	Medium (human judgement, can be wrong)	Only reviewed cases — i.e. only what you already flagged
Customer-reported fraud	Days	High	Only what the customer noticed
Chargeback	30–90 days	Highest	Only disputed transactions
Written-off loss	90+ days	Highest	Small tail

Three consequences to name:

1. **Censoring: unlabelled ≠ legitimate.** A transaction with no dispute after two days isn't confirmed-good, it's *unknown*. Training on "no chargeback yet □ negative" systematically mislabels recent fraud as legitimate, and it gets worse the fresher your data is. This is why you need a maturation window before a period is trainable.
2. **Selection bias / feedback loop.** You only observe outcomes for transactions you **allowed**. Blocked transactions have no ground truth, so the model progressively stops learning about the exact region it already blocks. Mitigations: a small randomised hold-out that's allowed through despite a high score, propensity weighting, or explicit exploration. Expensive, and necessary.
3. **My project's version of this:** analyst labels land via `set_label()` in Mongo, but `make_train` doesn't read them — it trains on synthetic data. **So the feedback loop is architecturally complete and not yet closed.** That's an honest, specific gap and it's a better answer than pretending otherwise.

2B.7 Compliance constraints on the data (fraud is a regulated domain)

- **PCI-DSS** — never store the raw card number. Work with a network token or a surrogate, and keep the cardholder-data environment scoped. My schema uses `user_id` and `device_id` identifiers deliberately: **there is no PAN anywhere in the contract.**
- **GDPR / India's DPDP Act 2023** — purpose limitation, data minimisation, and a right to explanation for automated decisions. Fraud prevention is generally a legitimate-interest basis, but automated *declines* need a reason-code path, which is where SHAP becomes a compliance artefact rather than a nice-to-have.
- **Data residency** — my currency default is INR, which in a real deployment means the data likely cannot leave the country. That immediately constrains which managed services and which LLM providers are usable.
- **The LLM boundary is the sharpest one** (see §6.3): my prompts contain transaction context, so calling a third-party model — and tracing prompts to LangSmith as SaaS — moves regulated data across a boundary. The real-deployment answer is redact/tokenise before the prompt, self-host the model, and self-host tracing with **Langfuse** instead of LangSmith.

2B.8 Where else this architecture applies (the transferability answer)

Interviewers frequently ask "how would this apply to what we do?" The generalisable pattern is **not "fraud"** — it's:

a high-velocity event stream → real-time scored decisions under a latency SLA → an explanation a human can act on → a review queue prioritised by business impact → a feedback loop that retrains the model.

Change the event and the label; the architecture is unchanged.

Domain	The event	The decision	What changes
AML / transaction monitoring	Wire, ACH, UPI transfer	Suspicious-activity alert	Multi-hop graph typologies matter far more (fan-in/fan-out/cycles); latency SLA relaxes to minutes; audit trail requirements get much stricter
Insurance claims fraud	Submitted claim	Route to investigation	Graph over claimant ↔ provider ↔ repairer ↔ adjuster; latency in minutes; document/NLP features become central

Domain	The event	The decision	What changes
Healthcare claims fraud, waste & abuse	Claim / billing line	Pre-payment review	Graph over provider ↔ patient ↔ referrer; real labels exist via exclusion lists; upcoding and phantom-billing patterns replace velocity
Medical-device / IoT telemetry anomaly detection	Device telemetry event	Predictive-maintenance or safety alert	Same streaming skeleton and the same SHAP-plus-review-queue pattern; unsupervised anomaly detection dominates because failure labels are rare; the "analyst" is a field service engineer
Account security / ATO	Login, password reset, device enrolment	Step-up authentication	Velocity and novelty features transfer almost directly; the action is a challenge rather than a decline
Marketplace / seller fraud	Listing, review, payout request	Suspend or hold payout	Graph over seller ↔ buyer ↔ device ↔ payout account; review-fraud rings are the classic graph win
Ad-tech click fraud	Impression / click	Discard or bill	Volume is orders of magnitude higher, so the latency budget forces everything into local state; no human review at all
Telecom fraud	Call detail record, SIM swap	Block or verify	Velocity + graph over SIM ↔ IMEI ↔ tower; near-identical shape to card testing

The strongest version of this answer: "The three-plane split is the transferable idea, not the fraud logic. Any domain where a fast automated decision needs a slow human explanation has the same tension I solved — and the answer is always to give the two planes separate SLAs instead of letting the slower one set the pace."

2B.9 Likely questions on this section

Q: Your data is synthetic. Why should I believe any of this works? "You shouldn't believe the *accuracy numbers* — I don't either, and I'd tell you that before you asked. Synthetic data validates the pipeline: the schema contract, the delivery semantics, the training and promotion path, drift detection, the explanation surface, the feedback loop. Those are engineering claims and they're testable. Model quality is a *data* claim and it needs real data. Keeping those two claims separate is the point."

Q: What would you use instead, concretely? "IEEE-CIS for supervised training because it's real e-commerce data with real imbalance; PaySim or Elliptic for anything graph, because they're the only ones with the sender/receiver structure a graph needs; the ULB set specifically to demonstrate extreme-imbalance handling — though it's PCA-anonymised, so it breaks my SHAP story and can't support behavioural features at all. That last caveat is the reason I'd use three datasets rather than one."

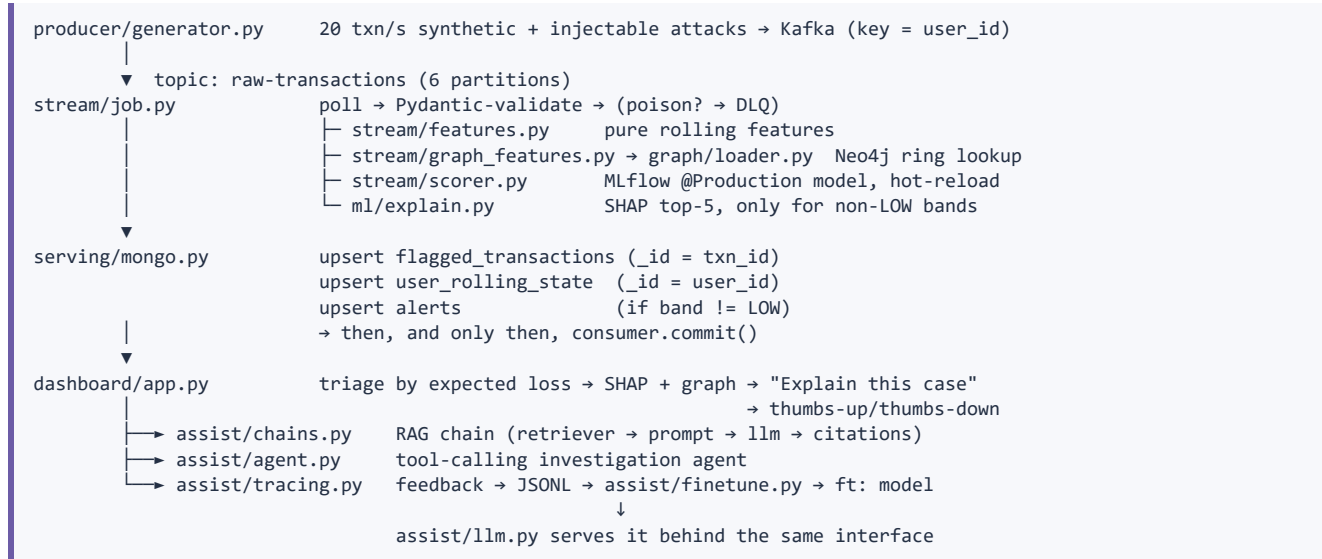
Q: Where do the labels come from? "Four sources at four latencies: analyst disposition in hours, customer reports in days, chargebacks in 30–90 days, write-offs later. The critical subtlety is that unlabelled is *not* negative — recent transactions are censored, not clean. And I only observe outcomes for transactions I allowed, so the model stops learning about what it already blocks."

Q: How would you handle PCI / PII? "No PAN in the contract at all — I key on `user_id` and `device_id`, and in production those would be network tokens or surrogates. The sharper problem is the LLM: my prompts contain transaction context, so a third-party model and SaaS tracing both move regulated data across a boundary. Redact before the prompt, self-host the model, and use self-hosted Langfuse instead of LangSmith."

Q: How would this apply to our domain? Use the §2B.8 table. Lead with the pattern — "a high-velocity stream, a fast automated decision, a slow human explanation, and a feedback loop" — then map their event, their decision, and their label source. **Ask them what their label latency is;** it's the question that shows you understand the actual constraint.

3. The 3-minute architecture walkthrough

Learn to narrate the data path. Point at these files by name — naming files is a strong credibility signal.



Cold path: `ml/synthetic.py` → `ml/train.py` (train, log to MLflow, register, promote on metric) → `ml/drift.py` (PSI per feature, >0.2 ☐ retrain).

The five sentences that carry the whole design

1. "Hot path is deterministic and has no LLM in it."
2. "Commit the Kafka offset only after an idempotent write, so replay is free."
3. "The stream loads the model by registry *alias*, so deploying a model isn't a deploy."
4. "Every feature the model sees is computed by one pure function that I can unit test."
5. "Alerts are ranked by expected loss, not by score — analyst time is the scarce resource."

3B. Annotated code walkthrough

Why this section exists. You may not be able to share the repository, so the code has to travel with you. These twenty exhibits are the load-bearing parts of the system — enough that a technical interviewer can assess the engineering without ever seeing the repo.

How to use it. Do not memorise it line by line. For each exhibit, learn (a) what problem it solves, (b) the one line that carries the design, and (c) the *What to say* bullets. If an interviewer asks "can you show me some code?", these are the four to open with: **Exhibit 5** (delivery semantics), **Exhibit 7** (pure features + Welford), **Exhibit 8** (model hot-reload), **Exhibit 15** (grounded RAG chain).

Exhibit 1 — Data contracts as the single source of truth

src/fraud/schemas.py

```
class TransactionEvent(BaseModel):
    """Raw event published to the Kafka `raw-transactions` topic."""

    txn_id: str
    user_id: str
    amount: float = Field(ge=0)
    currency: str = "INR"
    merchant: str
    merchant_category: str
    geo: Geo
    device_id: str
    ip: str
    timestamp: datetime = Field(default_factory=_utcnow)

    @field_validator("timestamp", mode="before")
    @classmethod
    def _parse_ts(cls, v: Any) -> Any:
        if isinstance(v, str):
            return datetime.fromisoformat(v.replace("Z", "+00:00"))
        return v

class Features(BaseModel):
    """Rolling-window + graph features computed by the stream processor."""

    amt_z: float = 0.0
    txn_1m: int = 0
    txn_5m: int = 0
    txn_1h: int = 0
    distinct_merchants_1h: int = 0
    distinct_geos_1h: int = 0
    seconds_since_last: float = 0.0
    new_geo: bool = False
    new_device: bool = False
    # graph features
    ring_id: str | None = None
    shared_device_flags: int = 0
    component_size: int = 0

    def to_vector(self) -> list[float]:
        """Ordered numeric feature vector for the ML model."""
        return [
            self.amt_z, float(self.txn_1m), float(self.txn_5m), float(self.txn_1h),
            float(self.distinct_merchants_1h), float(self.distinct_geos_1h),
            self.seconds_since_last, float(self.new_geo), float(self.new_device),
            float(self.shared_device_flags), float(self.component_size),
        ]

    @staticmethod
    def feature_names() -> list[str]:
        return ["amt_z", "txn_1m", "txn_5m", "txn_1h", "distinct_merchants_1h",
            "distinct_geos_1h", "seconds_since_last", "new_geo", "new_device",
            "shared_device_flags", "component_size"]
```

```

class ScoredTransaction(BaseModel):
    """Document persisted to Mongo `flagged_transactions` (`_id == txn_id`)."""

    id: str = Field(alias="_id")
    user_id: str
    amount: float
    risk_score: float
    risk_band: RiskBand
    features: dict[str, Any] = Field(default_factory=dict)
    graph: GraphInfo = Field(default_factory=GraphInfo)
    shap_top: list[tuple[str, float]] = Field(default_factory=list)
    status: CaseStatus = CaseStatus.OPEN
    label: bool | None = None
    model_version: str | None = None
    created_at: datetime = Field(default_factory=_utcnow)

    model_config = {"populate_by_name": True}

    @property
    def expected_loss(self) -> float:
        return self.risk_score * self.amount

```

What to say:

- "One module is simultaneously the **Kafka wire format**, the **Mongo document shape** and the **internal Python type**. That's what makes validation-at-the-boundary meaningful — I know exactly where bad data gets rejected, and that's the DLQ path."
- "to_vector() and feature_names() sitting next to each other is deliberate: **the ordering contract between training and serving lives in one place**. A test asserts they stay the same length. Feature-order mismatch between train and serve is a classic silent ML bug and this is the cheapest possible guard against it."
- "alias='_id' with populate_by_name bridges Python naming and Mongo's _id without polluting the domain model. expected_loss is a computed property, not a stored field — which, honestly, is a mistake I'd reverse: it should be persisted and indexed, because it's my hottest sort key (see §7 #11)."
- "Field(ge=0) on amount is why test_negative_amount_rejected passes — the contract, not a hand-written if, does the enforcing."

Exhibit 2 — Typed configuration with capability flags

config/settings.py

```

class Settings(BaseSettings):
    """Typed application settings loaded from environment / .env."""

    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8",
        case_sensitive=False, extra="ignore",
    )

    kafka_bootstrap_servers: str = "localhost:19092"
    kafka_raw_topic: str = "raw-transactions"
    kafka_dlq_topic: str = "raw-transactions-dlq"
    kafka_consumer_group: str = "fraud-stream"

    mongo_uri: str = "mongodb://root:example@localhost:27017"
    mongo_use_atlas_vector: bool = False

    neo4j_uri: str = "" # empty => graph layer degrades to zeros
    neo4j_password: str = ""
    neo4j_use_gds: bool = False # False => networkx fallback

    mlflow_model_name: str = "fraud-scorer"
    mlflow_model_alias: str = "Production"

    llm_provider: str = "fake" # fake | openai | anthropic | finetuned
    finetuned_model_id: str = ""

```

```

risk_high_threshold: float = 0.80
risk_medium_threshold: float = 0.50
model_reload_seconds: int = 60

# ---- derived helpers: capability flags, not raw config -----

@property
def neo4j_configured(self) -> bool:
    return bool(self.neo4j_uri and self.neo4j_password)

@property
def llm_enabled(self) -> bool:
    if self.llm_provider == "openai":
        return bool(self.openai_api_key)
    if self.llm_provider == "anthropic":
        return bool(self.anthropic_api_key)
    return False

@property
def finetuned_enabled(self) -> bool:
    return self.llm_provider == "finetuned" and bool(self.finetuned_model_id)

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """Return a cached Settings singleton."""
    return Settings()

settings = get_settings()

```

What to say:

- "Twelve-factor: **nothing is hard-coded and every field has a safe local default**, so the whole platform runs on a clean checkout with zero credentials. Adding a secret transparently upgrades a capability — that's the deployment story in one sentence."
- "The derived properties are the important bit. `neo4j_configured`, `llm_enabled`, `finetuned_enabled` turn raw config into **capability flags**, so the rest of the codebase asks '*can I do this?*' rather than '*is this string non-empty?*'. Every graceful-degradation branch in the system reads one of these."
- "`lru_cache` makes it a singleton — the `.env` is parsed and validated once, and every module sees identical values. The cost is that it's awkward in tests, which is why the tests monkeypatch attributes on the object rather than reconstructing it."

Exhibit 3 — Kafka client configuration (the durability decisions)

src/fraud/producer/kafka_utils.py

```

def base_conf() -> dict[str, Any]:
    """Common Kafka client config, adding SASL only when configured (cloud)."""
    conf: dict[str, Any] = {"bootstrap.servers": settings.kafka_bootstrap_servers}
    if settings.kafka_security_protocol:
        # local => plaintext, cloud => SASL
        conf["security.protocol"] = settings.kafka_security_protocol
        conf["sasl.mechanism"] = settings.kafka_sasl_mechanism
        conf["sasl.username"] = settings.kafka_sasl_username
        conf["sasl.password"] = settings.kafka_sasl_password
    return conf

def producer_conf() -> dict[str, Any]:
    conf = base_conf()
    conf.update({"acks": "all", "enable.idempotence": True, "linger.ms": 20})
    return conf

def consumer_conf(group: str | None = None) -> dict[str, Any]:
    conf = base_conf()
    conf.update({
        "group.id": group or settings.kafka_consumer_group,

```

```

    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,      # <-- the whole correctness story
  })
  return conf

```

What to say:

- "Four settings carry the durability guarantees. `acks=all`: the leader waits for every in-sync replica, so an acked write survives leader failure. `enable.idempotence`: the producer attaches a producer-id and sequence number so the broker dedupes retries — this is what makes retrying *safe* rather than duplicate-generating. `linger.ms=20` trades 20 ms of latency for batching throughput, which is free on the producer side because it's off the decision path."
- "`enable.auto.commit=False` is the single most important line in the file. Auto-commit commits on a timer regardless of whether you processed the message — which means a crash silently loses transactions. Everything in Exhibit 5 depends on this being off."
- "`auto.offset.reset=earliest` means a brand-new consumer group replays from the beginning of retention. That's what makes 'replay the last 30 days through a new model' a real capability rather than a slide."
- "SASL is added conditionally, so **the same code runs against local Redpanda and Confluent Cloud** with no branch in application logic — just config."

Exhibit 4 — Idempotent topic provisioning

src/fraud/producer/create_topics.py

```

def create_topics() -> None:
    admin = AdminClient(base_conf())
    topics = [
        NewTopic(settings.kafka_raw_topic, num_partitions=6, replication_factor=1),
        NewTopic(settings.kafka_dlq_topic, num_partitions=1, replication_factor=1),
    ]
    futures = admin.create_topics(topics)
    for name, fut in futures.items():
        try:
            fut.result()
            log.info("topic_created", topic=name)
        except Exception as exc:
            if "already exists" in str(exc).lower():
                log.info("topic_exists", topic=name) # idempotent: not an error
            else:
                log.error("topic_error", topic=name, error=str(exc))

```

What to say:

- "make seed must be safe to run repeatedly, so 'already exists' is a success case, not a failure. Every seeding operation in this project is idempotent — topics, Mongo indexes, Neo4j constraints, case embeddings."
- "6 partitions on the raw topic caps consumer parallelism at 6, with headroom for the demo's single consumer. Partitions are cheap to add and painful to remove, so you size up-front from target throughput divided by per-consumer throughput. 1 partition on the DLQ because ordering and throughput are irrelevant there."
- "`replication_factor=1` is a single-broker demo value. Production is 3 with `min.insync.replicas=2`, which is what makes `acks=all` meaningful."

Exhibit 5 — The hot-path consumer loop (delivery semantics)

src/fraud/stream/job.py — **the most important code in the project**

```

def run() -> None:
    signal.signal(signal.SIGTERM, _handle_sigterm) # graceful shutdown
    signal.signal(signal.SIGINT, _handle_sigterm)

    mongo.init_indexes()
    consumer = Consumer(consumer_conf())
    dlq = Producer(producer_conf())
    consumer.subscribe([settings.kafka_raw_topic])

```

```

try:
    while _running:
        msg = consumer.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            raise KafkaException(msg.error())

        raw = msg.value()
        start = time.perf_counter()

        # --- 1. validate at the boundary; poison => DLQ, never crash ---
        try:
            event = TransactionEvent(**json.loads(raw))
        except Exception as exc:
            # poison message
            log.warning("poison_message_to_dlq", error=str(exc))
            _to_dlq(dlq, raw, str(exc))
            consumer.commit(msg)
            continue

        # --- 2. process; failure => DLQ (preserves raw bytes) ---
        try:
            scored = process_event(event)
        except Exception as exc:
            log.error("processing_failed", txn=event.txn_id, error=str(exc))
            _to_dlq(dlq, raw, str(exc))
            consumer.commit(msg) # avoid poison-loop; raw preserved in DLQ
            continue

        # --- 3. commit ONLY after a successful, idempotent write ---
        consumer.commit(msg)

        latency = (time.perf_counter() - start) * 1000
        _record_latency(latency)
        if scored.risk_band != scored.risk_band.LOW:
            log.info("flagged", txn=scored.id, band=scored.risk_band.value,
                    score=scored.risk_score, latency_ms=round(latency, 1))
    finally:
        consumer.close() # leaves the group cleanly => fast rebalance
        dlq.flush() # never lose a buffered DLQ message on shutdown

def _to_dlq(producer: Producer, raw: bytes, error: str) -> None:
    producer.produce(
        settings.kafka_dlq_topic,
        value=raw, # original bytes, unmodified
        headers=[("error", error.encode()[:200])], # why it failed, in a header
    )
    producer.poll(0)

```

What to say:

- **"The comment on line 3 of the commit block is the whole design.** `consumer.commit(msg)` happens *after* `process_event` returns successfully, and every write inside `process_event` is an idempotent upsert on a natural key. So: at-least-once delivery + idempotent effects = **effectively-once for the serving store**, and Kafka can be replayed safely."
- "If I committed *before* writing, a crash in between would silently drop a transaction. In fraud, a miss is the expensive error — so the ordering is not a style preference, it's the correctness argument."
- **"Two distinct failure paths, both non-fatal.** Malformed JSON and a processing exception both go to the DLQ with the raw bytes intact and the reason in a Kafka header, then commit and advance. One bad message can neither crash the consumer (availability) nor vanish (auditability)."
- **"Volunteer the flaw here (§7 #5):** I don't distinguish retryable from permanent failures. A Mongo timeout gets DLQ'd exactly like malformed JSON. It needs bounded retry with exponential backoff — *tenacity* is already in my manifest for this — and a circuit breaker, with the DLQ reserved for genuinely bad data. Also, `commit()` defaults to `asynchronous=True` in `confluent-kafka`, so **commit failures are unobserved**, and committing per message is a throughput tax versus batching."

- "The finally block matters: `consumer.close()` leaves the consumer group cleanly so the rebalance is fast instead of waiting for a session timeout, and `dlq.flush()` guarantees buffered DLQ messages aren't lost on shutdown. SIGTERM flips a flag rather than killing mid-write, so the loop finishes the message it's on."

Exhibit 6 — Per-event orchestration

`src/fraud/stream/job.py`

```
def process_event(event: TransactionEvent) -> ScoredTransaction:
    """Pure-ish per-event pipeline: features -> graph -> score -> persist."""
    state = _load_state(event.user_id)          # Mongo read
    features, new_state = compute_features(event, state)    # pure, testable
    features = enrich_with_graph(features, event.user_id)    # Neo4j lookup

    scorer = get_scorer()
    score, band = scorer.score(features)          # MLflow model + hot-reload

    # SHAP only for cases a human will actually open (~3% of traffic)
    shap_top: list[tuple[str, float]] = []
    if band != band.LOW:
        try:
            from fraud.ml.explain import top_features
            shap_top = top_features(scorer.model, features.to_vector())
        except Exception as exc:
            log.warning("shap_failed", error=str(exc))        # never fail the decision

    scored = ScoredTransaction(
        _id=event.txn_id, user_id=event.user_id, amount=event.amount,
        merchant=event.merchant,
        risk_score=round(score, 4), risk_band=band,
        features=features.model_dump(),
        graph=GraphInfo(ring_id=features.ring_id,
                        shared_device_flags=features.shared_device_flags,
                        component_size=features.component_size),
        shap_top=shap_top,
        geo=event.geo, device_id=event.device_id,
        model_version=scorer.version,            # provenance: which model decided this
    )

    mongo.upsert_scored(scored)
    mongo.upsert_user_state(new_state)
    if band != band.LOW:
        mongo.upsert_alert(Alert(
            _id=event.txn_id, txn_id=event.txn_id, user_id=event.user_id,
            risk_band=band, expected_loss=scored.expected_loss,
            status=CaseStatus.OPEN,
            priority=2 if band == band.HIGH else 1,
        ))
    _write_bronze(event)
    return scored
```

What to say:

- "Read the sequence and you can read the architecture: features, graph, score, explain, persist. **No LLM appears anywhere in this function** — that's the constraint, enforced by the code rather than by a convention."
- "`model_version=scorer.version` stamps **provenance** on every decision. When someone asks 'why did this transaction get flagged in March?', you need to know which model version made the call. It's also what the dashboard's Model metric reads, which is how you notice you're accidentally serving the heuristic fallback."
- "SHAP is wrapped in try/except deliberately: **an explanation failure must never fail a scoring decision**. Explanation is a nice-to-have on top of a decision that has to happen."
- "**The flaws I'd name here** (§6.2): this function does 3–4 sequential network round trips — one Mongo read, two-to-three upserts, and a Neo4j call. At 20 events/second it's invisible; it is also precisely my throughput ceiling and the answer to 'where's the bottleneck?'. And `_write_bronze` appends a line to a JSONL file, which is the one **non-idempotent** write in the function — so replay is safe for Mongo and duplicates rows in the lake."

Exhibit 7 — Pure rolling-window feature engineering

src/fraud/stream/features.py — the most testable code in the project

```
def compute_features(
    event: TransactionEvent,
    state: UserRollingState,
) -> tuple[Features, UserRollingState]:
    """Compute features for `event` given prior `state`.

    Returns the computed Features and an updated UserRollingState.
    The input state is not mutated.
    """
    ts = event.timestamp
    recent_ts = list(state.recent_ts)          # copy: never mutate the input
    recent_amounts = list(state.recent_amounts)

    # --- multi-horizon velocity ---
    def count_within(window_s: float) -> int:
        return sum(1 for t in recent_ts if 0 <= (ts - t).total_seconds() <= window_s)

    txn_1m = count_within(60) + 1              # +1 counts the current event
    txn_5m = count_within(300) + 1
    txn_1h = count_within(3600) + 1

    # --- amount z-score vs the user's own baseline (pre-update stats) ---
    mean = state.amount_mean
    std = state.amount_std
    if state.count < 2:
        amt_z = 0.0                            # no baseline yet: assert nothing
    elif std > 1e-6:
        amt_z = (event.amount - mean) / std
    else:
        # Zero-variance baseline: a large jump must still register. Fall back to a
        # relative deviation with a sane floor so outliers are not masked.
        denom = max(abs(mean) * 0.25, 1.0)
        amt_z = (event.amount - mean) / denom

    # --- novelty ---
    geo_key = f"{event.geo.country}:{round(event.geo.lat, 1)}:{round(event.geo.lon, 1)}"
    new_geo = geo_key not in set(state.seen_geos)
    new_device = event.device_id not in set(state.seen_devices)

    seconds_since_last = (
        (ts - state.last_ts).total_seconds() if state.last_ts is not None else 0.0
    )

    features = Features(
        amt_z=round(amt_z, 4),
        txn_1m=txn_1m, txn_5m=txn_5m, txn_1h=txn_1h,
        distinct_merchants_1h=len({event.merchant}),          # <-- always 1: see §7 #1
        distinct_geos_1h=max(len(set(state.seen_geos)), 1),
        seconds_since_last=round(seconds_since_last, 2),
        new_geo=new_geo, new_device=new_device,
    )

    # --- update state: Welford's online algorithm for running variance ---
    count = state.count + 1
    delta = event.amount - mean
    new_mean = mean + delta / count
    delta2 = event.amount - new_mean
    m2 = state.amount_m2 + delta * delta2

    # --- bounded history: last 200 events within 1h, 100 geos, 100 devices ---
    recent_ts.append(ts)
    recent_amounts.append(event.amount)
    kept = [(t, a) for t, a in zip(recent_ts, recent_amounts, strict=False)
            if (ts - t).total_seconds() <= 3600][-200:]

    new_state = UserRollingState(
        _id=state.id, count=count, amount_mean=new_mean, amount_m2=m2, last_ts=ts,
        seen_geos=seen_geos[-100:], seen_devices=seen_devices[-100:],
        recent_amounts=[a for _, a in kept], recent_ts=[t for t, _ in kept],
    )
```

```
return features, new_state
```

And the variance it maintains, on the state model:

```
class UserRollingState(BaseModel):
    id: str = Field(alias="_id")
    count: int = 0
    amount_mean: float = 0.0
    amount_m2: float = 0.0          # Welford's aggregate for variance
    last_ts: datetime | None = None
    seen_geos: list[str] = Field(default_factory=list)
    seen_devices: list[str] = Field(default_factory=list)
    recent_amounts: list[float] = Field(default_factory=list)
    recent_ts: list[datetime] = Field(default_factory=list)

    @property
    def amount_std(self) -> float:
        if self.count < 2:
            return 0.0
        return (self.amount_m2 / (self.count - 1)) ** 0.5
```

What to say:

- **"The signature is the design.** (event, state) -> (features, new_state), no Kafka, no Mongo, no mutation of the input. That's why the tests are plain dicts with no broker, no database and no mocks — and why the same logic is used by the stream, the tests, and the training-data contract."
- **"Welford's algorithm** gives a streaming standard deviation in $O(1)$ time and memory: carry count, mean and $M2$, and variance is $M2/(count-1)$. The naive $E[x^2] - E[x]^2$ subtracts two large nearly-equal numbers and loses precision catastrophically when the mean is large and the variance small — which is *exactly* the transaction-amount regime. It's also why the state document stays tiny: three floats instead of a history."
- **"The zero-variance branch is the bug I'm proudest of catching.** A user whose transactions have all been identical has `std == 0`, so the naive z-score divides by zero — and their first ₹50,000 charge would score as *not anomalous*, which is exactly backwards. The fallback is a relative deviation with a floor. That single `elif/else` is the difference between a formula and a feature."
- **"Volunteer the skew** (§7 #1): `distinct_merchants_1h` is `len({event.merchant})` — always 1 — while the training generator draws it from a Poisson. And `distinct_geos_1h` counts *all-time* distinct geos, so the name lies. That's live train/serve skew in my own code, and it's why feature stores exist: to make that class of bug structurally impossible rather than merely avoidable."
- "State is bounded on three axes — 200 recent events within an hour, 100 geos, 100 devices — so one user's document can't grow without limit. The cost is write amplification: I rewrite the whole document with `$set` per event, where atomic `$push` with `$slice` would update in place."

Exhibit 8 — Model hot-reload from the MLflow registry

src/fraud/stream/scorer.py

```
def band_for(score: float) -> RiskBand:
    if score >= settings.risk_high_threshold:      # 0.80
        return RiskBand.HIGH
    if score >= settings.risk_medium_threshold:    # 0.50
        return RiskBand.MEDIUM
    return RiskBand.LOW

class HeuristicScorer:
    """Fallback used before any model is trained/registered."""

    kind = "heuristic"

    def score_one(self, fv: list[float]) -> float:
        f = dict(zip(Features.feature_names(), fv, strict=False))
        s = 0.0
        s += min(abs(f["amt_z"]) / 10.0, 0.4)        # capped contributions
```

```

s += min(f["txn_1m"] / 10.0, 0.25)
s += 0.15 if f["new_geo"] else 0.0
s += 0.10 if f["new_device"] else 0.0
s += min(f["shared_device_flags"] / 10.0, 0.2)
return float(min(s, 1.0))

class ModelScorer:
    """Wraps an MLflow-loaded model with periodic hot-reload."""

    def __init__(self) -> None:
        self._model: Any = HeuristicScorer()      # always start usable
        self._version: str = "heuristic"
        self._loaded_at: float = 0.0
        self.reload(force=True)

    def reload(self, force: bool = False) -> None:
        if not force and (time.time() - self._loaded_at) < settings.model_reload_seconds:
            return                                # rate-limit the check
        self._loaded_at = time.time()
        try:
            import mlflow
            from mlflow import MlflowClient
            mlflow.set_tracking_uri(settings.mlflow_tracking_uri)

            client = MlflowClient()
            mv = client.get_model_version_by_alias(
                settings.mlflow_model_name, settings.mlflow_model_alias
            )
            if mv.version == self._version:
                return                                # unchanged: skip the download
            import mlflow.sklearn
            model = mlflow.sklearn.load_model(
                f"models://{settings.mlflow_model_name}@{settings.mlflow_model_alias}"
            )
            self._model = model                        # atomic-ish swap
            self._version = str(mv.version)
            log.info("model_loaded", version=self._version)
        except Exception as exc:
            if self._version == "heuristic":
                log.warning("model_unavailable_using_heuristic", error=str(exc))
            # otherwise: keep serving whatever we already had

    def score(self, features: Features) -> tuple[float, RiskBand]:
        self.reload()                                # checked, then rate-limited
        fv = features.to_vector()
        if hasattr(self._model, "score_one"):
            score = self._model.score_one(fv)
        else:
            score = float(self._model.score(np.asarray(fv, dtype=float))[0])
        return score, band_for(score)

```

What to say:

- **"The stream never references a model version — only the Production alias.** models:/fraud-scorer@Production. So promoting a model is a registry operation, not a code deploy: zero downtime, and rollback is re-pointing the alias, which takes seconds. That is the single most operationally valuable idea in the project."
- **"Two-stage cheapness:** the reload check is **time-gated** by model_reload_seconds, and even when it fires it **compares versions before downloading**. So the steady-state cost is one metadata call a minute, not a model download."
- **"Failure is non-destructive.** If MLflow is unreachable, the except keeps serving whatever model is already loaded — a tracking-server outage degrades your *deployment* capability, not your *scoring* capability. And it starts on the heuristic so the pipeline is observable end-to-end before any model exists."
- **"The hasattr(self._model, 'score_one')** duck-type is how one code path serves the heuristic and the MLflow-loaded wrapper interchangeably."
- **"Volunteer the risk:** behaviour can change with no code change and no deploy gate. That needs an audit trail on alias moves, an approval step, and ideally shadow scoring — run both models on 100% of traffic, act on the champion, compare offline. Also get_scorer() is an unguarded module singleton; I'd wrap the swap in a lock."

Exhibit 9 — Training, evaluation and the champion/challenger promotion gate

src/fraud/ml/train.py

```
class FraudScorer:
    """Uniform wrapper producing a fraud probability in [0, 1]."""

    def __init__(self, model, kind: str, score_min: float = 0.0, score_max: float = 1.0):
        self.model, self.kind = model, kind # "iforest" | "xgboost"
        self.score_min, self.score_max = score_min, score_max
        self.feature_names = Features.feature_names() # ordering travels with the model

    def score(self, x: np.ndarray) -> np.ndarray:
        x = np.atleast_2d(x)
        if self.kind == "iforest":
            # score_samples: higher = more normal. Invert + normalise to [0,1].
            raw = -self.model.score_samples(x)
            rng = (self.score_max - self.score_min) or 1.0
            return np.clip((raw - self.score_min) / rng, 0.0, 1.0)
        proba = self.model.predict_proba(x)[:, 1]
        return np.clip(proba, 0.0, 1.0)

def train_xgboost(x: np.ndarray, y: np.ndarray) -> FraudScorer:
    from xgboost import XGBClassifier
    pos = max(int(y.sum()), 1)
    neg = max(int(len(y) - y.sum()), 1)
    model = XGBClassifier(
        n_estimators=300, max_depth=5, learning_rate=0.1,
        scale_pos_weight=neg / pos, # imbalance: reweight the gradient
        eval_metric="aucpr", # optimise PR, not error rate
        n_jobs=2,
    )
    model.fit(x, y)
    return FraudScorer(model, "xgboost")

def evaluate(scorer: FraudScorer, x: np.ndarray, y: np.ndarray) -> dict[str, float]:
    from sklearn.metrics import average_precision_score, precision_score, recall_score
    scores = scorer.score(x)
    preds = (scores >= settings.risk_high_threshold).astype(int)
    return {
        "pr_auc": float(average_precision_score(y, scores)) if y.sum() else 0.0,
        "precision": float(precision_score(y, preds, zero_division=0)),
        "recall": float(recall_score(y, preds, zero_division=0)),
        "flag_rate": float(preds.mean()), # operational, not statistical
    }

def train_and_register(model_kind: str = "auto") -> dict[str, Any]:
    """Train a model, log + register to MLflow, alias Production if it wins."""
    import mlflow
    df = make_training_frame(n=8000, fraud_ratio=0.03)
    x = df[Features.feature_names()].to_numpy(dtype=float)
    y = df["label"].to_numpy(dtype=int)

    # No labels yet -> unsupervised. Enough labels -> supervised. Mirrors the real lifecycle.
    use_supervised = model_kind == "xgboost" or (model_kind == "auto" and y.sum() >= 50)
    scorer = train_xgboost(x, y) if use_supervised else train_isolation_forest(x)

    metrics = evaluate(scorer, x, y) # <-- IN-SAMPLE: see §7 #2

    mlflow.set_tracking_uri(settings.mlflow_tracking_uri)
    mlflow.set_experiment(settings.mlflow_experiment)
    with mlflow.start_run() as run:
        mlflow.log_param("kind", scorer.kind)
        mlflow.log_metrics(metrics)
        import mlflow.sklearn
        mlflow.sklearn.log_model(scorer, artifact_path="model")
        model_uri = f"runs:{run.info.run_id}/model"
        result = _register_and_promote(mlflow, model_uri, metrics)
    return {"kind": scorer.kind, "metrics": metrics, **result}
```

```
def _register_and_promote(mlflow, model_uri: str, metrics: dict[str, float]) -> dict[str, Any]:
    """Register the model; promote to Production alias if it beats the incumbent."""
    from mlflow import MlflowClient
    client = MlflowClient()
    name = settings.mlflow_model_name
    mv = mlflow.register_model(model_uri, name)

    incumbent_pr = _incumbent_metric(client, name, "pr_auc") # -1.0 if none exists
    challenger_pr = metrics.get("pr_auc", 0.0)
    promote = challenger_pr >= incumbent_pr # <-- ties promote
    if promote:
        client.set_registered_model_alias(name, settings.mlflow_model_alias, mv.version)
        log.info("promoted", version=mv.version)
    else:
        log.info("held_challenger", version=mv.version, incumbent_pr=incumbent_pr)
    return {"version": mv.version, "promoted": promote}
```

What to say:

- "FraudScorer is a **uniform wrapper**: whatever the underlying model, `score()` returns a value in [0,1]. That's what lets the same stream code serve an IsolationForest or an XGBoost classifier, and it's why the anomaly score gets inverted and min-max normalised — `score_samples` returns *higher = more normal*, which is backwards for a risk score."
- "Imbalance is handled with two settings, not with resampling. `scale_pos_weight=neg/pos` reweights the gradient so the minority class isn't drowned, and `eval_metric='aucpr'` optimises the precision-recall curve. I deliberately avoided SMOTE — synthesising fraud samples in feature space is dubious when fraud is adversarial and multimodal; you invent plausible-looking frauds that don't exist."
- "flag_rate sits next to precision and recall because it's the **operational** metric: it tells you whether your review team can survive this model. A 0.02 improvement in PR-AUC that triples the queue is not an improvement."
- "The `y.sum() >= 50` switch mirrors the real fraud-ML lifecycle: day one you have no labels, so unsupervised anomaly detection is the only honest option; once labels accumulate, supervised learns *fraud* rather than merely *unusual*."
- "The promotion gate is architecturally right and statistically wrong, and I'd say so first (§7 #2). `evaluate()` runs on the same arrays it trained on — no holdout, no CV — so the gate compares two optimistic numbers, and `>=` means ties promote. The fix isn't just 'add a split', it's a **temporal** split, because fraud has time structure and a random split leaks future information into the past."

Exhibit 10 — PSI drift detection and drift-triggered retraining

src/fraud/ml/drift.py

```
PSI_THRESHOLD = 0.2 # >0.2 = significant population shift

def psi(expected: np.ndarray, actual: np.ndarray, buckets: int = 10) -> float:
    """Population Stability Index between two 1-D distributions."""
    quantiles = np.linspace(0, 100, buckets + 1)
    edges = np.percentile(expected, quantiles) # decile bins from the REFERENCE
    edges[0], edges[-1] = -np.inf, np.inf # catch out-of-range values
    e_perc = np.histogram(expected, bins=edges)[0] / max(len(expected), 1)
    a_perc = np.histogram(actual, bins=edges)[0] / max(len(actual), 1)
    e_perc = np.clip(e_perc, 1e-6, None) # avoid log(0) / div-by-0
    a_perc = np.clip(a_perc, 1e-6, None)
    return float(np.sum((a_perc - e_perc) * np.log(a_perc / e_perc)))

def compute_drift(reference, current) -> dict[str, float]:
    """Per-feature PSI between a reference and current feature frame."""
    names = Features.feature_names()
    return {n: round(psi(reference[n].to_numpy(), current[n].to_numpy()), 4) for n in names}

def run_drift_check(trigger_retrain: bool = True) -> dict[str, object]:
    reference = make_training_frame(n=6000, fraud_ratio=0.03, seed=1)
    # simulate an adversarial shift (attackers adapt): heavier fraud + higher amounts
    current = make_training_frame(n=6000, fraud_ratio=0.08, seed=99)
    current["amt_z"] = current["amt_z"] + 1.5
```

```

scores = compute_drift(reference, current)
drifted = {k: v for k, v in scores.items() if v > PSI_THRESHOLD}
log.info("drift_scores", **scores)

result = {"psi": scores, "drifted_features": list(drifted)}
if drifted and trigger_retrain:
    log.warning("drift_detected_triggering_retrain", features=list(drifted))
    from fraud.ml.train import train_and_register
    result["retrain"] = train_and_register("auto")    # closes the loop
return result

```

What to say:

- "PSI is a **symmetrised KL divergence over bins**: bin the reference into deciles, compare bin proportions, sum $(\text{actual}\% - \text{expected}\%) \times \ln(\text{actual}\%/\text{expected}\%)$. Conventional thresholds are <0.1 stable, 0.1–0.25 moderate, >0.25 significant. I use 0.2, and I chose PSI specifically because it's the retail-credit-risk convention — **the number already means something to a risk team**, which a Wasserstein distance doesn't."
- "Two details that matter: bins come from the **reference** distribution, not the current one, otherwise you're measuring nothing; and the $\pm\text{inf}$ edges plus the $1\text{e-}6$ clip handle out-of-range values and empty bins, which is where a naive PSI implementation divides by zero."
- "It's **per-feature**, not global. That's deliberate — a single drift number tells you something broke; a PSI vector tells you *which feature* broke, which is the difference between an alert and a diagnosis."
- "I hand-rolled it in ten lines rather than pulling in Evidently, because I wanted to be able to explain the maths rather than call a library. For production I'd use Evidently — more tests, reports and dashboards for free."
- "**The two honest caveats**: it runs on synthetic data with a *simulated* adversarial shift (higher fraud ratio, `amt_z + 1.5`), so it proves the maths and the drift-triggers-retrain wiring, not production monitoring. And PSI catches **input** drift only. The one that actually kills you is **concept** drift — inputs stable, but the feature-to-fraud relationship changes because fraudsters adapt. Catching that needs labels, and mine arrive delayed from analyst review, so it needs a delayed-label evaluation harness."

Exhibit 11 — SHAP explanations with a graceful fallback

src/fraud/ml/explain.py

```

def top_features(
    model: Any,
    feature_vector: list[float],
    feature_names: list[str] | None = None,
    top_n: int = 5,
) -> list[tuple[str, float]]:
    """Return the top-N (feature, signed_contribution) pairs via SHAP.

    Falls back to a simple magnitude ranking if SHAP cannot explain the model.
    """
    names = feature_names or Features.feature_names()
    x = np.asarray(feature_vector, dtype=float).reshape(1, -1)
    try:
        import shap
        explainer = _build_explainer(model, x)
        values = explainer(x)
        contribs = np.asarray(values.values).reshape(-1)
        pairs = sorted(zip(names, contribs.tolist(), strict=False),
                       key=lambda t: abs(t[1]), reverse=True)    # rank by magnitude,
        return [(n, round(float(v), 4)) for n, v in pairs[:top_n]]    # keep the sign
    except Exception as exc:
        log.warning("shap_fallback", error=str(exc))
        pairs = sorted(zip(names, x.reshape(-1).tolist(), strict=False),
                       key=lambda t: abs(t[1]), reverse=True)
        return [(n, round(float(v), 4)) for n, v in pairs[:top_n]]

def _build_explainer(model: Any, background: np.ndarray):
    import shap
    inner = getattr(model, "model", model)    # unwrap FraudScorer -> sklearn/xgb model

```

```

try:
    return shap.TreeExplainer(inner)          # exact + fast for tree ensembles
except Exception:
    predict = getattr(model, "score", None) or getattr(inner, "predict", None)
    return shap.Explainer(predict, background) # model-agnostic fallback

```

What to say:

- "SHAP values are **Shapley values from cooperative game theory**: a feature's attribution is its average marginal contribution to the prediction across all orderings of the features. That buys additivity — contributions sum to the difference between this prediction and the base value — plus local accuracy and consistency. TreeExplainer computes them *exactly* for tree ensembles in polynomial rather than exponential time, which is the only reason this is viable on a latency budget at all."
- "**Ranked by absolute value, returned with the sign.** The analyst needs to know which features mattered most *and* in which direction — a large negative contribution is exculpatory evidence, and dropping the sign would throw that away."
- "getattr(model, 'model', model) unwraps my FraudScorer to reach the raw estimator, because SHAP needs the actual tree ensemble, not my wrapper."
- "**The honest problem** (§7, §8.7): TreeExplainer support for IsolationForest is unreliable, so in unsupervised mode this can silently fall into the magnitude-ranking fallback — and that fallback is **not an attribution at all**, it's just 'which feature value is largest'. It's currently displayed in the UI as if it were SHAP output. That's an honesty bug and I'd fix it by labelling the explanation method in the document. Also, the explainer is rebuilt on every call instead of cached with the loaded model — and arguably explanation should move off the hot path entirely and be computed lazily when an analyst opens the case, since nobody reads 97% of explanations."

Exhibit 12 — Neo4j graph model, ring detection and lookup

src/fraud/graph/loader.py

```

def ensure_constraints() -> None:
    stmts = [
        "CREATE CONSTRAINT user_id IF NOT EXISTS FOR (u:User) REQUIRE u.id IS UNIQUE",
        "CREATE CONSTRAINT device_id IF NOT EXISTS FOR (d:Device) REQUIRE d.id IS UNIQUE",
        "CREATE CONSTRAINT merchant_id IF NOT EXISTS FOR (m:Merchant) REQUIRE m.id IS UNIQUE",
        "CREATE CONSTRAINT ip_id IF NOT EXISTS FOR (i:IP) REQUIRE i.id IS UNIQUE",
    ]
    with _session() as s:
        for stmt in stmts:
            s.run(stmt)

def upsert_transaction_edges(user_id, device_id, merchant, ip, flagged=False) -> None:
    """Idempotently MERGE the entity graph for one transaction."""
    query = """
    MERGE (u:User {id: $user_id})
    ON CREATE SET u.flagged = $flagged
    ON MATCH SET u.flagged = u.flagged OR $flagged
    MERGE (d:Device {id: $device_id})
    MERGE (m:Merchant {id: $merchant_id})
    MERGE (i:IP {id: $ip})
    MERGE (u)-[:USED]->(d)
    MERGE (u)-[:PAID]->(m)
    MERGE (u)-[:FROM_IP]->(i)
    """
    with _session() as s:
        s.run(query, user_id=user_id, device_id=device_id,
            merchant=merchant, ip=ip, flagged=flagged)

def query_graph(user_id: str) -> GraphInfo:
    """Return ring / shared-entity info for a user (features + agent tool)."""
    if not settings.neo4j_configured:
        return GraphInfo() # graceful degradation to zeros
    query = """
    MATCH (u:User {id: $user_id})
    OPTIONAL MATCH (u)-[:USED]->(d:Device)-[:USED]-(other:User)
    WHERE other.id <> u.id AND other.flagged = true
    """

```

```

RETURN u.ring_id AS ring_id,
       coalesce(u.component_size, 0) AS component_size,
       count(DISTINCT other) AS shared_device_flags
"""
try:
    with _session() as s:
        rec = s.run(query, user_id=user_id).single()
    if rec is None:
        return GraphInfo()
    return GraphInfo(ring_id=rec["ring_id"],
                    component_size=int(rec["component_size"] or 0),
                    shared_device_flags=int(rec["shared_device_flags"] or 0))
except Exception as exc:
    log.warning("query_graph_failed", error=str(exc))
    return GraphInfo() # never fail a scoring decision

```

Community detection, two engines behind one function:

```

def compute_communities() -> dict[str, Any]:
    """Assign ring_id / component_size via GDS or a networkx fallback."""
    if not settings.neo4j_configured:
        return {"status": "skipped", "reason": "neo4j_not_configured"}
    if settings.neo4j_use_gds:
        return _compute_communities_gds() # in-database, scales
    return _compute_communities_networkx() # for tiers without GDS

def _compute_communities_gds() -> dict[str, Any]:
    project = """
    CALL gds.graph.project.cypher(
        'fraud-graph',
        'MATCH (n) WHERE n:User OR n:Device OR n:Merchant OR n:IP RETURN id(n) AS id',
        'MATCH (a)--(b) RETURN id(a) AS source, id(b) AS target'
    ) YIELD graphName
    """
    write = """
    CALL gds.wcc.write('fraud-graph', {writeProperty: 'ring_component'})
    YIELD componentCount RETURN componentCount
    """
    with _session() as s:
        s.run(project).consume()
        rec = s.run(write).single()
        s.run("CALL gds.graph.drop('fraud-graph', false) ...").consume()
        s.run("""
            MATCH (u:User)
            WITH u.ring_component AS comp, collect(u) AS users
            UNWIND users AS u
            SET u.ring_id = 'R_' + toString(comp), u.component_size = size(users)
            """).consume()
    return {"status": "ok", "engine": "gds", "components": rec["componentCount"]}

def _compute_communities_networkx() -> dict[str, Any]:
    import networkx as nx
    g = nx.Graph()
    with _session() as s:
        rows = s.run("MATCH (a)--(b) WHERE a.id IS NOT NULL AND b.id IS NOT NULL "
                    "RETURN labels(a)[0]+'_'+a.id AS a, labels(b)[0]+'_'+b.id AS b")
        for r in rows:
            g.add_edge(r["a"], r["b"])

    updates = []
    for idx, comp in enumerate(nx.connected_components(g)):
        for node in comp:
            if node.startswith("User:"):
                updates.append({"uid": node.split(":", 1)[1],
                               "ring": f"R_{idx}", "size": len(comp)})

    with _session() as s:
        s.run("""UNWIND $updates AS row
            MATCH (u:User {id: row.uid})
            SET u.ring_id = row.ring, u.component_size = row.size""",
            updates=updates) # one batched round trip, not N
    return {"status": "ok", "engine": "networkx", "users_updated": len(updates)}

```

What to say:

- "It's an **entity-resolution graph**: users aren't linked to each other directly, they're linked *through* shared attributes — device, merchant, IP. That means a fraud ring **emerges as a connected component** rather than needing to be asserted by anyone. That's the whole reason a graph database earns its place here."
- "MERGE everywhere makes ingestion idempotent — replay the same transaction and the graph is unchanged. The ON CREATE / ON MATCH pair on the flagged property is a monotonic OR: once a user is flagged they stay flagged, so re-ingesting a clean transaction can't clear the flag."
- "query_graph is doing three things in one Cypher statement: read the precomputed ring_id and component_size, and live-count how many **other flagged users** share a device. The OPTIONAL MATCH means a user with no shared devices still returns a row instead of nothing — the difference between 'no ring' and 'no answer'."
- "WCC rather than Louvain because for 'does this user share infrastructure with known-fraud accounts', **reachability is exactly the right semantics** and the result is trivially explainable to an analyst. I'd name the failure mode: at real volume WCC collapses into one giant component, because everyone eventually shares *an* IP. That's when you move to Louvain for modularity-based communities, or add edge weights and thresholds."
- "Two engines, one interface: GDS runs the algorithm in-database on a projected graph and scales; the networkx path exports edges and computes locally, for Aura tiers without GDS. Note the UNWIND \$updates write-back — one batched round trip instead of N."
- "**Volunteer the two flaws** (§7 #1, #4): the stream never calls upsert_transaction_edges — only the demo seeder does — so graph features are zero for every real streamed user. And query_graph runs **synchronously per event against a cloud cluster**, which is the most expensive thing on my hot path. Ring membership changes hourly and I'm querying it per millisecond; it should be cached with a TTL or precomputed into the serving store."

Exhibit 13 — Idempotent serving writes, indexes and vector search

src/fraud/serving/mongo.py

```
def init_indexes() -> None:
    """Create indexes; idempotent."""
    flagged().create_index([("user_id", ASCENDING)]) # history lookups
    flagged().create_index([("risk_band", ASCENDING)]) # alert filtering
    flagged().create_index([("created_at", DESCENDING)]) # recency
    flagged().create_index([("status", ASCENDING)]) # review workflow
    alerts().create_index([("status", ASCENDING), ("priority", DESCENDING)]) # compound

# --- idempotent writes: the foundation of replay safety -----

def upsert_scored(txn: ScoredTransaction) -> None:
    doc = txn.to_mongo()
    flagged().update_one({"_id": doc["_id"]}, {"$set": doc}, upsert=True)

def upsert_user_state(state: UserRollingState) -> None:
    doc = state.model_dump(by_alias=True, mode="json")
    user_state().update_one({"_id": doc["_id"]}, {"$set": doc}, upsert=True)

def set_label(txn_id: str, label: bool) -> None:
    """Analyst decision -> training label."""
    flagged().update_one(
        {"_id": txn_id},
        {"$set": {"label": label, "status": "closed", "labeled_at": _now_iso()}},
    )

# --- vector search: managed index or local fallback -----

def vector_search(query: list[float], k: int = 3) -> list[dict[str, Any]]:
    if settings.mongo_use_atlas_vector:
        return _atlas_vector_search(query, k)
    return _bruteforce_search(query, k)
```

```
def _atlas_vector_search(query: list[float], k: int) -> list[dict[str, Any]]:
    pipeline = [
        {"$vectorSearch": {
            "index": "case_vector_index", "path": "vector",
            "queryVector": query, "numCandidates": 100, "limit": k,
        }},
        {"$project": {"vector": 0, "score": {"$meta": "vectorSearchScore"}}},
    ]
    return list(case_embeddings().aggregate(pipeline))

def _bruteforce_search(query: list[float], k: int) -> list[dict[str, Any]]:
    q = np.asarray(query, dtype=float)
    qn = np.linalg.norm(q) + 1e-9
    scored = []
    for doc in case_embeddings().find({}): # O(n) full scan: see §7 #10
        v = np.asarray(doc.get("vector", []), dtype=float)
        if v.size != q.size:
            continue # dimension guard
        sim = float(np.dot(q, v) / (qn * (np.linalg.norm(v) + 1e-9)))
        d = {kk: vv for kk, vv in doc.items() if kk != "vector"}
        d["score"] = sim
        scored.append((sim, d))
    scored.sort(key=lambda t: t[0], reverse=True)
    return [d for _, d in scored[:k]]
```

What to say:

- **"Every write is update_one(..., upsert=True) on the natural key.** `_id == txn_id` for cases, `_id == user_id` for state. That single pattern is what makes the whole at-least-once story work: reprocess the same event and the second write is a no-op, so Kafka replay is free. It also means no separate dedupe table and no uniqueness constraint to maintain."
- "Each index maps to a real query: `user_id` for the history the LLM and agent read, `risk_band` for alert filtering, `created_at` descending for recency, `status` for the review workflow. The alerts index is **compound** (`status`, `priority`) because the query filters on `status` and sorts on `priority` — prefix ordering matters, and getting it backwards means the index isn't used for the sort."
- **"Volunteer the missing one:** there's no index on `expected_loss`, which is my hottest sort key, because I compute it in Python instead of storing it. `top_alerts` therefore loads every high/medium document into process memory and sorts there, every three seconds. It should be an aggregation, or better, persist `expected_loss` at write time and index it."
- "Two vector paths behind one function: Atlas `$vectorSearch` is HNSW-backed with `numCandidates` controlling the recall/latency trade-off, and note `{'vector': 0}` in the projection so I don't ship 384 floats per hit back over the wire. The fallback is brute-force cosine — correct at `n=5`, indefensible past a few thousand, because it re-reads every vector from the database on every query."

Exhibit 14 — Grounded prompt templates

`src/fraud/assist/prompts.py`

```
SYSTEM_RULES = (
    "You are a fraud analyst assistant. You may ONLY assert facts supported by the "
    "provided SHAP features, user history, and retrieved cases. Cite retrieved case "
    "ids in square brackets like [CASE_017]. If evidence is insufficient, say so "
    "explicitly. Keep the summary under 120 words. Do not invent data."
)

RAG_PROMPT = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_RULES),
    ("human",
        "Flagged transaction:\n{txn}\n\n"
        "Top SHAP feature attributions:\n{shap}\n\n"
        "Retrieved similar resolved cases and policy snippets:\n{context}\n\n"
        "Summarise the risk, grounding every claim in the SHAP features or the "
        "retrieved cases, and cite the case ids you rely on."),
])
```

```
# Plain (non-chat) template used by local HuggingFace fine-tuned pipelines.
CASE_SUMMARY_TEXT = PromptTemplate.from_template(
    """## Fraud analyst summary\n
    "Transaction: {txn}\nSHAP: {shap}\nHistory: {history}\nRationale:"
)

```

What to say:

- "Four constraints, each doing a specific job. **'ONLY assert facts supported by'** scopes the model to the evidence. **'Cite ids in square brackets'** makes grounding *machine-checkable* — I extract those brackets in code, which is what turns grounding from a request into a measurement. **'If evidence is insufficient, say so explicitly'** gives the model a licence to abstain, which is the cheapest hallucination control there is. And **'under 120 words'** is an operational constraint: an analyst triaging a queue will not read a page."
- "SYSTEM_RULES is a single constant reused by the summary chain, the RAG chain, and the fine-tuning dataset export — so the model is fine-tuned under exactly the system prompt it will be served under. Train/serve consistency applies to prompts too, and that's an easy thing to get wrong."
- "The separate non-chat PromptTemplate exists because a local LoRA'd distilgpt2 isn't a chat model — it has no notion of system/human roles. Same information, different surface."

Exhibit 15 — The RAG chain (LCEL) with citation extraction

src/fraud/assist/chains.py

```
def _extract_cited_ids(text: str) -> list[str]:
    return sorted(set(re.findall(r"\[([A-Z0-9_]+\])]", text)))

def rag_summary(
    txn_doc: dict[str, Any],
    history: list[dict[str, Any]] | None = None,
) -> CaseSummary:
    """RAG chain: retrieve similar resolved cases + policy, ground with citations."""
    configure_tracing()  # LangSmith on, or no-op

    query = _fmt_txn(txn_doc) + " | " + _fmt_shap(txn_doc.get("shap_top", []))
    cases = similar_cases(query, k=3)  # vector search
    context = format_context(cases)  # "[CASE_017] (sim=0.91) ..."

    llm = get_llm()  # provider-agnostic
    chain = RAG_PROMPT | llm | StrOutputParser()  # LCEL composition
    text = chain.invoke({
        "txn": _fmt_txn(txn_doc),
        "shap": _fmt_shap(txn_doc.get("shap_top", [])),
        "context": context,
    })

    cited = _extract_cited_ids(text) or [c.get("_id") for c in cases]
    return CaseSummary(
        txn_id=txn_doc.get("_id", "?"),
        summary=text.strip(),
        cited_case_ids=[c for c in cited if c],
        grounded=bool(cases),  # grounded == "real context was retrieved"
        model=describe_provider(),  # provenance, shown in the UI
    )

```

And how retrieved context is rendered for the prompt (assist/retriever.py):

```
def format_context(cases: list[dict[str, Any]]) -> str:
    """Render retrieved cases into a cited context block for prompts."""
    lines = []
    for c in cases:
        lines.append(f"[{c.get('_id', '?')}] (sim={c.get('score', 0.0):.2f}) {c.get('text', '')}")
    return "\n".join(lines) if lines else "No similar cases found."

def embed(text: str) -> list[float]:
    """Return an embedding vector for `text` (real model or hashing fallback)."""

```

```

model = _model()                                # SentenceTransformer or None
if model is not None:
    return model.encode(text, normalize_embeddings=True).tolist()
return _hash_embed(text, settings.embedding_dim)  # deterministic offline fallback

```

What to say:

- "The chain is **LCEL**: `RAG_PROMPT | llm | StrOutputParser()`. Declarative composition, and because it's a Runnable it gets streaming, batching, retries and LangSmith tracing without me writing any of that."
- "**Citations are extracted by code, not trusted from prose.** The regex pulls `[CASE_017]` out of the response into a typed field on `CaseSummary`, so grounding becomes an observable property of the output — something the eval harness can score and the UI can display. That is the difference between 'I asked it not to hallucinate' and an engineering control."
- "`_extract_cited_ids(text)` or `[c['_id'] for c in cases]` — if the model cited nothing, fall back to recording what was *retrieved*, so the case file always shows the evidence the summary was generated from, even when the model was sloppy."
- "`describe_provider()` stamps which model wrote the summary. Same instinct as `model_version` on the scored transaction: **provenance on every generated artefact.**"
- "**Be precise about the limit:** extraction proves the model cited *something*, not that the citation supports the claim. It's a necessary condition, not a sufficient one. The honest upgrade is claim-level entailment checking against the retrieved text — an LLM-as-judge with a rubric, and the judge itself validated against human labels first."
- "Note the embedding fallback: `all-MiniLM-L6-v2` at 384 dimensions when sentence-transformers is available, otherwise a deterministic hashing embedder. Retrieval quality degrades; the pipeline and the tests keep working with no model download."

Exhibit 16 — Provider abstraction: the one seam

`src/fraud/assist/llm.py` — this is where the fine-tuned model joins the system

```

def get_llm(temperature: float = 0.1) -> Any:
    """Return a LangChain LLM/chat model for the configured provider."""
    provider = settings.llm_provider

    if provider == "openai" and settings.openai_api_key:
        from langchain_openai import ChatOpenAI
        return ChatOpenAI(model=settings.llm_model,
                           api_key=settings.openai_api_key, temperature=temperature)

    if provider == "anthropic" and settings.anthropic_api_key:
        from langchain_anthropic import ChatAnthropic
        return ChatAnthropic(model=settings.llm_model,
                              api_key=settings.anthropic_api_key, temperature=temperature)

    if provider == "finetuned" and settings.finetuned_model_id:
        return _get_finetuned_llm(temperature)

    # default: offline deterministic fallback (tests + no-key demos)
    from langchain_community.llms import FakeListLLM
    return FakeListLLM(responses=_FAKE_RESPONSES)

def _get_finetuned_llm(temperature: float) -> Any:
    """Serve the fine-tuned analyst model (hosted OpenAI ft or local LoRA)."""
    mid = settings.finetuned_model_id
    if mid.startswith("ft:") and settings.openai_api_key:
        from langchain_openai import ChatOpenAI
        return ChatOpenAI(model=mid, api_key=settings.openai_api_key,
                           temperature=temperature)
    if _is_finetuned_local():
        return _load_local_finetuned(mid, temperature)
    log.warning("finetuned_unavailable_falling_back_to_fake", model_id=mid)
    from langchain_community.llms import FakeListLLM
    return FakeListLLM(responses=_FAKE_RESPONSES)

```

```
def _load_local_finetuned(adapter_path: str, temperature: float) -> Any:
    """Load a local base model + LoRA adapter and wrap as a LangChain pipeline."""
    from langchain_community.llms import HuggingFacePipeline
    from peft import PeftModel
    from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

    base = settings.finetune_base_model
    tokenizer = AutoTokenizer.from_pretrained(base)
    model = AutoModelForCausalLM.from_pretrained(base)
    model = PeftModel.from_pretrained(model, adapter_path) # base + adapter
    gen = pipeline("text-generation", model=model, tokenizer=tokenizer,
                  max_new_tokens=160, temperature=max(temperature, 0.01),
                  do_sample=temperature > 0)
    return HuggingFacePipeline(pipeline=gen)

def describe_provider() -> str:
    """Human-readable provider label for the dashboard ops panel."""
    if settings.llm_provider == "finetuned" and settings.finetuned_model_id:
        kind = "openai-ft" if settings.finetuned_model_id.startswith("ft:") else "local-lora"
        return f"finetuned ({kind})"
    if settings.llm_enabled:
        return f"{settings.llm_provider}:{settings.llm_model}"
    return "fake (offline)"
```

What to say:

- **"This one function is the most valuable abstraction in the project.** Four providers — fake, OpenAI, Anthropic, fine-tuned — behind a single factory. Consequences: tests run offline with zero network, the demo works with no keys, and **the fine-tuned model joins the system without a single chain being modified.** LLM_PROVIDER=finetuned and the RAG chain, the summary chain and the agent all use it."
- "temperature=0.1 because this is analytical summarisation, not creative writing — I want near-deterministic, evidence-tracking output."
- "Note that finetuned routes two ways off one config value: a hosted ft: id goes through ChatOpenAI, a filesystem path loads a base model plus a LoRA adapter through PeftModel and wraps it as a HuggingFacePipeline. **Same interface, radically different runtime** — that's the abstraction earning its keep."
- "All the heavy imports are lazy, inside the branches. Nobody pays torch import time to run the offline fake path, and the finetune extra can be entirely absent without breaking the import graph."
- **"Volunteer the manifest bug (§7 #7):** langchain_anthropic is imported here but is **not** in pyproject.toml, so LLM_PROVIDER=anthropic fails with an ImportError on a clean install. That's a missing runtime dependency for a documented code path, and I should have caught it."

Exhibit 17 — The investigation agent: tools, plus a deterministic fallback

src/fraud/assist/agent.py

```
@tool
def query_graph_tool(user_id: str) -> str:
    """Return ring / shared-device info for a user from the Neo4j graph."""
    info = query_graph(user_id)
    return (f"ring_id={info.ring_id}, component_size={info.component_size}, "
          f"shared_device_flags={info.shared_device_flags}")

@tool
def get_user_history_tool(user_id: str) -> str:
    """Return the user's recent transactions from the serving store."""
    hist = mongo.get_user_history(user_id, limit=5)
    if not hist:
        return "no prior transactions"
    return "; ".join(f"{h.get('amount')} at {h.get('merchant')} "
                    f"(score {h.get('risk_score')})" for h in hist)

@tool
```

```

def similar_cases_tool(query: str) -> str:
    """Return similar resolved fraud cases for the given transaction description."""
    return format_context(similar_cases(query, k=3))

TOOLS = [query_graph_tool, get_user_history_tool, similar_cases_tool]

def _supports_tool_calling() -> bool:
    if settings.llm_provider in {"openai", "anthropic"} and settings.llm_enabled:
        return True
    # OpenAI-hosted fine-tuned models support tool calling; local LoRA does not.
    if settings.finetuned_enabled and settings.finetuned_model_id.startswith("ft:"):
        return True
    return False

def investigate(user_id: str, question: str, txn_doc=None) -> dict[str, Any]:
    configure_tracing()
    if _supports_tool_calling():
        return _run_tool_calling_agent(user_id, question)
    return _run_deterministic(user_id, question, txn_doc)

def _run_tool_calling_agent(user_id: str, question: str) -> dict[str, Any]:
    from langchain.agents import AgentExecutor, create_tool_calling_agent

    prompt = ChatPromptTemplate.from_messages([
        ("system",
         "You are a fraud investigation agent. Use the tools to gather graph, "
         "history and precedent evidence. Ground every claim in tool output and "
         "cite case ids in [brackets]. Be concise."),
        ("human", "User under review: {user_id}\nQuestion: {question}"),
        ("placeholder", "{agent_scratchpad}"),
    ])
    agent = create_tool_calling_agent(get_llm(), TOOLS, prompt)
    executor = AgentExecutor(agent=agent, tools=TOOLS, verbose=False, max_iterations=6)
    result = executor.invoke({"user_id": user_id, "question": question})
    return {"answer": result.get("output", ""), "mode": "tool_calling_agent",
            "model": describe_provider()}

def _run_deterministic(user_id: str, question: str, txn_doc) -> dict[str, Any]:
    """Offline-safe orchestration: call every tool, compose a grounded answer."""
    graph = query_graph_tool.invoke({"user_id": user_id})
    history = get_user_history_tool.invoke({"user_id": user_id})
    query = question
    if txn_doc:
        query = f"{txn_doc.get('merchant')} amount {txn_doc.get('amount')} {question}"
    cases = similar_cases_tool.invoke({"query": query})

    info = query_graph(user_id)
    in_ring = info.shared_device_flags > 0 or (info.component_size or 0) > 2
    verdict = (
        f"User {user_id} appears to be part of ring {info.ring_id} "
        f"(component size {info.component_size}, {info.shared_device_flags} "
        f"shared-device links to other flagged users)."
        if in_ring else f"No strong ring signal for user {user_id}."
    )
    return {"answer": f"{verdict}\n\nGraph: {graph}\nRecent history: {history}\n"
            f"Precedent cases:\n{cases}",
            "mode": "deterministic", "model": describe_provider()}

```

What to say:

- "Three tools, and each one is a **docstring plus a function** — the `@tool` decorator turns the docstring into the description the model uses to decide when to call it. So the docstring is a prompt, not a comment, and writing it sloppily degrades tool selection."
- "The tools deliberately return **strings, not objects**. The consumer is a language model, so a compact `ring_id=R_2, component_size=5` line is the right interface; JSON would just burn tokens."

- "_supports_tool_calling() encodes a real distinction: **tool calling is a model capability, not a framework feature**. distilgpt2 cannot emit a tool call, and neither can FakeListLLM. Rather than have the feature disappear offline, I made the orchestration explicit — call all three tools, apply a stated ring rule, compose the answer."
- "The deterministic path is arguably *better* for this specific task: a fixed three-step investigation is deterministic, auditable and cheap, and an LLM adds no value in choosing the order. **The honest framing is that my 'agent' doesn't need to be an agent for this question** — and if the investigation genuinely branched, my fallback is a hand-rolled state machine and **LangGraph is the principled version** of it."
- "max_iterations=6 is a cost, latency and runaway-loop bound — enough for three tools plus a couple of refinements."

Exhibit 18 — The closed human-feedback loop: capture → dataset → LoRA → serve

src/fraud/assist/tracing.py and src/fraud/assist/finetune.py

```
# --- 1. CAPTURE: analyst thumbs-up/down, with the prompt context -----

def record_feedback(txn_id, summary, score, prompt=None, run_id=None) -> None:
    """Persist analyst thumbs-up(1)/down(0) locally (+ LangSmith when enabled)."""
    FEEDBACK_LOG.parent.mkdir(parents=True, exist_ok=True)
    record = {"txn_id": txn_id, "summary": summary, "score": int(score),
              "prompt": prompt or {}, "run_id": run_id,
              "ts": datetime.now(timezone.utc).isoformat()}
    with FEEDBACK_LOG.open("a", encoding="utf-8") as fh:
        fh.write(json.dumps(record) + "\n")          # append-only JSONL

    if settings.langsmith_enabled and run_id:        # <-- run_id never passed: §7 #8
        from langsmith import Client
        Client().create_feedback(run_id, key="analyst_thumb", score=float(score))

# --- 2. CURATE: export thumbs-up summaries as a chat dataset -----

def build_dataset() -> Path:
    """Export thumbs-up, grounded summaries to a chat-format JSONL file."""
    records = [r for r in load_feedback()
                if int(r.get("score", 0)) == 1 and r.get("summary")]
    out = Path(settings.finetune_dataset_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as fh:
        for r in records:
            example = {"messages": [
                {"role": "system", "content": SYSTEM_RULES},          # same rules as serving
                {"role": "user", "content": _user_prompt(r.get("prompt") or {})},
                {"role": "assistant", "content": r["summary"]},        # the approved answer
            ]}
            fh.write(json.dumps(example) + "\n")
    return out

# --- 3a. TRAIN (hosted): OpenAI fine-tuning API -----

def finetune_openai(dataset_path: Path) -> dict[str, Any]:
    from openai import OpenAI
    client = OpenAI(api_key=settings.openai_api_key)
    up = client.files.create(file=dataset_path.open("rb"), purpose="fine-tune")
    job = client.fine_tuning.jobs.create(training_file=up.id,
                                         model="gpt-4o-mini-2024-07-18")
    while True:
        # poll to a terminal state
        job = client.fine_tuning.jobs.retrieve(job.id)
        if job.status in {"succeeded", "failed", "cancelled"}:
            break
        time.sleep(30)
    return {"status": job.status, "model_id": job.fine_tuned_model}

# --- 3b. TRAIN (local): LoRA / PEFT adapter -----

def finetune_local(dataset_path: Path) -> dict[str, Any]:
    from peft import LoraConfig, get_peft_model
    from transformers import AutoModelForCausalLM, AutoTokenizer,
```

```

DataCollatorForLanguageModeling, Trainer, TrainingArguments)

base = settings.finetune_base_model # distilgpt2
tokenizer = AutoTokenizer.from_pretrained(base)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token # GPT-2 has no pad token
model = AutoModelForCausalLM.from_pretrained(base)

lora = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.05, task_type="CAUSAL_LM")
model = get_peft_model(model, lora)
model.print_trainable_parameters() # ~0.1-1% of params

rows = []
with dataset_path.open(encoding="utf-8") as fh:
    for line in fh:
        msgs = json.loads(line)["messages"]
        rows.append({"text": "".join(f"<|{m['role']}|>\n{m['content']}\n" for m in msgs)})
ds = Dataset.from_list(rows).map(
    lambda b: tokenizer(b["text"], truncation=True, max_length=512,
        padding="max_length"),
    batched=True, remove_columns=["text"])

trainer = Trainer(
    model=model,
    args=TrainingArguments(output_dir=..., per_device_train_batch_size=2,
        num_train_epochs=3, learning_rate=2e-4,
        logging_steps=5, save_strategy="no", report_to=[]),
    train_dataset=ds,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False), # causal LM
)
trainer.train()
model.save_pretrained(str(out_dir)) # adapter only: megabytes, not gigabytes
return {"status": "succeeded", "model_id": str(out_dir)}

# --- 4. ORCHESTRATE: gate on dataset size, pick a backend -----

def run(mode: str | None = None) -> dict[str, Any]:
    dataset = build_dataset()
    n = sum(1 for _ in dataset.open(encoding="utf-8"))
    if n < settings.finetune_min_examples: # default 10
        log.warning("insufficient_examples", have=n, need=settings.finetune_min_examples)
        return {"status": "insufficient_data", "examples": n}
    use_openai = (mode == "openai") or (mode == "auto" and bool(settings.openai_api_key))
    return finetune_openai(dataset) if use_openai else finetune_local(dataset)

```

What to say:

- "This is the loop that makes the project a *platform* rather than a pipeline: **a human judgement becomes training data becomes a served model**, with no manual data wrangling in between."
- "The dataset uses the **same** SYSTEM_RULES **constant** the chains serve with, so the model is fine-tuned under exactly the system prompt it will see in production. Prompt train/serve consistency is easy to get wrong and expensive when you do."
- "**LoRA in one sentence**: freeze the base weights and inject trainable low-rank matrices — $W + BA$ where B is $d \times r$, A is $r \times d$ and $r \ll d$. I use $r=8$, $\alpha=16$ (the scaling is α/r), dropout 0.05. You train well under 1% of the parameters, the artefact is an adapter of megabytes rather than a whole model, and you can host many task adapters over one shared base. Full fine-tuning needs several times the model size in memory for optimiser state."
- "Small details that show it actually ran: GPT-2 has **no pad token**, so you alias it to EOS or the collator crashes. `mlm=False` on the collator because this is causal LM, not masked. And `save_pretrained` on a PEFT model saves **only the adapter**."
- "**Volunteer both flaws**: the *result* is a toy — `distilgpt2` is 82M parameters and the gate is 10 examples, so three epochs of LoRA produces nothing useful. The *pipeline* is real, and that's the transferable part; a real run needs a 7B+ base via QLoRA and low thousands of examples. Second: I'm doing **SFT on thumbs-up only**, which throws away half of a paired preference signal. **DPO** is the technically correct method for thumbs-up/thumbs-down data because it uses the negatives, and it needs no separate reward model."

Exhibit 19 — Tests: pure functions and an injected fake LLM

tests/test_features.py, tests/test_chains.py, tests/test_finetune.py

```
# --- Pure function tests: no Kafka, no Mongo, no mocks -----

def test_velocity_counts_within_window():
    state = empty_state("user_1")
    base = datetime(2026, 1, 1, tzinfo=timezone.utc)
    for i in range(4):
        feats, state = compute_features(_txn(ts=base + timedelta(seconds=i * 10)), state)
    # 4th txn: three prior within 60s + itself
    assert feats.txn_1m == 4

def test_amount_zscore_grows_with_outlier():
    state = empty_state("user_1")
    base = datetime(2026, 1, 1, tzinfo=timezone.utc)
    for i in range(10):
        _, state = compute_features(_txn(amount=1000.0, ts=base + timedelta(minutes=i)), state)
    feats, _ = compute_features(_txn(amount=50000.0, ts=base + timedelta(minutes=11)), state)
    assert feats.amt_z > 3.0

def test_state_is_not_mutated():
    state = empty_state("user_1")
    _, new_state = compute_features(_txn(), state)
    assert state.count == 0      # the input is untouched...
    assert new_state.count == 1  # ...and the output moved forward

def test_feature_vector_matches_names():
    f = Features()
    assert len(f.to_vector()) == len(Features.feature_names())  # train/serve ordering guard

# --- LLM chain tests: deterministic, offline, structural assertions -----

@pytest.fixture
def fake_llm():
    from langchain_community.llms import FakeListLLM
    return FakeListLLM(
        responses=["Amount 8x baseline; new device; matches [CASE_017]. Recommend hold."]
    )

def test_rag_summary_uses_retrieved_context(monkeypatch, fake_llm):
    import fraud.assist.chains as chains
    monkeypatch.setattr(chains, "get_llm", lambda *a, **k: fake_llm)
    monkeypatch.setattr(chains, "similar_cases",
        lambda q, k=3: [{"_id": "CASE_017", "text": "card testing ring",
            "score": 0.9}])
    result = chains.rag_summary(TXN)
    assert result.grounded is True
    assert "CASE_017" in result.cited_case_ids  # assert structure, not prose

# --- Data-pipeline test: the thumbs-up filter -----

def test_build_dataset_only_uses_thumbs_up(monkeypatch, tmp_path):
    feedback = [
        {"txn_id": "t1", "summary": "good grounded summary [CASE_017]", "score": 1,
         "prompt": {"txn": "Steam 12000", "shap": "amt_z=+0.4", "history": "none"}},
        {"txn_id": "t2", "summary": "bad summary", "score": 0, "prompt": {}},
        {"txn_id": "t3", "summary": "another good one", "score": 1, "prompt": {...}},
    ]
    monkeypatch.setattr(ft, "load_feedback", lambda: feedback)
    monkeypatch.setattr(ft.settings, "finetune_dataset_path", tmp_path / "ds.jsonl")

    lines = [json.loads(x) for x in ft.build_dataset().read_text().splitlines()]
    assert len(lines) == 2  # only thumbs-up
    assert lines[0]["messages"][0]["role"] == "system"
    assert lines[0]["messages"][-1]["role"] == "assistant"
```

What to say:

- "Test the logic, not the infrastructure. Everything worth asserting lives in pure functions, so these tests need no broker, no database and — for the feature tests — no mocks at all. They run in milliseconds and they'd catch a real regression."
- "test_state_is_not_mutated is testing a *property*, not a behaviour: the input state must be untouched. That's the invariant that makes the function safe to call from a retry path."
- "test_feature_vector_matches_names is cheap insurance against **feature-order drift between training and serving** — a classic silent ML bug that produces a model that is confidently wrong rather than obviously broken."
- "For the LLM, the pattern is **dependency injection at the provider seam**: monkeypatch get_llm to a FakeListLLM with a fixed response, and similar_cases to a known document. Then assert on *structure* — is it a CaseSummary, was CASE_017 extracted, is grounded True — never on prose. **You cannot unit-test generated text; you can absolutely unit-test the contract around it.**"
- "What's missing, and I'd say it: no integration tests. Testcontainers for Kafka and Mongo would let me test the actual delivery semantics — including the rebalance case — and Hypothesis property-based tests on the feature functions are where I'd expect to find real bugs. Also, the quality gates exist (pytest, ruff, black, mypy) but aren't enforced by CI, so they only run when someone remembers."

Exhibit 20 — Infrastructure, task runner and the chaos drill

docker-compose.yml, Makefile, src/fraud/stream/chaos.py

```
# docker-compose.yml - Redpanda + Mongo + MLflow. Neo4j runs on Aura (cloud).
services:
  redpanda:
    image: redpandadata/redpanda:v24.1.2
    command:
      - redpanda
      - start
      - --smp=1
      - --overprovisioned                # laptop-friendly scheduling
      - --kafka-addr=PLAINTEXT://0.0.0.0:29092,OUTSIDE://0.0.0.0:19092
      - --advertise-kafka-addr=PLAINTEXT://redpanda:29092,OUTSIDE://localhost:19092
    healthcheck:
      test: ["CMD-SHELL", "rpk cluster health | grep -q 'Healthy:.*true'"]
      interval: 10s
      retries: 10

  mongo:
    image: mongo:7
    volumes: [mongo_data:/data/db]      # named volume: survives restarts
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]

  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.11.3
    command: >
      mlflow server
      --backend-store-uri sqlite:///mlflow/mlflow.db
      --default-artifact-root /mlflow/artifacts
      --host 0.0.0.0 --port 5000
```

```
# Makefile - the operational surface of the whole platform
seed: create-topics                # every seed step is idempotent
    $(PY) -m fraud.serving.mongo --init
    $(PY) -m fraud.graph.loader --seed
    $(PY) -m fraud.assist.retriever --seed

stream:    ; $(PY) -m fraud.stream.job
train:     ; $(PY) -m fraud.ml.train      # trains, registers, promotes if it wins
drift:     ; $(PY) -m fraud.ml.drift      # PSI -> may trigger retrain
finetune:  ; $(PY) -m fraud.assist.finetune # feedback -> fine-tuned analyst LLM
eval-llm:  ; $(PY) -m fraud.assist.eval   # faithfulness / relevance / hallucination
chaos:     ; $(PY) -m fraud.stream.chaos  # kill + recover, prove no loss
test:      ; $(PY) -m pytest
```

```
lint:      ; $(PY) -m ruff check src config tests
typecheck: ; $(PY) -m mypy src config
```

```
# chaos.py – prove the recovery claim instead of asserting it
def run_chaos(n: int = 200) -> None:
    ids = _publish(n)                # publish N transactions with known ids

    proc = mp.Process(target=_run_consumer, daemon=True)
    proc.start()
    time.sleep(6)                   # let it consume some
    log.warning("killing_consumer_mid_stream")
    proc.terminate()                # SIGTERM mid-stream
    proc.join()

    proc2 = mp.Process(target=_run_consumer, daemon=True)
    proc2.start()                   # restart: resumes from committed offset
    time.sleep(8)
    proc2.terminate(); proc2.join()

    found = mongo.flagged().count_documents({"_id": {"$in": ids}})
    print(f"CHAOS RESULT: published={len(ids)} persisted={found} (no loss if equal)")
```

What to say:

- "Three services, each with a **healthcheck** so `make up` followed by `make seed` doesn't race a broker that isn't ready yet — the most common cause of a flaky first-run experience. Redpanda's dual listener config is the classic Kafka gotcha: containers reach the broker at `redpanda:29092`, the host reaches it at `localhost:19092`, and `advertise-kafka-addr` is what makes both work."
- "Neo4j is deliberately **not** in Compose — it's a managed Aura cluster. So the project has one genuinely cloud-hosted stateful dependency, which is what forced the `neo4j_configured/degrade-to-zeros` path to exist at all."
- "The Makefile is the **operational surface**: one verb per capability, so a reviewer can go from clone to a running demo without reading source. `make seed` is idempotent end to end."
- "`make chaos` **is the part I'd point at**. I claimed replay safety, and a claim you don't test is a guess. It publishes N known ids, starts the consumer, kills it mid-stream, restarts it, and asserts every id persisted. **And I'd be precise about what it proves: no loss and idempotency — not exactly-once**, because the upsert makes duplicate processing invisible rather than absent. Being exact about what a test demonstrates matters as much as having one."

The four exhibits to lead with

If you only get to show a little code, show these — in this order.

Show	File	The one sentence
Exhibit 5	<code>stream/job.py</code>	"Commit the offset only after an idempotent write — that's the entire correctness story, and here's the DLQ path that keeps one bad message from taking down the consumer."
Exhibit 7	<code>stream/feature.py</code>	"(event, state) -> (features, new_state), pure and non-mutating — which is why it's unit-testable with plain dicts, and here's Welford plus the zero-variance branch."
Exhibit 8	<code>stream/score.py</code>	"The stream resolves an <i>alias</i> , never a version, so promoting a model is a registry operation with zero downtime."
Exhibit 15	<code>assist/chains.py</code>	"LCEL composition, and citations extracted in code so grounding is a measurable field rather than a hope."

4. Design decisions and why

Each row: the decision, the one-line justification, **and the cost you accepted**. Always state the cost — a decision presented without a tradeoff reads as an accident.

#	Decision	Why	Cost accepted
1	Three planes, three SLAs (hot / assist / cold)	A 300 ms scoring decision and a 4-second LLM narration have irreconcilable latency budgets. Coupling them means the slowest one sets the SLA.	Two stores to keep coherent (Mongo serving vs lake), more moving parts.
2	LLM never on the hot path	Non-determinism, variable latency, per-call cost and vendor availability are unacceptable in an authorisation decision. Also: an LLM decision is hard to audit for a regulator.	The LLM can't contribute signal to the score. That's fine — it contributes <i>comprehension</i> , which is a human-throughput problem, not a detection problem.
3	Partition Kafka by user_id	Velocity features are per-user and order-sensitive. Same key → same partition → single-consumer ordering, no distributed locking.	Hot-key skew: one whale user can hot-spot a partition. Also caps parallelism at partition count (6).
4	At-least-once + idempotent upsert (not exactly-once transactions)	Kafka EOS requires a transactional sink; Mongo isn't in that transaction. Natural-key upserts give the same <i>observable</i> outcome far more cheaply.	Duplicate <i>processing</i> still happens (side effects like the bronze append are not idempotent — see §7).
5	Commit after write, never before	Committing first means a crash between commit and write silently drops a transaction — the worst failure mode in fraud.	Duplicate processing on crash; slower than batched commits.
6	DLQ instead of crash or skip	A single malformed message must not take down the consumer (availability) and must not vanish (auditability). Headers carry the error.	Someone must own DLQ triage; there's no automated replay tooling.
7	MLflow <i>alias</i>, not version pinning	Decouples model deployment from code deployment. Promotion is a registry operation; the stream picks it up on its next reload tick.	The stream can silently change behaviour without a code change — needs an audit trail.
8	Champion/challenger promotion gate	Prevents an accidental regression from reaching production just because it trained more recently.	The gate is only as good as the eval set (see §7, item 2).
9	Heuristic scorer fallback	The pipeline must be observable end-to-end on a clean checkout before any model exists — otherwise nobody can debug the plumbing.	Risk of shipping the heuristic to prod unnoticed; mitigated by <code>model_version</code> being surfaced in the dashboard.
10	Mongo as serving store	Document shape matches the case exactly (nested features, SHAP array, graph sub-doc). Upsert-on- <code>_id</code> is native. Low-latency point reads by natural key.	Weak analytical querying; no joins; not the right home for training data.
11	Lakehouse separate from serving	Serving wants tiny point reads; training wants full history scans. One store optimised for both is optimised for neither.	Two write paths, dual-write consistency risk.
12	Neo4j for ring detection	"Which other users share this device, and how large is that component?" is a traversal. In SQL it's a recursive CTE that degrades; in a graph DB it's the native operation.	A network hop per lookup on the hot path (see §7, item 4) and an extra system to operate.
13	GDS + networkx dual implementation	Aura Free doesn't include GDS. A fallback keeps the <i>feature</i> available even when the <i>platform tier</i> isn't.	Two code paths to keep semantically identical; networkx doesn't scale past a few hundred thousand edges.
14	IsolationForest → XGBoost auto-switch	Cold start has no labels; unsupervised is the only honest option. Once ≥50 labels exist, supervised is strictly better. Matches the real lifecycle of a fraud system.	Two model families to explain, calibrate and monitor.
15	PR-AUC as the headline metric	At 3% (really <1%) positives, ROC-AUC is flattered by the huge negative class. Precision-recall is the honest curve.	Less familiar to non-ML stakeholders; needs translating into "review capacity".

#	Decision	Why	Cost accepted
16	SHAP only for non-LOW bands	Explanation is only consumed for cases a human will see. Computing it for 97% of traffic is pure waste on the latency budget.	Can't retroactively explain a LOW case without recomputation.
17	Pure feature function	Testability, and it lets the same logic serve the stream, the tests and the synthetic generator's contract.	Requires passing state in/out explicitly; the state store round-trip becomes visible (and is the bottleneck — \$7 item 3).
18	Welford's online variance	Streaming z-score without retaining all history, and numerically stable vs the naive sum-of-squares formula.	Not windowed — it's a lifetime baseline, so it adapts slowly to genuine behaviour change.
19	Grounded prompts + citation extraction	An LLM that confidently invents a precedent case in a fraud file is worse than no LLM. Prompt forbids unsupported claims; code <i>extracts</i> [CASE_xxx] ids so grounding is measurable, not just requested.	Extraction is a proxy — a cited id doesn't prove the claim about it is true.
20	Expected loss = score × amount for triage	A 0.95 on ₹200 and a 0.72 on ₹80,000 are not equally urgent. Ranking by risk alone burns analyst hours on cheap fraud.	Ignores non-monetary harm (account takeover, reputational, regulatory).
21	Provider abstraction for the LLM	One seam (<code>assist/llm.py</code>) means fake/OpenAI/Anthropic/fine-tuned are a config change, and tests run offline with zero network. It's also how the fine-tuned model joins without touching a single chain.	An abstraction over four providers with genuinely different capabilities (tool-calling, chat vs completion) leaks — handled by <code>_supports_tool_calling()</code> .
22	Fine-tune from real human feedback, thumbs-up only	Analyst-approved summaries are exactly the target distribution: house style, right length, right hedging. Cheaper and more controllable than prompt-engineering house style forever.	Only ~10 examples minimum gate; thumbs-up-only discards the (arguably richer) negative signal.
23	Streamlit, not React + FastAPI	The UI's job is to prove the loop closes. Streamlit gets a reviewer workflow in ~200 lines. Time went into the pipeline, which is what the project is about.	Not production-grade: no auth, full re-render on refresh, single-user assumptions.
24	Redpanda, not Kafka + ZooKeeper	Kafka-API compatible, single binary, no JVM/ZooKeeper, starts in seconds on a laptop. Client code is unchanged against real Kafka.	Not the exact broker most enterprises run; some Kafka-specific admin tooling differs.
25	Everything degrades gracefully	A demo that fails on a missing key never gets demoed. More importantly: it forces you to name every external dependency and decide its failure mode.	Failures become <i>silent</i> . The mitigation — surfacing <code>model_version</code> , <code>describe_provider()</code> and the Neo4j status in the UI — is partial.
26	Pydantic v2 contracts everywhere	One schema module is the wire format, the DB document and the API type. Validation at the boundary is what makes the DLQ path meaningful.	Validation cost per event; Pydantic v2 is fast but not free.

5. Alternatives you did NOT use, and why

This is the section that separates "I followed a tutorial" from "I made choices." For each, know: **what it is** → **why it's the popular pick** → **why not here** → **when you'd switch**.

5.1 Stream processing engine

Alternative	Why people pick it	Why not here	When I'd switch
Apache Flink	The correct answer for stateful streaming: true event-time semantics, watermarks, RocksDB keyed state, exactly-once via checkpointing + two-phase-commit sinks.	Enormous operational surface (JobManager/TaskManagers, checkpoint storage) for a laptop-runnable project, and the JVM/PyFlink boundary would have obscured the feature logic I wanted to make legible and testable.	The moment state exceeds what a remote store can serve per-event, or the business needs real watermarked windows with late-data handling. Flink is the <i>destination</i> architecture for this system.
Spark Structured Streaming	Same code for batch and stream; huge ecosystem; Delta Lake integration.	Micro-batch means seconds of inherent latency; a fraud authorisation decision wants sub-second. Also heavy for this footprint.	If the business accepts 5–30s latency and I want unified batch/stream feature code.
Kafka Streams / ksqlDB	Library not cluster; RocksDB local state stores; exactly-once; changelog-backed fault tolerance.	JVM-only (Kafka Streams). It's the <i>right</i> answer for "stateful Kafka processing", and I'd name it as such — but it would have meant writing the platform in Java/Scala.	If the team is a JVM shop.
Faust / Bytewax / Quix	Python-native stateful streaming, closest thing to Kafka Streams for Python.	Faust is effectively unmaintained (faust-streaming is a community fork); Bytewax was the real contender. I chose a raw consumer because it makes the delivery-semantics reasoning explicit and visible — offset commits, DLQ, idempotency are in my code, not hidden in a framework. That's the thing I wanted to demonstrate.	If I needed windowing/joins/rescaling for real, Bytewax over hand-rolling.
Managed: Kinesis Data Analytics / Confluent Flink / Databricks DLT	Zero ops.	Requires paid cloud; the project's constraint was zero paid dependencies.	Any real deployment with a cloud budget.

Say this: "Hand-rolling a consumer was a *pedagogical* choice — I wanted the delivery semantics to be code I wrote and can defend, not a framework default I'd inherited. For production I'd move the stateful part to Flink or Kafka Streams, and the reason is specifically **keyed local state**, not throughput."

5.2 Message bus

Alternative	Why not here	Notes
Apache Kafka (real)	Nothing wrong with it — Redpanda is Kafka-API compatible, so my client code is unchanged. Redpanda just avoids ZooKeeper/KRaft-era JVM setup on a laptop.	Emphasise: <i>the code is Kafka code</i> .
Apache Pulsar	Tiered storage and multi-tenancy are genuinely nice; smaller ecosystem, and the segment/broker split is more to operate.	
AWS Kinesis / GCP Pub/Sub	Vendor lock-in and a paid dependency; Kinesis's 5 reads/s/shard limits and 24h–7d retention are constraining for replay-heavy fraud work.	Kafka's long retention is what makes "replay the last 30 days through a new model" possible.
RabbitMQ	Wrong primitive. It's a broker with queues, not a replayable partitioned log. Fraud needs replay and ordered per-key streams.	Good crisp answer if asked "why not RabbitMQ?"

Alternative	Why not here	Notes
Redis Streams	Fine for lightweight fan-out; weaker durability/replay story and no consumer-group rebalancing at Kafka's maturity.	

5.3 Serialisation & schema

Alternative	Why it's popular	Why not here
Avro + Confluent Schema Registry	The industry-standard answer: compact binary, enforced schema evolution (backward/forward compatibility checks <i>at publish time</i>), no bad data on the topic.	I used JSON + Pydantic validation on the <i>consumer</i> . That's a real weakness: an incompatible producer change is only caught downstream, at DLQ time. Adding Schema Registry is the #1 thing I'd do to the ingest layer.
Protobuf	Same benefits, better cross-language codegen.	Same reason.
JSON + Pydantic (chosen)	Zero infra, human-readable topics, one schema module doubles as the DB contract. Right for a project where the schema is authored in one place.	Pay for it in schema-evolution safety.

5.4 Feature state store

Alternative	Why	Why not here
Redis	The obvious production answer for per-key hot state: sub-ms, atomic ops (HINCRBY, sorted sets for sliding windows), TTL for free window expiry.	Adding a fourth store for a demo; Mongo already had the durability. But this is the honest answer to "what would you change first?" — Redis for hot state, Mongo for the durable case record.
Flink/Kafka Streams local state (RocksDB)	Fastest possible: state is <i>co-located with the partition</i> , no network hop at all, changelog-backed for recovery.	Requires the framework (§5.1). This is the architecturally correct end state.
Feast / Tecton (feature store)	Solves <i>training-serving skew</i> by definition — one feature definition materialised to both an offline and an online store, with point-in-time-correct historical joins.	Not used, and the project has exactly the skew problem a feature store exists to prevent (§7, item 1). Naming Feast and explaining <i>why it would fix my specific bug</i> is a very strong answer.
Mongo (chosen)	One less system; durable; document shape fits.	Read-modify-write per event = 2 network round trips on the hot path, and a lost-update window during consumer rebalance.

5.5 Serving database

Alternative	Why not here
PostgreSQL	Would have worked, and pgvector would have collapsed my vector store into the same DB — genuinely tempting. Rejected because the case document is deeply nested and schema-fluid (features dict, SHAP array, graph sub-doc), and <code>upsert</code> on <code>natural</code> key is Mongo's native idiom.
Cassandra / DynamoDB	Excellent write throughput and partition-key semantics that mirror Kafka's. Overkill at this scale and awkward for the ad-hoc queries the review console needs.
Elasticsearch / OpenSearch	Great for analyst <i>search</i> over cases, and it has vector search now. I'd add it alongside Mongo for investigator search, not instead of it.
ClickHouse / DuckDB	Right for the analytical/cold side, wrong for point-read serving. DuckDB over the bronze layer would be a very cheap upgrade for the training path.

5.6 Graph

Alternative	Why not here
networkx only (no graph DB)	It's already the fallback. In-memory, single-process, no persistence, no concurrent queries — fine for offline community detection, not for a per-event lookup by many consumers.
SQL recursive CTE	Genuinely viable for shallow shared-attribute joins, and cheaper operationally. Degrades badly at depth ≥ 3 and makes multi-hop ring queries unreadable.
Amazon Neptune / TigerGraph / ArangoDB / Memgraph	Neptune = AWS lock-in; TigerGraph = licence cost; Memgraph is the interesting one (in-memory, Cypher-compatible, faster for streaming graph updates). Neo4j won on Cypher familiarity, GDS's algorithm library, and a genuinely usable free managed tier.
Graph Neural Networks (GraphSAGE / PyG / DGL)	The research-frontier answer for fraud ring detection, and it <i>would</i> likely beat hand-crafted features by learning ring topology end-to-end. Rejected for three reasons I'd state plainly: (a) it needs labelled graph data I don't have, (b) inference latency on a hot path with neighbour sampling is a real problem, (c) explainability to an analyst and a regulator collapses — and explainability was a hard requirement here. Correct framing: hand-crafted graph features are the interpretable baseline you must beat before a GNN is justified.

5.7 Model choice

Alternative	Why not here
LightGBM / CatBoost	Both likely as good or better — LightGBM is usually faster to train, CatBoost handles high-cardinality categoricals (merchant, device, country) natively, which is genuinely relevant here. XGBoost was chosen for ubiquity and <code>scale_pos_weight</code> + <code>aucpr</code> being first-class. Honest answer: "a bake-off between XGB/LGBM/CatBoost is table stakes and I didn't run one."
Logistic regression	Should have been the baseline. Fully interpretable, well-calibrated probabilities out of the box, trivially fast, and regulator-friendly. Not having a linear baseline is a legitimate criticism.
Autoencoder / VAE for anomaly detection	Reconstruction error as an anomaly score is elegant and handles high-dimensional/latent structure. Needs more data, more tuning, and gives no per-feature attribution — which conflicts with the SHAP requirement.
LSTM / Transformer over transaction sequences	The genuinely powerful approach: model each user's transaction <i>sequence</i> , not a snapshot of aggregates. This is where the real headroom is. Rejected on latency, data volume, and explainability. Great answer to "how would you improve the model?"
Rules engine (Drools / decision tables) only	What most banks actually run, and what regulators like: deterministic, auditable, instantly editable by a fraud analyst. Weak against novel patterns and a maintenance swamp at scale. Best answer: production fraud systems are hybrid — rules for known-bad and hard policy, ML for novel/graded risk. My heuristic scorer is a rudimentary version of that; a real system keeps the rules layer permanently, not as a fallback.
Isolation Forest alone	It's the cold-start path. Unsupervised anomaly $\neq$ fraud — an unusual-but-legitimate transaction scores identically. Once labels exist, supervised wins decisively.
One-class SVM / LOF / Elliptic Envelope	Poorer scaling (One-class SVM is roughly quadratic), and LOF doesn't naturally support out-of-sample scoring — which a streaming system requires.

5.8 Explainability

Alternative	Why not here
LIME	Local surrogate models; slower per explanation and less stable across runs. SHAP's additive attributions have a theoretical guarantee (Shapley values) and TreeExplainer is exact and fast for tree ensembles.
Built-in feature importance (gain/split)	Global, not per-case. Useless for "why was <i>this</i> transaction flagged?" — which is the actual analyst question.
Counterfactual explanations (DiCE) / Anchors	Arguably <i>more</i> useful to an analyst — "this would not have flagged if amount were under X". Strong "what's next" answer.
Nothing (black box)	Non-starter. Adverse-action reasoning in financial services is a regulatory requirement, and an analyst can't triage without a reason.

5.9 MLOps / experiment tracking

Alternative	Why not here
Weights & Biases	Better UI and experiment comparison; SaaS with a paid tier, and its model registry is less central to its product than MLflow's. MLflow's registry-alias-driven deployment is the actual load-bearing feature I needed.
DVC	Data/model versioning in Git — complementary, not competing. Would pair well for versioning the bronze snapshots that train each model. Genuine gap: my models are versioned, my training data is not.
Kubeflow / SageMaker / Vertex AI / Azure ML	Cloud lock-in and cost; Kubeflow needs Kubernetes. Wrong weight class for a laptop-runnable project.
Metaflow	Nice ergonomics for DAG-shaped ML; less registry-centric.
Airflow / Dagster / Prefect	This is a real gap, not a rejection. My cold path is orchestrated by a Makefile a human runs. Retraining, drift checks and lake compaction should be scheduled, retried, alerted and backfillable. Dagster would be my pick — it's asset-oriented, which maps naturally onto "the gold feature table" and "the production model" as assets.

5.10 Drift detection

Alternative	Why not here
Evidently AI	Purpose-built: drift reports, dashboards, test suites, more statistical tests than PSI. I hand-rolled PSI (~10 lines) to keep the dependency count down and because I wanted to be able to <i>explain the maths</i> , not just call a library. Evidently is what I'd use in production.
KS test / Wasserstein / Jensen-Shannon / Chi-square	All defensible. PSI is the retail-credit/fraud industry convention with well-known thresholds (<0.1 stable, 0.1–0.25 moderate, >0.25 significant), which is exactly why I picked it — the threshold has an accepted meaning to a risk team.
Great Expectations / Soda	Data <i>quality</i> validation (nulls, ranges, uniqueness) — orthogonal to distribution drift, and also missing. Both belong in the cold path.
Concept-drift monitoring (label-based)	The one that actually matters: PSI catches <i>input</i> drift, but performance decay from changing fraud behaviour needs labels. My labels arrive from analyst review with a delay, so this needs a delayed-feedback evaluation harness. Strong "what's missing" answer.

5.11 LLM orchestration

Alternative	Why not here
Raw OpenAI/Anthropic SDK	The strongest counter-argument, honestly. My chains are shallow (prompt llm parse) — the SDK plus 30 lines would do it with fewer dependencies and less abstraction churn. What LangChain actually bought me: the provider abstraction (fake/OpenAI/Anthropic/fine-tuned behind one interface, which is what makes offline testing and the fine-tune swap free), the tool-calling agent scaffolding, and native LangSmith tracing. Say exactly that — "I know where the abstraction earns its keep and where it doesn't."
LlamaIndex	Better at the retrieval/indexing half (advanced retrievers, rerankers, structured indices). My retrieval is 5 documents and brute-force cosine — LlamaIndex would be over-tooled. I'd reach for it if the case corpus grew to real size and needed hybrid search + reranking.
DSPy	Very compelling for the <i>right</i> reason: it optimises prompts against a metric instead of hand-tuning them, which fits my faithfulness/relevance eval harness perfectly. The honest reason I didn't: it's a different mental model and I'd have had to give up the LangSmith/LangChain ecosystem integration. Naming DSPy as the upgrade path for prompt optimisation is a strong signal.
Semantic Kernel / Haystack	SK is .NET-first; Haystack is solid but a smaller ecosystem and I'd already committed to LangSmith.
LangGraph	The right answer for the <i>agent</i> . My investigation agent is a linear AgentExecutor loop with a hand-written deterministic fallback. LangGraph gives explicit state-machine control, cycles, checkpointing and human-in-the-loop interrupts — all of which a real multi-step investigation wants. Say: "the deterministic fallback I wrote is a poor man's state graph; LangGraph is the principled version."

5.12 Vector store & embeddings

Alternative	Why not here
FAISS	It's in <code>pyproject.toml</code> and I never wired it up (be upfront — see §7). Right choice for in-process ANN at moderate scale; needs its own persistence/rebuild story.
pgvector	If I'd used Postgres for serving, this collapses two systems into one, with transactional consistency between the case record and its embedding. Cleanest architecture of all the options.
Pinecone / Weaviate / Qdrant / Milvus	Managed/dedicated ANN with filtering and hybrid search. Overkill for 5 seed documents and a paid dependency. Qdrant would be my self-hosted pick.
Mongo Atlas \$vectorSearch (implemented, off by default)	Zero new systems, HNSW-backed, filterable, transactionally adjacent to the case documents. Requires Atlas. The code path exists behind <code>MONGO_USE_ATLAS_VECTOR</code> .
Brute-force cosine (default)	$O(n)$ per query and it re-reads every vector from Mongo each time. Completely fine at $n=5$, indefensible at $n=100k$. I know the exact crossover reason: no index, no ANN, full scan.
BM25 / hybrid search	For case retrieval, keyword matching on merchant names and card BINs would likely beat dense embeddings. Hybrid (BM25 + dense, reciprocal-rank fusion) is the known-best default and I don't do it.
OpenAI embeddings vs all-MiniLM-L6-v2	MiniLM is 384-dim, runs locally, free, ~80MB, and good enough for short case text. OpenAI's are better but add a per-query cost, latency and a hard network dependency to a path I wanted to work offline.

5.13 LLM observability

Alternative	Why not here
Langfuse	Open-source and self-hostable — genuinely better for a fraud domain where sending prompt contents (which contain transaction data) to a third-party SaaS is a compliance problem. Strong answer: "for a real bank I'd use Langfuse self-hosted, precisely because LangSmith is SaaS."
Arize Phoenix	Open-source, strong on embedding drift and eval; local-first.
W&B Weave / Braintrust / Helicone	Weave overlaps W&B; Braintrust is eval-first; Helicone is a proxy (cheapest possible integration — one base-URL change).
OpenTelemetry + OpenLLMetry	Vendor-neutral, and it puts LLM traces in the <i>same</i> system as the rest of your service traces. Architecturally the most correct answer for a company that already has observability.

5.14 Adapting the LLM

Alternative	Why not here
RAG only, no fine-tuning	The correct default, and I do it. Rule of thumb: RAG for knowledge, fine-tuning for form . I fine-tune for <i>form</i> — house style, length, hedging, citation habit — which is exactly what analyst thumbs-up data encodes. Say that; it shows you know what fine-tuning is and isn't for.
Few-shot prompting	Cheaper and instantly updatable — should be the first attempt, and it burns context on every call. Fine-tuning amortises the examples into weights and shortens the prompt.
DPO / RLHF / ORPO	The <i>technically correct</i> use of thumbs-up/thumbs-down data, because it's preference data — it uses the thumbs-down signal, which my SFT approach throws away. Honest gap and a great "what's next": "I have paired preference data and I'm only using half of it."
Full fine-tuning vs LoRA	LoRA: ~0.1–1% of parameters trainable, one small adapter per task, trains on modest hardware, base model stays shared. Full FT needs many multiples of model memory for optimiser state and gives you a whole new model to store per task. No contest at this scale.
QLoRA	4-bit quantised base + LoRA — how you'd fine-tune a 7B+ model on one consumer GPU. My base is <code>distilgpt2</code> , so quantisation buys nothing. Know the term.
Prompt/prefix tuning	Even fewer parameters than LoRA but generally weaker and fiddlier; LoRA has won in practice.

5.15 UI, packaging and infra

Alternative	Why not here
React + FastAPI	The production answer: real auth, RBAC, WebSocket push instead of 3-second polling, component tests, multi-analyst concurrency. Multiple days of work that would have bought zero pipeline capability.
Grafana / Metabase / Superset	Great for the <i>ops metrics</i> panel; can't express a review workflow with buttons that write labels and capture LLM feedback.
Dash / Panel / Gradio / Retool	Dash = more control, more code. Gradio is ML-demo-shaped, not workflow-shaped. Retool is paid/low-code.
Kubernetes + Helm	Correct for production, wrong for "clone and run in five minutes". Also: I have no Dockerfile for the app itself, only for infra — a real gap for deployability.
Poetry / uv / Pipenv	<code>pyproject.toml</code> + pip + extras is the lowest-friction, most portable option and needs no extra tool installed. uv would be a straight speed win today.
Taskfile / just / nox / tox	Makefile is universally present and the targets are one-liners. make on Windows is the friction point (the Makefile notes PowerShell compatibility).
Hydra / dynaconf	Hydra's composition and sweeps are excellent for research; overkill for one flat, typed settings object. pydantic-settings gives type validation and IDE completion, which mattered more.
loguru / stdlib logging	structlog's key-value events (<code>log.info("flagged", txn=..., latency_ms=...)</code>) are machine-parseable, which is what you want for a pipeline you'll aggregate. loguru is nicer to read, worse to query.
Prometheus + Grafana for metrics	Missing and it shouldn't be. I collect latencies in a list (<code>_latencies</code> in <code>stream/job.py</code>) and never expose them. A Prometheus counter/histogram (<code>events_processed</code> , <code>scoring_latency_seconds</code> , <code>dlq_total</code> , <code>flag_rate</code>) is a genuinely small change with a large payoff.

6. Difficulties

Interviewers ask "what was hard?" Weak candidates say "setting up the environment." Strong candidates name a **design tension** and how they resolved it. Here are yours, in three tiers.

6.1 Difficulties visibly solved in the code — claim these confidently

- 1. "Where does the LLM belong?"** The tempting design is to let the LLM contribute to the risk score. That fails on four axes at once: latency (seconds vs milliseconds), determinism (the same transaction must score the same twice), cost (per-call, on 100% of traffic), and auditability (you cannot explain a declined authorisation to a regulator with "the language model felt uneasy"). **Resolution:** the LLM's job is *human throughput*, not detection. It compresses a case file into a paragraph an analyst can act on. That reframing is the single best answer in this project and I'd lead with it.
- 2. Exactly-once, without exactly-once.** Kafka EOS needs a transactional sink; MongoDB isn't in that transaction, so a true transactional boundary was off the table. **Resolution:** make every write idempotent on a natural key (`_id == txn_id` for cases, `_id == user_id` for state) and commit the offset only *after* the write. The composite is at-least-once delivery with idempotent effects — observationally effectively-once for the serving store, at a fraction of the complexity. Then prove it: `stream/chaos.py` publishes known ids, kills the consumer mid-stream, restarts it, and asserts every id landed. *Be precise about what the drill proves: no loss and idempotency. It does not prove exactly-once — the upsert makes duplicate processing invisible rather than absent.*
- 3. Poison messages vs availability.** One malformed payload must not crash-loop a consumer (availability) and must not be silently dropped (auditability). **Resolution:** validate at the boundary with Pydantic, route failures to a DLQ with the error in a header, commit the offset so the consumer advances. The raw bytes survive for replay.
- 4. Deploying a model without deploying code.** Restarting the stream to pick up a new model means fraud goes unscored during the restart, and it couples ML cadence to release cadence. **Resolution:** the stream resolves `models:/fraud-scorer@Production` — an alias, not a version — and re-checks on a timer (`MODEL_RELOAD_SECONDS`). It compares the resolved version to the loaded one and only reloads on change. Promotion becomes a registry operation with zero downtime.
- 5. A z-score with no variance.** The naive $(x - \text{mean}) / \text{std}$ divides by zero for a user whose transactions have all been identical — so their first ₹50,000 charge would score as *not anomalous*, which is exactly backwards. **Resolution:** fall back to a relative deviation with a floor: `denom = max(abs(mean) * 0.25, 1.0)` (`stream/features.py:50`). A small fix that shows you thought about the degenerate case — mention it; it reads as real engineering rather than tutorial-following.
- 6. Streaming variance without unbounded memory.** Recomputing standard deviation needs the history you can't keep per user at scale. **Resolution:** Welford's online algorithm — carry count, mean, M2 and update in $O(1)$, numerically stable (unlike naive sum-of-squares, which cancels catastrophically for large means and small variances). Know the recurrence; it's a plausible whiteboard follow-up.
- 7. Making the LLM's grounding measurable.** "Please don't hallucinate" is not an engineering control. **Resolution:** three layers — (a) the prompt forbids unsupported claims and mandates `[CASE_ID]` citations, (b) code *extracts* the cited ids with a regex and stores them on the `CaseSummary`, (c) the eval harness scores relevance and faithfulness. Grounding becomes an observable field, not a hope.
- 8. A system that must run with zero credentials.** Every external dependency needed a named failure mode: MLflow → heuristic scorer; Neo4j → zero-valued graph features; OpenAI → `FakeListLLM`; sentence-transformers → deterministic hashing embedder; Atlas → brute-force cosine; GDS → `networkx WCC`. **The forcing function was useful in itself:** you cannot write a graceful fallback without first deciding what your system *means* when a dependency is absent.
- 9. Testing a streaming system.** You can't unit-test a Kafka consumer meaningfully. **Resolution:** push all the logic worth testing into a pure function (`compute_features`) that takes state and an event and returns features and new state, mutating nothing. Then the tests are plain dicts — velocity windows, z-score growth, non-mutation of input state, haversine distance — with no broker, no DB, no mocks. The LLM chains are tested the same way, with `FakeListLLM` injected via `monkeypatch`.

6.2 Difficulties latent in the code — be ready, don't volunteer unprompted

10. The state round-trip is the throughput ceiling. Every event does `find_one` on `user_rolling_state`, then two or three upserts. That's 3–4 network round trips per transaction, serially. Sub-millisecond features, then ~1–5 ms of network per hop. **Know the fix and why it's the fix:** keyed local state (Flink/Kafka Streams RocksDB) eliminates the hop entirely; Redis shrinks it. This is your answer to "where's the bottleneck?" and it is a *much* better answer than guessing at CPU.

11. Rebalance can lose an update. Read-modify-write is not atomic. During a consumer-group rebalance, two consumers can briefly own the same partition; two concurrent read-modify-writes on one `user_id` and one update is lost. Partitioning by `user_id` makes this rare, not impossible. **Fixes:** atomic Mongo update operators (`$inc/$push`) instead of whole-document `$set`, optimistic concurrency via a version field, or — properly — keyed local state where the partition assignment is the lock.

12. SHAP on the hot path, with a fresh explainer per call. `ml/explain.py` constructs the explainer on every invocation. For a tree model that's cheap-ish but strictly wasteful; for the model-agnostic fallback it's expensive. It only runs for non-LOW bands, which bounds the damage to ~3% of traffic. **Fix:** build the explainer once when the model loads and cache it alongside; or move explanation off the hot path entirely and compute it lazily when an analyst opens the case — arguably the right design, since nobody reads 97% of explanations.

13. A synchronous cloud round trip per event. `enrich_with_graph` → `query_graph` opens a Neo4j Aura session and runs Cypher **per transaction**, against a *cloud* cluster. That is the largest single latency contributor on the hot path and the least defensible. **Fixes, in order:** cache ring lookups per user with a TTL (rings change on the order of hours, not milliseconds); or precompute ring membership into Mongo/Redis in the cold path and read it locally; or maintain it as a Kafka-Streams-style enrichment join. Have this answer ready — it's the first thing a streaming specialist will find.

14. Bronze writes are not idempotent. `_write_bronze` appends a line to a JSONL file. Replay Kafka and you get duplicate rows in the lake — even though Mongo stays correct. So the "replay is safe" claim is true of the serving store and false of the lake. **Fix:** dedupe on `txn_id` at read/compaction time, or write to a real table format (Delta/Iceberg) with a merge-on-key, which is what the docs already imply. It also opens and closes a file handle per event and isn't safe for concurrent writers.

15. In-sample evaluation gates the promotion. `train_and_register` calls `evaluate(scorer, x, y)` on the *same* arrays it trained on (`ml/train.py:98`). No train/test split, no cross-validation. So the reported PR-AUC is optimistic, and the promotion gate `challenger_pr >= incumbent_pr` compares two in-sample numbers — with `>=`, ties promote. **Say it before they ask, and say the fix:** a stratified holdout (or better, a *temporal* split, because fraud has time structure and random splits leak future information into the past).

16. Latency is measured and then thrown away. `stream/job.py` appends to a `_latencies` list and trims it. Nothing computes a percentile, nothing exports it. **This is a 5-line fix worth doing before your interview** — see §10.

6.3 Difficulties you'd face in production — use these to show range

- **Label delay.** Chargebacks arrive 30–90 days later. Your training labels are censored: a transaction not yet disputed isn't confirmed-legitimate, it's *unknown*. This breaks naive supervised training and any "recent performance" metric. Handling it: delayed-label evaluation windows, and treating unlabelled data as unlabelled rather than negative.
- **Feedback loops / selection bias.** You only observe outcomes for transactions you *allowed*. Blocked transactions have no ground truth, so the model progressively stops learning about the region it already blocks. Mitigations: a small random hold-out that's allowed through despite a high score, propensity weighting, or explicit exploration.
- **Adversarial adaptation.** Fraudsters probe and adapt within days. Static thresholds decay. This is *why* drift detection and fast retraining exist here, and why my drift simulation models an adversarial shift (heavier fraud, higher amounts) rather than random noise.
- **The false-positive cost asymmetry.** Declining a legitimate ₹80,000 purchase can cost more in churn than absorbing a ₹2,000 fraud. The 0.80 threshold should be derived from a cost matrix and review capacity, not chosen by hand. Right framing: pick the threshold that maximises expected value, or the top-K that saturates your analysts' daily capacity.
- **PII and the third-party LLM.** Sending transaction context to OpenAI/Anthropic — and prompt contents to LangSmith — moves regulated data across a boundary. Real answer: redact/tokenise before the prompt, or self-host both the model and the tracing (Langfuse), and gate it on a DPA. **Volunteering this unprompted in a fintech interview is a**

big credibility win.

- **Fairness and adverse-action.** Proxy features can encode protected attributes (geography correlating with ethnicity). You need disparate-impact testing per segment and a reason-code path for declines. SHAP is the raw material for the latter; the audit is missing.
- **Hot-key skew.** Partitioning by `user_id` breaks down for a merchant aggregator or a corporate card with 10,000× normal volume. Mitigations: composite keys with salting, or a separate high-volume lane.
- **Multi-region / clock skew.** Event-time features across regions need watermarks and out-of-order tolerance. Currently there's no lateness handling at all — a late event just produces slightly wrong velocity counts.

7. Known gaps — pre-empt these

Strategy. Pick three of these and volunteer them, unprompted, near the end of your walkthrough: *"Three things I'd fix before calling this production-ready."* You convert every one of them from a discovered weakness into demonstrated judgement. If they find them first, you're defending; if you name them first, you're evaluating your own work like a senior engineer.

Recommended three: #1 (train/serve skew), #2 (in-sample eval), #4 (Neo4j on the hot path). They're the most technically substantive and they're the ones a strong interviewer will find.

#	Gap	The honest line to say
1	Train/serve skew — concrete and severe. Training features come from <code>ml/synthetic.py</code> , not from the lake. Three of eleven features are effectively constant in production but <i>informative</i> in training: <code>distinct_merchants_1h</code> is hardcoded to <code>len({event.merchant})</code> — always 1 — while training draws <code>Poisson(2)/Poisson(6)</code> ; <code>shared_device_flags</code> and <code>component_size</code> are 0 for every streamed user because the stream never writes edges to Neo4j (<code>upsert_transaction_edges</code> is only called by <code>seed_demo_graph</code>). <code>distinct_geos_1h</code> counts <i>all-time</i> distinct geos, not one hour, so its name lies.	"This is the textbook train/serve skew problem and my project has a live case of it. The model is learning to rely on three features that are constant at inference. Two fixes: train from the bronze lake so training and serving share one feature computation, and add graph-edge writes to the stream so the graph features are actually populated. A feature store like Feast exists precisely to make this class of bug structurally impossible."
2	In-sample evaluation. <code>evaluate()</code> runs on the training arrays; no holdout, no CV. The promotion gate therefore compares two optimistic numbers, and <code>>=</code> means ties promote. The docs claim "held-out PR-AUC" — the code doesn't do that.	"The gate is architecturally right and statistically wrong. It needs a temporal split, not just any holdout — fraud is time-structured, so a random split leaks future information."
3	Lakehouse is bronze-only. <code>silver_dir/gold_dir</code> are configured and never written. Bronze is append-only JSONL, not Parquet or Delta, despite the docs saying Delta/Parquet. Nothing reads the lake — training uses synthetic data.	"The medallion architecture is scaffolded, not implemented. Bronze is real JSONL; silver and gold are directories waiting for a compaction job. I'd write that as a scheduled Dagster asset."
4	Neo4j Aura called synchronously per event. A cloud network round trip inside the scoring loop.	"The most expensive thing on my hot path is a network call I don't need to make in real time. Ring membership changes hourly; I'm querying it per millisecond. Cache with a TTL, or precompute into the serving store."
5	DLQ on transient failure, with no retry. If Mongo blips, <code>process_event</code> raises, the message goes to the DLQ and the offset commits. A recoverable error is treated as a permanent one. <code>tenacity</code> is a declared dependency and never imported.	"I don't distinguish retryable from non-retryable failures. It needs bounded retry with exponential backoff — that's what tenacity is in the manifest for — and a circuit breaker, with the DLQ reserved for genuinely malformed data."
6	Offset commit per message, asynchronously. <code>consumer.commit(msg)</code> defaults to <code>asynchronous=True</code> in <code>confluent-kafka</code> , so commit failures are unobserved; committing per message is also a throughput tax.	"Batch the commits and check the result. Per-message async commit is the wrong default for both throughput and error visibility."
7	Declared but unused dependencies. <code>tenacity</code> , <code>faiss-cpu</code> , <code>plotly</code> , <code>pydeck</code> are in <code>pyproject.toml</code> and never imported. Conversely, <code>langchain-anthropic</code> is imported in <code>assist/llm.py</code> but is <i>not</i> in the dependency list — <code>LLM_PROVIDER=anthropic</code> fails with an <code>ImportError</code> on a clean install.	"That's a manifest hygiene bug and a genuine one — a missing runtime dependency for a documented code path. Pruning the unused four and adding <code>langchain-anthropic</code> is a five-minute fix I should have caught."
8	LangSmith feedback never fires. <code>record_feedback</code> only calls <code>Client().create_feedback</code> when <code>run_id</code> is not <code>None</code> , and the dashboard never passes one. Feedback lands in local JSONL only.	"The wiring is incomplete: I'd need to capture the LangSmith run id from the chain invocation — via a callback handler or <code>RunCollector</code> — and thread it through to the feedback call."

#	Gap	The honest line to say
9	Eval metrics are proxies, not judges. faithfulness = 1.0 if cited else 0.5; relevance is keyword overlap; hallucination is a constant.	"These are smoke tests, not evaluations. Real faithfulness needs claim-level entailment checking against the retrieved context — an LLM-as-judge with a rubric, ideally with the judge validated against human labels first. What I have proves the harness runs; it doesn't measure truth."
10	Vector search doesn't scale. _bruteforce_search reads every document from Mongo and computes cosine in Python, per query.	"Fine at n=5, absurd at n=100k. The Atlas \$vectorSearch path is already implemented behind a flag — that's the switch. Otherwise FAISS or pgvector."
11	top_alerts loads then sorts in Python. find() on band, list(cur), sort by score*amount, slice. Full result set into process memory every 3 seconds.	"Should be a Mongo aggregation with \$project/\$sort/\$limit, or better: store expected_loss as a field at write time and put an index on it. I compute it in the app when I could compute it once."
12	No CI, no app Dockerfile. No .github/workflows; Compose only covers infra. Quality gates exist as Makefile targets nobody enforces.	"The gates are all there — pytest, ruff, black, mypy — they're just not wired to a pipeline. A GitHub Actions workflow running those four on PR is an hour of work, and a multi-stage Dockerfile for the stream and dashboard is what makes it deployable."
13	No auth, no secret management, default creds. Streamlit is wide open; Mongo runs root/example; secrets live in a .env.	"Correct for a local demo, disqualifying for anything else. Real version: OIDC on the console, Mongo with least-privilege users and TLS, secrets in Vault or a cloud secret manager, and an audit log on every analyst action — which for fraud review is a compliance requirement, not a nice-to-have."
14	Single consumer instance, never scaled. Six partitions means it <i>could</i> run six ways parallel; that's untested.	"The design supports it — key-based partitioning, consumer groups, idempotent writes. But 'supports' and 'verified' aren't the same word, and I'd want a rebalance test before claiming it."
15	Fine-tuning on distilgpt2 with ≥10 examples. A 3-epoch LoRA on ten samples of an 82M-parameter model produces nothing useful.	"The <i>pipeline</i> is real and correct — feedback capture, dataset export, LoRA training, adapter serving through the same interface. The <i>result</i> is a toy, and I know why: distilgpt2 is far too small and ten examples is far too few. It's the plumbing for a real fine-tune, demonstrated end to end at laptop scale."
16	No windowing/watermarking; no late-data handling. Velocity windows are computed from a retained list, event-time, with no lateness tolerance. Producer timestamps are wall-clock at emit.	"There's no watermark concept, so an out-of-order event just yields a slightly wrong velocity count. Real event-time semantics is a big part of why Flink is the right destination."
17	get_scorer() is a module singleton with no lock. Streamlit and the stream both import it; concurrent reload isn't synchronised.	"Benign in practice — a torn reload would just re-load — but it's an unguarded shared mutable and I'd wrap the swap in a lock."
18	user_rolling_state grows unboundedly-ish. recent_ts/recent_amounts are capped at 200 within 1h, seen_geos/seen_devices at 100 — but the whole document is rewritten with \$set on every event.	"Bounded, but rewriting a document per event is write amplification. Atomic \$push with \$slice would update in place."

8. Interview questions by skill

Format: **Q** → the answer's *spine* (say this), then depth to hold in reserve.

8.1 Kafka & streaming fundamentals

Q: Why partition by `user_id`? Ordering and locality. Velocity features are per-user and order-dependent — "4 transactions in 60 seconds" is only computable if you see that user's events in sequence. Same key → same partition → one consumer → ordering for free, no distributed coordination. Cost: hot-key skew, and parallelism capped at partition count.

Q: Explain your delivery semantics precisely. At-least-once. `enable.auto.commit=false`; commit only after a successful write. `auto.offset.reset=earliest` so a new group replays from the start. All writes upsert on a natural key, so duplicate delivery is idempotent. **Net effect is effectively-once for the serving store — not exactly-once**, because duplicate *processing* still occurs and some side effects (the bronze append) aren't idempotent.

Q: How would you get true exactly-once? Kafka transactions with a transactional sink — read-process-write inside one transaction, so offsets and output commit atomically. That requires a participating sink (another Kafka topic, or a store with a two-phase-commit connector, which is how Flink does it). Mongo isn't in that transaction, so I chose idempotency instead. Idempotency is usually the cheaper and more robust answer in practice.

Q: What happens on consumer crash? / What if you committed before writing? Crash: uncommitted messages are redelivered on rebalance, reprocessed, and the upserts make it a no-op. Committing first would mean a crash between commit and write silently loses a transaction — unacceptable for fraud, where a miss is the expensive error.

Q: Poison message handling? Pydantic validates at the boundary. Failure → produce raw bytes to `raw-transactions-dlq` with the error in a header → commit the offset so the consumer advances. Never crash-loop, never silently drop.

Q: What's wrong with your DLQ policy? (*they will ask*) It doesn't distinguish transient from permanent failures. A Mongo timeout gets DLQ'd like malformed JSON. It needs retry-with-backoff and a circuit breaker first; the DLQ should be the terminal path for genuinely bad data only.

Q: `acks=all` and `enable.idempotence` — what do they do? `acks=all`: the leader waits for all in-sync replicas before acking, so an acked write survives leader failure. `enable.idempotence`: the producer attaches a producer id and sequence number so the broker dedupes retries — this is what makes retries safe rather than duplicate-generating. Together they're the durability floor for financial data.

Q: Consumer lag — how do you monitor it, what do you do when it grows? `kafka-consumer-groups --describe` or Redpanda console / a broker exporter. Lag growing means consumption is slower than production. Diagnose: is it one partition (skew) or all (capacity)? Then: add consumers up to the partition count, then increase partitions, then reduce per-event work — which for me means killing the per-event Neo4j call and the state round-trip first.

Q: Why 6 partitions? It sets max consumer parallelism, and partitions are cheap to have but painful to reduce. Six was a demo-scale choice with headroom. In production you'd size from target throughput divided by per-consumer throughput, with a multiple to allow rescaling.

Q: Redpanda vs Kafka? Kafka-API compatible; C++ single binary, no JVM, no ZooKeeper; thread-per-core with direct I/O. My client code is unchanged against real Kafka — that's the point. On a laptop it starts in seconds and uses a fraction of the memory.

8.2 Stream processing & state

Q: Where is your state, and why is that a problem? MongoDB, one document per user, read-modify-write per event. It's durable and simple, and it's my throughput ceiling — 3–4 network round trips per transaction. Correct answers are, in order: Redis (sub-ms, atomic ops, TTL-based window expiry), or keyed local state in Flink/Kafka Streams backed by RocksDB with a changelog topic, where state is co-located with the partition and there's *no network hop at all*.

Q: Is your read-modify-write safe? Not fully. It isn't atomic, so during a rebalance two consumers can briefly own a partition and one update can be lost. Partitioning by user makes it rare, not impossible. Fixes: atomic update operators (`$inc`, `$push` with `$slice`), optimistic concurrency with a version field, or keyed local state where partition ownership is the

mutual exclusion.

Q: Explain Welford's algorithm and why not just keep a running sum of squares. Welford carries `count`, `mean`, `M2`:
`delta = x - mean; mean += delta/count; M2 += delta * (x - new_mean);` variance is `M2/(count-1)`. $O(1)$ time and memory. The naive $E[x^2] - E[x]^2$ subtracts two large nearly-equal numbers and loses precision catastrophically when the mean is large and the variance small — exactly the transaction-amount regime.

Q: Your windows aren't real windows. What's missing? Correct — I filter a retained list by timestamp delta. There's no watermark, no lateness tolerance, no window state expiry policy, no allowed-lateness or side-output for late events. Event-time correctness with out-of-order data is precisely what Flink's watermarking exists for and is the main reason it's the destination architecture.

Q: How do you handle out-of-order events today? I don't, beyond using event time in the comparisons. A late event yields a slightly under-counted velocity feature. In a system where authorisation decisions are final in milliseconds that's an acceptable approximation, but I'd want to *measure* the lateness distribution before saying so with confidence.

8.3 MongoDB & serving

Q: Why Mongo, honestly? Document shape matches the case exactly — nested features dict, SHAP array, graph sub-document. Upsert-on-natural-key is native, which is the linchpin of my replay-safety story. Point reads by `_id` are fast. Weaknesses: poor analytics, no joins, and it is emphatically not the right home for training data — which is why the lakehouse exists as a separate plane.

Q: Walk me through your indexes and why each exists. `flagged_transactions`: `user_id` (history lookups for the LLM and the agent), `risk_band` (alert filtering), `created_at` descending (recency queries and the latest-model-version probe), `status` (review workflow). `alerts`: compound (`status`, `priority`) — compound ordering matters because the query filters on status and sorts on priority, so the prefix rule applies. **Missing**: an index on `expected_loss`, which is my hottest sort. I compute it in Python instead of storing it — that's the fix.

Q: `top_alerts` — critique it. It loads every high/medium document into process memory and sorts in Python, every 3 seconds. Should be an aggregation (`$match` → `$addFields/$project` → `$sort` → `$limit`) or, better, persist `expected_loss` at write time and index it. Currently $O(\text{matching documents})$ per refresh.

Q: How does your vector search work and when does it break? Two paths behind one flag. Atlas `$vectorSearch` — an aggregation stage with `numCandidates` and `limit`, HNSW-backed, returns `$meta: vectorSearchScore`. Or brute-force cosine in Python over every stored vector. The fallback is $O(n)$ with a full collection read per query: correct at $n=5$, indefensible past a few thousand.

Q: Why is `_id == txn_id` a good idea? What's the risk? It makes the natural key the primary key, so the upsert is inherently idempotent — no separate uniqueness constraint, no dedupe table. Risk: you've committed to that key forever. If `txn_id` ever turns out to be non-unique across sources you have a migration, not a bug fix.

8.4 Neo4j & graph

Q: Why a graph database at all? Because the question "which other users share this device, and how large is the connected component they sit in?" is a traversal, not a join. Fraud rings are *topology*: collusion shows up as shared devices, IPs and merchants. In SQL that's a recursive CTE that degrades with depth; in Cypher it's the native operation, and the algorithm library gives me community detection for free.

Q: Describe your graph model. Nodes: User, Device, Merchant, IP, each with a uniqueness constraint on `id`. Edges: `(:User)-[:USED]->(:Device)`, `-[:PAID]->(:Merchant)`, `-[:FROM_IP]->(:IP)`. MERGE everywhere so ingestion is idempotent. It's an entity-resolution graph — users are linked *through* shared attributes rather than to each other directly, which means a ring emerges as a connected component rather than needing to be asserted.

Q: WCC vs Louvain vs Label Propagation — why WCC? WCC finds *connected* components: any path at all puts you in the same group. For "does this user share infrastructure with known fraud accounts", reachability is exactly the right semantics and the result is trivially explainable to an analyst. Louvain optimises modularity and finds densely-connected *communities* inside a component — better when the graph is big and WCC gives you one giant blob, which is the realistic failure mode at scale. Label propagation is faster and less stable. **Say the failure mode**: at real volume WCC collapses into a single component (everyone shares *an* IP eventually), and that's when you move to Louvain or add edge weights and thresholds.

Q: GDS vs networkx — why both? GDS runs the algorithm in the database on a projected in-memory graph — fast, scales, and not available on Aura's free tier. The networkx path exports edges over the wire and computes components locally, then writes `ring_id/component_size` back with an UNWIND batch. Same feature, different platform tier. Cost: two code paths that must stay semantically identical, and networkx dies somewhere in the hundreds of thousands of edges.

Q: Biggest problem with your graph layer? (*volunteer this*) Two. First, the stream never writes edges — `upsert_transaction_edges` is only called by the demo seeder, so graph features are zero for every real streamed user. Second, `query_graph` runs synchronously per event against a cloud cluster, which is the most expensive thing on my hot path. Fixes: write edges from the stream (or from a CDC consumer off the same topic), and cache ring membership with a TTL since rings change hourly, not per-millisecond.

Q: Why not a GNN? It would probably win on raw detection — learning ring topology end-to-end beats three hand-crafted summary statistics. Three reasons not to, here: no labelled graph data; neighbour-sampling inference inside a millisecond budget is a hard problem; and explainability to an analyst and a regulator collapses. Hand-crafted graph features are the interpretable baseline you have to beat before the complexity is justified.

8.5 ML & fraud modelling

Q: Why IsolationForest first, then XGBoost? It mirrors the actual lifecycle. Day one you have no labels — unsupervised anomaly detection is the only honest option, and IsolationForest is well suited: it isolates points with short random partition paths, is $O(n \log n)$, handles high dimensions, and needs no distance metric. Once analyst labels accumulate (my switch is ≥ 50 positives), supervised is decisively better because it learns *fraud*, not merely *unusual*. The code picks automatically via `model_kind="auto"`.

Q: What's the difference between "anomalous" and "fraudulent"? This is the whole point. A surgeon buying a ₹4,00,000 watch is a five-sigma anomaly and perfectly legitimate. A ₹200 card-testing charge is statistically unremarkable and definitely fraud. Unsupervised methods find the first; only labels find the second. It's also why the explainability layer matters — the analyst supplies the context the model lacks.

Q: How do you handle class imbalance? Three levers. `scale_pos_weight = negatives/positives` reweights the gradient so the minority class isn't drowned. `eval_metric="aucpr"` optimises the precision-recall curve rather than error rate. And PR-AUC as the headline metric. What I *didn't* do: SMOTE (synthesising fraud samples in feature space is dubious when fraud is adversarial and multimodal — you invent plausible-looking frauds that don't exist), or undersampling negatives (throws away information and breaks calibration).

Q: Why PR-AUC over ROC-AUC? With 3% positives — and real fraud is under 1% — ROC-AUC is dominated by the huge negative class: you can score 0.95 while being useless in the region you care about. Precision-recall only involves the positive class and the false positives, so it reflects the actual operating question: "of the cases I send to an analyst, how many are real?"

Q: What metric would the business care about? Not PR-AUC. Money and capacity: fraud value caught per unit of analyst time; precision at the top-K the review team can actually process daily; false-positive rate translated into declined good customers; and net expected value against a cost matrix. My expected-loss ranking is a gesture at this — the threshold should be *derived* from it, not hand-set at 0.80.

Q: Your evaluation has a bug. Find it. (*volunteer it*) `evaluate()` runs on the training data. No holdout, no CV. So the metrics are optimistic and the promotion gate compares two optimistic numbers. The fix isn't just "add a split" — it's a **temporal** split, because fraud has time structure and a random split leaks future information into the training set.

Q: Your training data is synthetic. Doesn't that invalidate everything? It bounds what I can claim. It validates the *pipeline* — the schema contract, the training path, registration, promotion, hot-reload, drift, and the SHAP surface — and it produces meaningless accuracy numbers. Worse, it's the source of my train/serve skew: three features are informative in the synthetic frame and constant in production. The fix is to train from the bronze lake so training and serving share one feature computation.

Q: How would you set the decision threshold? Cost-based. Build a cost matrix — false negative costs the fraud amount plus chargeback fees; false positive costs the declined transaction's margin plus churn risk — then choose the threshold maximising expected value. Constrain it by review capacity: if analysts can process 500 cases a day, the threshold is whatever puts 500 cases in the queue, and then you optimise precision at that K. Also: per-segment thresholds, since the cost asymmetry differs between a ₹200 and a ₹2,00,000 transaction.

Q: Model calibration? Not addressed, and it matters — I threshold at 0.80 and rank by `score × amount`, both of which assume the score behaves like a probability. XGBoost's `predict_proba` is reasonably calibrated with log-loss but not guaranteed; my IsolationForest score is min-max normalised, which is **not a probability at all** — so the expected-loss ranking is only ordinally meaningful in that mode. Fix: Platt scaling or isotonic regression on a validation set, plus a reliability diagram to check.

8.6 MLOps

Q: How do you deploy a new model without downtime? The stream never references a version. It resolves `models:/fraud-scorer@Production` — an alias — and re-checks on a timer. It compares the resolved version to the loaded one and only pulls on change. So promotion is a registry operation: zero code deploy, zero restart, and the dashboard's model-version metric updates on the next tick. Rollback is re-pointing the alias.

Q: What's dangerous about that? Behaviour changes with no code change and no deploy gate. You need an audit trail on alias moves, an approval step, and ideally a canary — score a fraction of traffic with the challenger and compare before full promotion. Shadow scoring is the safest version: run both models on 100% of traffic, act on the champion only, and compare offline.

Q: Explain your promotion gate. Register the challenger, read the incumbent's `pr_auc` from the run behind the `Production` alias, and promote only if `challenger >= incumbent`. Architecturally right — this is what stops a fresh-but-worse model reaching production. Statistically wrong today: both numbers are in-sample, and `>=` promotes ties.

Q: Explain PSI. Population Stability Index: bin the reference distribution into deciles, compare bin proportions, and sum $(\text{actual\%} - \text{expected\%}) \times \ln(\text{actual\%}/\text{expected\%})$. It's a symmetrised KL divergence over bins. Conventional thresholds: `<0.1` stable, `0.1–0.25` moderate shift, `>0.25` significant. I use 0.2. It's the retail-credit-risk convention, which is exactly why I picked it — the number already means something to a risk team.

Q: PSI catches input drift. What about concept drift? Right, and that's the one that actually kills you: the input distribution can be stable while the *relationship* between features and fraud changes, because fraudsters adapt. Catching it needs labels, and my labels arrive delayed from analyst review. So you need a delayed-label evaluation harness — recompute PR-AUC on a rolling window as labels mature — and alert on performance decay, not just distribution shift.

Q: Your drift check uses synthetic data. Yes — it generates a reference frame and an artificially shifted one (higher fraud ratio, `amt_z + 1.5`) to model adversarial adaptation. It proves the PSI maths and the drift-triggers-retrain wiring. It does not monitor production. Real version: reference = the training window's feature distribution, current = last 24h of gold features from the lake, run on a schedule, alert on the per-feature PSI vector.

Q: What's missing from your MLOps story? Data versioning (models are versioned, training data isn't — DVC or Delta time-travel would fix it), orchestration (a Makefile a human runs, instead of Dagster/Airflow with retries and alerting), CI (all four quality gates exist, none are enforced on a PR), and production model monitoring (score-distribution and feature-drift dashboards off live traffic, not synthetic).

8.7 Explainability

Q: How does SHAP work, in one breath? Shapley values from cooperative game theory: a feature's attribution is its average marginal contribution to the prediction across all possible orderings of the features. That gives additivity — contributions sum to the difference between this prediction and the base value — plus consistency and local accuracy. TreeExplainer computes it exactly for tree ensembles in polynomial rather than exponential time, which is the only reason it's practical here.

Q: Why only explain non-LOW bands? Explanations are consumed by humans, and no human opens the 97% of transactions that pass. Computing SHAP for all of them buys nothing and spends the latency budget.

Q: Critique your SHAP implementation. Two things. It builds a fresh explainer on every call instead of caching one alongside the loaded model. And SHAP's TreeExplainer support for IsolationForest is unreliable, so in unsupervised mode it can silently fall into my magnitude-ranking fallback — which is *not* an attribution, it's just "which feature value is biggest", and it's mislabelled as SHAP output in the UI. That's an honesty bug worth fixing. Arguably explanation should move off the hot path entirely and be computed lazily when an analyst opens the case.

Q: What would an analyst actually want, beyond SHAP? A counterfactual: "this would not have flagged if the amount had been below ₹X" or "if the device were recognised". That's directly actionable, whereas an attribution needs

interpretation. DiCE-style counterfactuals plus the SHAP attribution is the strong combination, and the counterfactual is also the natural raw material for a regulatory adverse-action reason code.

8.8 LangChain, RAG & agents

Q: Why is the LLM off the hot path? Defend it hard. Four independent reasons, any one sufficient: latency (seconds against a millisecond budget), determinism (the same transaction must score identically twice — audit requirement), cost (per-call, on 100% of traffic, forever), and auditability (you cannot justify a declined authorisation to a regulator with a language model's opinion). The LLM's contribution is *analyst throughput*, not detection. It turns a case file into a paragraph. That reframing is the single most important design idea in the project.

Q: Walk me through your RAG chain. Format the transaction plus its SHAP attributions into a query string → embed it (MiniLM, 384-dim, or a hashing fallback) → retrieve top-3 from `case_embeddings` by cosine → format as a cited context block (`[CASE_017] (sim=0.91) ...`) → into a `ChatPromptTemplate` whose system message forbids unsupported claims and mandates bracketed citations → LLM → `StrOutputParser` → regex-extract the cited ids → return a `CaseSummary` with `summary`, `cited_case_ids`, `grounded`, `model`. LCEL composition: `RAG_PROMPT | llm | StrOutputParser()`.

Q: How do you prevent hallucination? Three layers, and I'd be clear that none of them is a guarantee. (1) Prompt constraints: assert only what SHAP/history/retrieved cases support, cite ids in brackets, say so explicitly when evidence is insufficient, stay under 120 words. (2) Structural: the model only sees retrieved context, and citations are *extracted by code* so grounding is an observable field. (3) Eval: the harness scores relevance and faithfulness. What's genuinely missing is claim-level entailment verification — checking each assertion against the retrieved text, which is where an LLM-as-judge belongs.

Q: Retrieved-citation extraction — what does it actually prove? That the model cited something, not that the citation supports the claim. It's a necessary condition, not a sufficient one. The honest upgrade is entailment checking per claim.

Q: Describe your agent and its tools. Three tools via the `@tool` decorator: `query_graph_tool` (Neo4j ring/shared-device info), `get_user_history_tool` (recent transactions from Mongo), `similar_cases_tool` (vector search). With a tool-calling-capable model it's `create_tool_calling_agent + AgentExecutor` with `max_iterations=6` and a system prompt requiring grounded, cited answers. With a model that can't tool-call — offline fake, or a local LoRA — it falls back to deterministic orchestration: call all three tools, apply an explicit ring rule (`shared_device_flags > 0` or `component_size > 2`), compose the answer. The capability check is `_supports_tool_calling()`.

Q: Why the deterministic fallback? Isn't that admitting the agent doesn't work? It's admitting that *tool calling is a model capability, not a framework feature*. `distilgpt2` cannot emit a tool call. Rather than have the feature vanish offline, I made the orchestration explicit — which, honestly, is *better* for a fixed three-step investigation: it's deterministic, auditable and cheap. The real lesson is that my "agent" doesn't need to be an agent for this task. If the investigation genuinely branched, `LangGraph` would be the right tool — my fallback is a hand-rolled state machine, and `LangGraph` is the principled version.

Q: `max_iterations=6` — why? A cost and latency bound, and a runaway-loop guard. Agents can ping-pong between tools indefinitely; six is enough for three tools plus a couple of refinements.

Q: Do you need LangChain at all? (a real senior-interviewer probe) For the chains alone, no — they're `prompt | llm | parse` and the raw SDK plus 30 lines would do it with fewer dependencies. What it actually bought me: the provider abstraction, which is what makes offline testing free and the fine-tuned-model swap a config change; the tool-calling agent scaffolding; and native `LangSmith` tracing. I'd keep it for those three and I'd be comfortable dropping it if they weren't needed. Knowing where an abstraction earns its keep is the point.

Q: How do you test LLM code without network calls? Dependency injection at the provider seam. Tests monkeypatch `chains.get_llm` to return a `FakeListLLM` with a fixed response, and monkeypatch `similar_cases` to return a known document. Then I assert on *structure*, not prose: is it a `CaseSummary`, is the `txn_id` right, was `CASE_017` extracted into `cited_case_ids`, is `grounded` `True` when context was retrieved. Deterministic, offline, fast.

Q: Chunking strategy? None — my documents are one to three sentences, so chunking would be harmful. At real scale (policy documents, historical case files) I'd need it, and the choice is semantic-boundary-aware chunking with overlap, plus retaining the parent document for context (small-to-big retrieval). Also: for case retrieval, hybrid BM25 + dense with reciprocal-rank fusion would very likely beat pure dense, because merchant names and BINs are exact-match signals.

8.9 LLMOps & evaluation

Q: What's the MLOps/LLMOps symmetry you keep mentioning? Deliberate parallel structure. Tracking: MLflow for the model, LangSmith for the LLM. Quality: PR-AUC/precision/recall/drift vs faithfulness/relevance/hallucination. Feedback: analyst labels → retrain vs analyst thumbs-up/thumbs-down → eval and fine-tune. The claim is that an LLM in production needs the same lifecycle discipline as a classifier — versioning, evaluation, monitoring, feedback — and the industry often skips it because prompts feel like configuration rather than models.

Q: Critique your eval metrics. They're smoke tests. `faithfulness = 1.0 if cited else 0.5` is a proxy for "did it cite anything", not "is it true". Relevance is keyword overlap against expected terms. Hallucination is a constant. What they *do* prove is that the harness runs offline and produces a score every time, so it can gate a change. Real evaluation: claim-level entailment against retrieved context, an LLM-as-judge with a written rubric — and critically, the judge validated against human labels before you trust it — plus a golden dataset with human-written reference summaries.

Q: How do you evaluate a RAG system properly? Split retrieval from generation. Retrieval: `recall@k`, `precision@k`, `MRR/NDCG` against known relevant documents. Generation: faithfulness (is every claim entailed by the context), answer relevance (does it address the question), context relevance (was the retrieved context useful at all). RAGAS packages these. Then end-to-end human preference on a golden set, and regression-test on every prompt change — prompt edits are code changes and deserve CI.

Q: LangSmith — what does it give you, and what did you get wrong? Traces every chain and agent run: prompt, retrieved context, tool calls, tokens, latency, cost, with project scoping. It's the observability layer for a non-deterministic system, and without it you're debugging by reading logs of prose. What I got wrong: feedback capture is incomplete — `create_feedback` needs a `run_id` and the dashboard never passes one, so thumbs-up/thumbs-down lands in local JSONL only. I'd capture the run id with a callback handler and thread it through.

Q: Compliance concern with LangSmith in a fraud system? (*volunteer this*) Prompt contents include transaction data, so tracing to a SaaS moves regulated data across a boundary. For a real deployment: self-host Langfuse or Phoenix, or redact/tokenise before the prompt, and gate it behind a DPA. The same argument applies to the LLM provider itself.

8.10 Fine-tuning

Q: Why fine-tune when you already have RAG? Different jobs. **RAG for knowledge, fine-tuning for form.** RAG injects facts the model doesn't have — this specific case, these precedents. Fine-tuning teaches *behaviour*: house style, length discipline, the hedging conventions a fraud team uses, the habit of citing. My training signal is analyst-approved summaries, which is a near-perfect encoding of "what good looks like here". You can chase house style with prompt engineering forever, or amortise it into weights and shorten every prompt.

Q: Walk through your fine-tune pipeline. Analyst clicks thumbs-up → `record_feedback` appends `{txn_id, summary, score, prompt, ts}` to JSONL. `build_dataset` filters to `score == 1` with a non-empty summary and emits chat-format JSONL: `system` = the same grounding rules, `user` = reconstructed prompt (`txn` + SHAP + history), `assistant` = the approved summary. Gate at ≥10 examples. Then either the OpenAI FT API (upload file → create job → poll to terminal state → print the `ft: id`) or local LoRA. Serving: `LLM_PROVIDER=finetuned + FINETUNED_MODEL_ID`, and `assist/llm.py` routes `ft: ids` to ChatOpenAI and local paths through `PeftModel` + `HuggingFacePipeline`. **The key property: not one chain changes.** Same seam.

Q: Explain LoRA. Freeze the base weights and inject trainable low-rank matrices into the attention projections: $W + BA$, where B is $d \times r$ and A is $r \times d$ with $r \ll d$. I use $r=8$, $\alpha=16$ (the scaling is α/r), `dropout=0.05`, `task_type=CAUSAL_LM`. You train well under 1% of the parameters, the adapter is megabytes not gigabytes, and you can host many task adapters over one shared base. Full fine-tuning needs several times the model size in memory for optimiser state and gives you a whole new model per task.

Q: Your fine-tune result is worthless. Why? (*pre-empt this*) Correct — `distilgpt2` is 82M parameters and the gate is ten examples. Three epochs of LoRA on ten samples of a model that small produces nothing useful. What's real is the *pipeline*: feedback capture, dataset construction, training, adapter persistence, and serving through the same interface. That's the transferable part. A real run needs a 7B+ base (QLoRA on one GPU) and low thousands of examples.

Q: You have preference data and you're doing SFT. What's the better method? DPO. thumbs-up/thumbs-down is *paired preference* data — supervised fine-tuning on thumbs-up only throws away the negative half of the signal, which is often the more informative half ("don't write it like this"). DPO optimises directly on preference pairs without a separate

reward model, which makes it the natural fit. Being able to name that gap is more valuable than having built it.

Q: How do you know the fine-tuned model is better? Run `make eval-llm` before and after and compare faithfulness/relevance — accepting that those metrics are proxies. Properly: a held-out set of cases with human-preferred reference summaries, blind side-by-side analyst preference, and a regression check that grounding didn't degrade (fine-tuning on style can absolutely erode citation discipline). Fine-tuning without an eval harness is guessing, which is why the harness exists at all.

8.11 Python & software engineering

Q: Why Pydantic for everything? One schema module (`src/fraud/schemas.py`) is simultaneously the Kafka wire format, the Mongo document shape, and the internal type. Validation at the boundary is what makes the DLQ path meaningful — I know exactly where bad data is rejected. Specific v2 features used: `Field(ge=0)` for the amount constraint, a `field_validator(mode="before")` to accept ISO-8601 with `Z`, `alias="_id"` with `populate_by_name` to bridge Python naming and Mongo's `_id`, computed properties (`expected_loss`, `amount_std`), and `model_dump(by_alias=True, mode="json")` for serialisation.

Q: Why `lru_cache` on `get_settings()`? It makes `Settings` a singleton, so the `.env` is parsed and validated once rather than on every import, and every module observes the same values. Cost: it's genuinely awkward in tests — which is why the tests monkeypatch attributes on the settings object instead of re-constructing it.

Q: Why `structlog`? Key-value events (`log.info("flagged", txn=..., band=..., latency_ms=...)`) rather than interpolated strings. That means logs are machine-parseable and queryable in aggregate — "p95 latency for HIGH-band events" is a query, not a regex. For a pipeline you'll ship to a log aggregator that's the right default.

Q: What's your testing philosophy here? Test the logic, not the infrastructure. Everything worth asserting lives in pure functions, so the tests need no broker and no database: feature engineering (velocity windows, z-score growth, non-mutation, haversine), schema contracts (alias round-trip, validation rejects a negative amount, feature-vector/name alignment), scoring (band thresholds, monotonicity of the heuristic, range bounds), LLM chains (injected `FakeListLLM`, assert structure not prose), and the fine-tune dataset export (thumbs-up filter, message roles, insufficient-data path). What's missing: integration tests with testcontainers for Kafka and Mongo, and property-based tests via Hypothesis for the feature functions — where I'd expect to find real bugs.

Q: Lazy imports — why are `import mlflow`, `import shap`, `import networkx` inside functions? Import cost and optional dependencies. `mlflow`, `shap` and `torch` are heavy; importing them at module scope would slow every process that imports the module, including the ones that never need them. It also means an optional extra (`.[finetune]`) can be absent without breaking the import graph. Cost: an `ImportError` surfaces at call time rather than start-up, which is worse for fail-fast.

Q: Where would you improve the code quality? `mypy` runs with `disallow_untyped_defs = false`, so typing is aspirational rather than enforced. There's a scatter of `except Exception with noqa: BLE001` — defensible for the graceful-degradation goal, but it means real bugs can be logged as warnings and swallowed; they should be narrowed to expected exception types. And the quality gates aren't enforced by CI, so they only run when someone remembers.

8.12 System design & fraud domain

Q: A transaction arrives. Walk me through every step to a decision. Producer keys the event by `user_id` and publishes JSON to `raw-transactions`. The consumer polls, Pydantic-validates (failure → DLQ, commit, continue), loads that user's rolling state from Mongo, computes features purely (multi-horizon velocity, Welford z-score, geo/device novelty, seconds-since-last), enriches with graph features from Neo4j (ring id, shared-device count, component size), scores with the MLflow Production model (heuristic if absent), maps the score to a band via configured thresholds, computes SHAP top-5 if the band isn't LOW, upserts the scored case, upserts the new user state, upserts an alert if not LOW, appends the raw event to bronze — **then** commits the offset. The dashboard picks it up on a 3-second poll and ranks by expected loss.

Q: How do you keep the false-positive rate acceptable? Four levers. Banding rather than a binary decision — MEDIUM goes to review, HIGH gets held, so you're not declining on uncertainty. Expected-loss ranking so analyst time goes where the money is. A cost-derived threshold instead of a hand-picked 0.80. And a feedback loop: every analyst label is training data, so the model learns *this* institution's false positives. What I'd add: per-segment thresholds, and

reason codes so a declined customer can be recovered.

Q: Real-time vs batch — which parts and why? Real-time: anything on the authorisation path — feature computation, scoring, alerting. The decision is worthless late. Batch: training, drift detection, community detection on the graph, lake compaction, fine-tuning. These are expensive, tolerate hours of staleness, and benefit from seeing all the data at once. The interesting boundary is graph features — currently queried in real time, but ring membership only changes on the order of hours, so it *should* be batch-computed and read locally. That's a mis-assignment in my current design and I know it.

Q: How do you detect an entirely new fraud pattern? Supervised models can't, by construction — they've never seen it. That's precisely why the unsupervised path stays in the system rather than being a temporary bootstrap: IsolationForest flags "weird" without needing a label. Plus drift detection as an early warning that inputs have shifted, graph analytics to surface emergent clusters, and the analyst as the ultimate novelty detector — which is exactly why the LLM copilot exists, to make each analyst faster so more novel cases get looked at.

Q: What if the model starts flagging 40% of traffic? Alert on flag rate as a first-class SLO — it's the fastest signal that something broke. Triage: is it a real attack (check the graph for a new ring, look at merchant/geo concentration), a data-quality break (a feature computing to null or a constant — most likely an upstream schema change), or a bad promotion (check `model_version` against the alias history)? Immediate mitigation: re-point the `Production` alias to the previous version — that's seconds, no deploy — or raise the threshold to protect review capacity while you diagnose. This is the strongest argument for alias-based deployment.

Q: How would you A/B test a new model? Not A/B first — **shadow** first. Score 100% of traffic with both, act only on the champion, compare offline. That's free of customer risk and gives you a paired comparison on identical traffic. Then canary on a small traffic slice with guardrail metrics (flag rate, precision on reviewed cases, decline rate) and automatic rollback. True A/B on fraud is genuinely hard because outcomes are delayed by the chargeback window and treatment affects the labels you observe.

9. The scale-up whiteboard question

"This handles 20 transactions per second. Take it to 50,000."

Answer in this order — the order itself demonstrates seniority. Do not jump to "add Kafka partitions"; start by naming the bottleneck.

- 1. Name the bottleneck first.** It isn't CPU. It's the per-event network calls: a Mongo `find_one` plus two-to-three upserts plus a Neo4j round trip — 4–5 sequential hops per transaction. At 50k/s that's 200k+ round trips per second against remote systems. Everything else follows from removing them.
- 2. Move state next to the compute.** Migrate the stateful stage to Flink or Kafka Streams with RocksDB keyed state and changelog-topic backing. Feature computation becomes a local memory read: no hop, and correct-by-construction under rebalance because partition ownership *is* the lock. This single change is most of the win.
- 3. Get the graph off the hot path.** Precompute ring membership in the cold path and materialise it into the local state store or Redis, refreshed every few minutes. Ring membership changes hourly; querying it per event was always a mis-assignment.
- 4. Make writes asynchronous and batched.** Don't write to Mongo from the scoring path. Emit scored events to an output Kafka topic and let a separate sink consumer batch-upsert with bulk writes. Scoring throughput decouples from database throughput, and the sink can be scaled, retried and backpressured independently.
- 5. Then scale horizontally.** Partitions to ~200+ (rough target: throughput ÷ per-consumer throughput, times a headroom factor), consumers to match, autoscaled on consumer lag. Handle hot keys with a composite salted key for the top-N whale accounts.
- 6. Fix the model path.** Batch inference (score micro-batches of 100–1000 rows — tree ensembles vectorise extremely well), keep the model in-process rather than behind an RPC, and move SHAP entirely off the hot path: compute it lazily when an analyst opens a case, since nobody reads 97% of explanations.
- 7. Storage tiering.** Mongo (or Cassandra/DynamoDB at that write volume) for hot serving with a TTL of days; the lake for history; a columnar store (ClickHouse) for analyst analytics. Don't ask one store to serve point reads, full scans and aggregations.
- 8. Observability that survives the scale.** Prometheus histograms for scoring latency, counters for events/DLQ/flag-rate, consumer-lag alerting, and SLOs — p99 scoring latency, lag under N seconds, flag rate within a band. At 50k/s you cannot debug from logs; you need percentiles and traces.
- 9. Cost.** At that volume the LLM is a line item you must bound: it's per-case rather than per-transaction, so cap it by review capacity, cache summaries per case, and consider a smaller/self-hosted model for the common path with escalation to a frontier model for hard cases.

10. Numbers & facts cheat sheet

Config defaults (all in `config/settings.py`, all env-overridable):

Setting	Value
Kafka bootstrap / raw topic / DLQ	localhost:19092 / raw-transactions / raw-transactions-dlq
Partitions	6 (raw), 1 (DLQ)
Consumer group	fraud-stream, auto.offset.reset=earliest, enable.auto.commit=false
Producer	acks=all, enable.idempotence=true, linger.ms=20
Risk thresholds	HIGH $\geq$ 0.80, MEDIUM $\geq$ 0.50, else LOW
Model reload interval	60 s
MLflow model / alias	fraud-scorer @ Production
Embedding model / dim	all-MiniLM-L6-v2 / 384
PSI threshold	0.2
LoRA	r=8, alpha=16, dropout=0.05, 3 epochs, lr 2e-4, batch 2
Fine-tune gate	$\geq$ 10 thumbs-up examples
Base model (local FT)	distilgpt2
Producer rate / fraud ratio	20 events/s / 2%
Training frame	8,000 rows, 3% fraud
XGBoost	300 trees, depth 5, lr 0.1, scale_pos_weight=neg/pos, eval_metric=aucpr
IsolationForest	200 estimators, contamination 0.03
Feature vector	11 features
SHAP	top 5, non-LOW bands only
Agent	3 tools, max_iterations=6
Dashboard refresh	3 s (st.cache_data(ttl=3))
Bounded state	200 recent txns (1h), 100 seen geos, 100 seen devices

The 11 features: `amt_z`, `txn_1m`, `txn_5m`, `txn_1h`, `distinct_merchants_1h`, `distinct_geos_1h`, `seconds_since_last_new_geo`, `new_device`, `shared_device_flags`, `component_size`.

Mongo collections: `flagged_transactions` (`_id=txn_id`), `user_rolling_state` (`_id=user_id`), `alerts` (`_id=txn_id`), `case_embeddings` (`_id=case id`).

Test coverage: 5 files — `test_features.py` (5 tests), `test_schemas.py` (4), `test_scorer.py` (3), `test_chains.py` (2), `test_finetune.py` (2). Last recorded pytest run: no failures.

Get real latency numbers before your interview (10 minutes, high ROI)

`stream/job.py` already collects per-event latencies in `_latencies` and never reports them. Add a periodic percentile log:

```
import statistics
def _record_latency(ms: float) -> None:
    _latencies.append(ms)
    if len(_latencies) % 500 == 0:
        s = sorted(_latencies[-500:])
        log.info("latency_percentiles",
                p50=round(statistics.median(s), 1),
                p95=round(s[int(len(s) * 0.95)], 1),
                p99=round(s[int(len(s) * 0.99)], 1))
    if len(_latencies) > 5000:
        del _latencies[:1000]
```

Then run `make up` && `make seed` && `make train` && `make producer` && `make stream` and note p50/p95/p99 in two configurations: **with** `NEO4J_URI` set and **without**. The delta is the cost of your per-event cloud call — which turns your best-known weakness into a *quantified* one. "Removing the synchronous graph lookup cuts p99 from X to Y" is a dramatically stronger sentence than "I think the graph call is slow." Also capture throughput by raising the producer rate until consumer lag starts growing; that's your measured ceiling.

11. Behavioural / STAR stories

Each is a real decision from this codebase. Rehearse them out loud — 60–90 seconds each.

"Tell me about a difficult technical decision." → *Where the LLM belongs*. **S/T**: Everyone expects an LLM to be central to an "AI fraud project"; I had to decide whether it participates in the scoring decision. **A**: I mapped the requirements per plane and found four hard conflicts — latency, determinism, cost, auditability — and concluded the LLM's value was in analyst throughput, not detection. So I architecturally forbade it from the hot path and gave it its own plane with its own SLA and its own observability stack. **R**: Scoring stays deterministic and sub-second and is fully auditable, while analysts get a cited summary on demand. The constraint became the project's clearest idea. **Learning**: The interesting question about a new technology is usually *where it doesn't belong*.

"Tell me about a trade-off you made and regret / would revisit." → *Mongo for hot state*. I used the durable store I already had rather than adding Redis, to keep the system to three services. It works, and it's my throughput ceiling — 3–4 network round trips per event — plus a lost-update window during rebalance. If I revisited it, keyed local state in Flink or Kafka Streams is the right answer, because it removes the hop entirely rather than making it faster. Learning: "one fewer system" is a real benefit and it isn't free; I should have priced the per-event round trip explicitly at design time instead of discovering it later.

"Tell me about a bug you found in your own work." → *Train/serve skew*. Reviewing the feature code I noticed `distinct_merchants_1h` is computed as `len({event.merchant})` — always 1 — while the training generator draws it from a Poisson. Then I found the same class of problem with the two graph features, which are always zero in production because the stream never writes graph edges. Three of eleven features are informative in training and constant at inference. It's the textbook train/serve skew failure, and finding it in my own code taught me why feature stores exist: not as convenience tooling, but to make that bug structurally impossible.

"Tell me about designing for failure." → *Graceful degradation*. I set a constraint that the system must run end-to-end with zero credentials, then worked through every external dependency and gave each a named failure mode: heuristic scorer, zeroed graph features, fake LLM, hashing embedder, brute-force vector search, networkx instead of GDS. Result: `git clone` to running demo with no accounts, and full offline tests. The unexpected benefit was the forcing function — you can't write a fallback without first deciding what your system *means* when a dependency is missing. The cost is that failures became quiet, which is why the console surfaces model version, LLM provider and Neo4j status.

"Tell me about how you verify correctness." → *The chaos drill*. I claimed replay safety, and a claim you don't test is a guess. `make chaos` publishes N known transaction ids, starts the consumer, kills it mid-stream, restarts it, and asserts every id persisted. It caught the real question: what exactly does it prove? It proves no loss and idempotency. It does *not* prove exactly-once, because the upsert makes duplicate processing invisible rather than absent. Being precise about what a test proves is as important as having one.

"Tell me about scope you cut." → *Streamlit over React + FastAPI*. The UI's purpose was to prove the human-in-the-loop cycle closes: triage → explain → label → retrain. Streamlit did that in about 200 lines, so the time went into the pipeline. I named the cost explicitly: no auth, full re-render polling, single-user assumptions — which is why I list it as a known gap rather than a feature. Cutting scope is only defensible if you can state what the cut costs.

12. Questions you ask them

Pick three or four. Each one signals something specific about you.

1. "How do you handle the label-delay problem — chargebacks arriving 30–90 days after the decision? Do you train on delayed labels, or use a proxy?" *(signals: you know real fraud ML)*
2. "Where does your stateful stream processing state live — a remote store, or keyed local state in Flink or Kafka Streams?" *(signals: you know the actual bottleneck in this class of system)*
3. "Do you run rules and ML together, and who owns the rules layer — engineering or the fraud team?" *(signals: you know production fraud systems are hybrid, and that it's an org question)*
4. "How is your decision threshold chosen — a cost matrix, or review capacity?" *(signals: you think about business metrics, not just PR-AUC)*
5. "How do you deploy a new model — is it a code deploy, or a registry operation? Do you shadow or canary?" *(signals: MLOps maturity)*
6. "Do you have a feature store, and if not, how do you prevent training-serving skew?" *(signals: you've hit the problem yourself)*
7. "Are you using LLMs anywhere in the fraud workflow, and if so, is it on the decision path or the analyst path?" *(signals: you have a considered position, and invites your best story)*
8. "How do you measure whether the analyst review queue is well-calibrated — precision at the capacity you can actually staff?" *(signals: you think about the humans in the loop)*
9. "What does your on-call look like for the streaming pipeline, and what's the most common page?" *(signals: you expect to operate what you build)*

13. 48-hour prep plan

Hour 1–2 — Make it run and get real numbers. make up && make seed && make train && make producer && make stream && make dashboard. Add the percentile logging from §10 and record p50/p95/p99 with and without Neo4j configured. Note your throughput ceiling. **Replace every "I think" in your talk track with a measured number.**

Hour 3 — Rehearse the pitches aloud, timed. 30 s, 90 s, 3 min. Out loud, not in your head. The 90-second one is the one you'll actually use; it should be smooth enough to survive an interruption.

Hour 4 — Practise the walkthrough with the file map (§3). Narrate the path of one transaction from producer to dashboard, naming files. Then do it again backwards from the dashboard. Naming files is a credibility signal that's very hard to fake.

Hour 4B — Drill the four lead exhibits (§3B: 5, 7, 8, 15). For each, be able to reproduce the *shape* of the code on a whiteboard from memory — not character-perfect, but the structure and the one load-bearing line — and deliver the *What to say* bullets without reading them. Then skim the other sixteen exhibits once so nothing in the codebase can surprise you. If you're allowed to bring materials, bring §3B; if you're screen-sharing, have it open in a second window.

Hour 5 — Memorise the top three gaps and your fixes (§7 #1, #2, #4). Rehearse volunteering them: *"Three things I'd fix before calling this production-ready..."* This is the highest-leverage five minutes in this guide.

Hour 6 — Drill the alternatives table (§5). Cover the "why not here" column and reconstruct it. You must be able to answer "why not Flink / why not Redis / why not a GNN / why not Avro / why not just the raw OpenAI SDK" in two sentences each, without hesitating.

Hour 7 — Whiteboard the scale-up (§9) from memory, on paper. Bottleneck first. Practise resisting the urge to open with "add partitions."

Hour 8 — The maths you might be asked to derive. Welford's recurrence. PSI's formula and thresholds. Why PR-AUC beats ROC-AUC under imbalance. What a Shapley value is. What LoRA's $w + BA$ decomposition means and why α/r is the scaling.

Hour 9 — Rehearse the STAR stories (§11) out loud. 60–90 seconds each, and make sure each one ends with a *learning*, not just an outcome.

Hour 10 — Adversarial self-interview. Read §7 and, for each gap, ask *"why didn't you do it properly?"* — then answer in two sentences without defensiveness. The pattern that works: **acknowledge** → **explain the constraint** → **name the fix**. Never "I ran out of time" alone; always "I prioritised X over Y, and here's exactly what Y would take."

Three lines to keep in your pocket

- **When you don't know something:** "I haven't worked with that directly. My closest analogue here is X — is the mechanism similar?" (*curiosity beats bluffing, every time*)
- **When they find a flaw you know about:** "Yes — that's one I'd flagged. Here's the failure mode and here's the fix." (*then give it concretely*)
- **When they find a flaw you don't know about:** "That's a good catch — walk me through it? ... You're right. The fix would be Z." (*fast, unfussy agreement reads as senior; arguing a losing point does not*)

Last thing. The strongest signal you can send in this interview is not knowing every answer — it's demonstrating that you evaluate your own work the way a senior engineer would. You built a system with a genuinely good architectural idea (three planes, LLM off the hot path, alias-driven deployment) and some real, findable flaws (train/serve skew, in-sample evaluation, a cloud call on the hot path). Own both halves. Candidates who can articulate exactly where their own work is weak, and why, and what it would cost to fix, are rare — and they are the ones who get hired.